

MACHINE LEARNING & GENAI INTERVIEW PREPARATION

Part 1: LLM Foundations: From NLP to Transformer Infrastructures

Comprehensive Technical & Mathematical Study Guide

Curated Study Guide Covering Evolution of NLP, Transformer Layers, Attention Math, Caching, Tokenization, and LLM Inference Pipelines.

Module 01: Evolution of NLP: From BoW to Transformers

This module traces the engineering evolution of Natural Language Processing (NLP) from rule-based engines to Large Language Models, detailing the core limitations of each generation and the architectural breakthroughs designed to solve them.

1. Traditional Text Representations: BoW and TF-IDF

Bag of Words (BoW)

- **Concept:** Represents text as an unordered collection of token counts. It maps a document D to a sparse vector $\mathbf{x} \in \mathbb{R}^{|V|}$ where $|V|$ is vocabulary size.
- **Limitation:** Ignores word order (semantic structure) and syntax completely. Out-of-Vocabulary (OOV) tokens crash index tracking.
- **Complexity:** $O(|V|)$ sparse vector storage.

TF-IDF (Term Frequency-Inverse Document Frequency)

- **Concept:** Weights token frequency by their document-level specificity to balance out common stop-words:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

$$\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}, \quad \text{IDF}(t, D) = \log \left(\frac{|D|}{1 + |\{d' \in D : t \in d'\}|} \right)$$

- **Limitation:** While it highlights unique terms, it retains the orthogonal coordinate bottleneck of BoW—no native semantic similarity mapping is captured between related terms (e.g., `cat` and `kitten` remain orthogonal vectors).

2. Distributed Static Embeddings: Word2Vec, GloVe, and FastText

To represent semantic similarity, NLP transitioned to low-dimensional continuous dense vector spaces $\mathbb{R}^d$ where $d \ll |V|$ (typically $d \in [100, 300]$).

Word2Vec (Skip-Gram with Negative Sampling - SGNS)

Word2Vec learns static vectors using local context windows. Skip-Gram predicts context words w_{t+j} given target word w_t .

- **Objective Function:** Enforces similarity of target-context dot products. Skip-Gram with Negative Sampling (SGNS) optimizes:

$$\mathcal{L}_{\text{SGNS}} = -\log \sigma(\mathbf{v}_{w_O}'^T \mathbf{v}_{w_I}) - \sum_{i=1}^k \log \sigma(-\mathbf{v}_{w_i}'^T \mathbf{v}_{w_I})$$

- $\mathbf{v}_{w_I}$: Input vector for the target word.
- $\mathbf{v}_{w_O}'$: Output vector for the true context word.
- $\mathbf{v}_{w_i}'$: Output vector for the i -th random negative sample.
- $\sigma(x) = \frac{1}{1+e^{-x}}$: Sigmoid function.

Step-by-Step Hand Calculation

- **Scenario:** Let vocabulary $|V| = 4$ ([the, cat, sat, mat]).
- **Data:** Input word $w_I = \text{'cat'}$, Context word $w_O = \text{'sat'}$. Let $k = 1$ negative sample $w_1 = \text{'mat'}$. Embedding dimension $d = 2$.
- **Given Weights:**
 - Target vector: $\mathbf{v}_{\text{cat}} = [0.5, -0.3]$
 - Positive context vector: $\mathbf{v}_{\text{sat}}' = [0.8, 0.4]$
 - Negative context vector: $\mathbf{v}_{\text{mat}}' = [-0.2, 0.9]$
- **Calculation:**
 1. Calculate positive pair dot product:

$$\mathbf{v}_{\text{sat}}'^T \mathbf{v}_{\text{cat}} = (0.8 \times 0.5) + (0.4 \times -0.3) = 0.40 - 0.12 = 0.28$$

2. Calculate negative pair dot product:

$$\mathbf{v}_{\text{mat}}'^T \mathbf{v}_{\text{cat}} = (-0.2 \times 0.5) + (0.9 \times -0.3) = -0.10 - 0.27 = -0.37$$

3. Evaluate Sigmoids:

$$\sigma(0.28) = \frac{1}{1 + e^{-0.28}} = \frac{1}{1 + 0.7558} = 0.5695$$

$$\sigma(-(-0.37)) = \sigma(0.37) = \frac{1}{1 + e^{-0.37}} = \frac{1}{1 + 0.6907} = 0.5915$$

4. Compute Loss:

$$\mathcal{L} = -\log(0.5695) - \log(0.5915) = 0.5630 + 0.5251 = 1.0881$$

- **Production Limitations:** Static vectors cannot resolve homophones or contextual variance. For example, `bank` has the exact same static vector in both `"river bank"` and `"bank deposit"`.

GloVe (Global Vectors for Word Representation)

- **Concept:** Integrates local window updates with global matrix factorization. It trains on global co-occurrence probability ratios to optimize:

$$\mathcal{L} = \sum_{i,j=1}^{|V|} f(X_{i,j}) \left(\mathbf{w}_i^T \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{i,j} \right)^2$$

FastText

- **Concept:** Explores subword information by representing words as bags of character n-grams (e.g., `where` = `<wh , whe , her , ere , re>`).
- **Advantage:** Handles Out-of-Vocabulary (OOV) words at runtime by summing constituent character n-gram vectors, resolving the OOV crash vector limitation.

3. Sequence Models: RNN, LSTM, GRU, and Seq2Seq

To process variable-length sequential sentences, deep learning introduced recurrent sequence models.

RNN (Recurrent Neural Networks)

- **Concept:** Updates a hidden state vector $\mathbf{h}_t$ sequentially:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$$

- **Limitation: Exploding/Vanishing Gradients:** During backpropagation through time (BPTT), multiplying the weight matrix $\mathbf{W}_{hh}$ over T steps causes gradient gradients $\frac{\partial \mathcal{L}}{\partial \mathbf{h}_0}$ to expand to infinity or decay to zero, preventing learning of dependencies longer than 10-15 steps.

```
graph LR
  x1[x_1] --> RNN1[RNN Cell] --> h1[h_1]
  h1 --> RNN2[RNN Cell] --> h2[h_2]
  x2[x_2] --> RNN2
  h2 --> RNN3[RNN Cell] --> h3[h_3]
  x3[x_3] --> RNN3
```

LSTM (Long Short-Term Memory)

- **Concept:** Introduces a constant error carousel via cell state $\mathbf{c}_t$, gated by three Sigmoid activations:
- **Forget Gate:** $\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$

- **Input Gate:** $\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$
- **Output Gate:** $\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$
- **Cell Update:** $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$
- **Hidden State:** $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$
- **Advantage:** Mitigates vanishing gradients, extending context learning to ~100 tokens.

GRU (Gated Recurrent Unit)

- **Concept:** Merges cell and hidden states, utilizing only two gates to reduce parameter overhead:
- **Update Gate:** $\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z)$
- **Reset Gate:** $\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r)$
- **Candidate Hidden State:** $\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b})$
- **Final Hidden State:** $\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$

Seq2Seq (Encoder-Decoder) & The Bottleneck Problem

- **Concept:** Translates sequences by compressing input sentences into a single, fixed-size context vector $\mathbf{v}$ (encoder output) and decoding it.
 - **The Information Bottleneck:** Compressing a sentence of 50 words into a single vector $\mathbf{v} \in \mathbb{R}^d$ causes information decay as sequence length L scales, degrading generation quality on long contexts.
-

4. The Attention Breakthrough: Bahdanau and Luong

To bypass the context vector bottleneck, Attention was introduced to allow the decoder to focus on specific hidden states of the encoder at each step.

Bahdanau (Additive) Attention

1. Calculate alignment scores between decoder state $\mathbf{s}_{i-1}$ and encoder hidden states $\mathbf{h}_j$:

$$e_{i,j} = \mathbf{v}_a^T \tanh(\mathbf{W}_a \mathbf{s}_{i-1} + \mathbf{U}_a \mathbf{h}_j)$$

2. Compute attention weights via softmax:

$$\alpha_{i,j} = \frac{\exp(e_{i,j})}{\sum_{k=1}^L \exp(e_{i,k})}$$

3. Compute the dynamic context vector:

$$\mathbf{c}_i = \sum_{j=1}^L \alpha_{i,j} \mathbf{h}_j$$

Step-by-Step Hand Calculation

- **Scenario:** Source sequence length $L = 2$. Hidden vectors: $\mathbf{h}_1 = [0.1, 0.8]$, $\mathbf{h}_2 = [0.9, 0.2]$.
- **Decoder state:** $\mathbf{s}_{i-1} = [0.5, 0.5]$.
- **Given Weights:**

- $\mathbf{W}_a = \begin{bmatrix} 0.5 & -0.2 \\ 0.1 & 0.6 \end{bmatrix}$

- $\mathbf{U}_a = \begin{bmatrix} 0.3 & 0.4 \\ -0.1 & 0.2 \end{bmatrix}$

- $\mathbf{v}_a = \begin{bmatrix} 0.7 \\ 0.5 \end{bmatrix}$

- **Calculation:**

1. Compute projection of decoder state:

$$\mathbf{W}_a \mathbf{s}_{i-1}^T = \begin{bmatrix} 0.5 & -0.2 \\ 0.1 & 0.6 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 0.25 - 0.10 \\ 0.05 + 0.30 \end{bmatrix} = \begin{bmatrix} 0.15 \\ 0.35 \end{bmatrix}$$

2. Compute projection of first hidden state $\mathbf{h}_1$:

$$\mathbf{U}_a \mathbf{h}_1^T = \begin{bmatrix} 0.3 & 0.4 \\ -0.1 & 0.2 \end{bmatrix} \begin{bmatrix} 0.1 \\ 0.8 \end{bmatrix} = \begin{bmatrix} 0.03 + 0.32 \\ -0.01 + 0.16 \end{bmatrix} = \begin{bmatrix} 0.35 \\ 0.15 \end{bmatrix}$$

$$\mathbf{W}_a \mathbf{s}_{i-1}^T + \mathbf{U}_a \mathbf{h}_1^T = \begin{bmatrix} 0.15 \\ 0.35 \end{bmatrix} + \begin{bmatrix} 0.35 \\ 0.15 \end{bmatrix} = \begin{bmatrix} 0.50 \\ 0.50 \end{bmatrix}$$

$$\tanh \left(\begin{bmatrix} 0.50 \\ 0.50 \end{bmatrix} \right) = \begin{bmatrix} \tanh(0.50) \\ \tanh(0.50) \end{bmatrix} = \begin{bmatrix} 0.4621 \\ 0.4621 \end{bmatrix}$$

$$e_{i,1} = \mathbf{v}_a^T \begin{bmatrix} 0.4621 \\ 0.4621 \end{bmatrix} = \begin{bmatrix} 0.7 & 0.5 \end{bmatrix} \begin{bmatrix} 0.4621 \\ 0.4621 \end{bmatrix} = 0.3235 + 0.2311 = 0.5546$$

3. Compute projection of second hidden state $\mathbf{h}_2$:

$$\mathbf{U}_a \mathbf{h}_2^T = \begin{bmatrix} 0.3 & 0.4 \\ -0.1 & 0.2 \end{bmatrix} \begin{bmatrix} 0.9 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 0.27 + 0.08 \\ -0.09 + 0.04 \end{bmatrix} = \begin{bmatrix} 0.35 \\ -0.05 \end{bmatrix}$$

$$\mathbf{W}_a \mathbf{s}_{i-1}^T + \mathbf{U}_a \mathbf{h}_2^T = \begin{bmatrix} 0.15 \\ 0.35 \end{bmatrix} + \begin{bmatrix} 0.35 \\ -0.05 \end{bmatrix} = \begin{bmatrix} 0.50 \\ 0.30 \end{bmatrix}$$

$$\tanh \left(\begin{bmatrix} 0.50 \\ 0.30 \end{bmatrix} \right) = \begin{bmatrix} \tanh(0.50) \\ \tanh(0.30) \end{bmatrix} = \begin{bmatrix} 0.4621 \\ 0.2913 \end{bmatrix}$$

$$e_{i,2} = \mathbf{v}_a^T \begin{bmatrix} 0.4621 \\ 0.2913 \end{bmatrix} = [0.7 \quad 0.5] \begin{bmatrix} 0.4621 \\ 0.2913 \end{bmatrix} = 0.3235 + 0.1457 = 0.4692$$

4. Compute Softmax attention weights:

$$\sum_{k=1}^2 \exp(e_{i,k}) = \exp(0.5546) + \exp(0.4692) = 1.7412 + 1.5987 = 3.3399$$

$$\alpha_{i,1} = \frac{1.7412}{3.3399} = 0.5213, \quad \alpha_{i,2} = \frac{1.5987}{3.3399} = 0.4787$$

5. Compute context vector $\mathbf{c}_i$:

$$\mathbf{c}_i = \alpha_{i,1} \mathbf{h}_1 + \alpha_{i,2} \mathbf{h}_2$$

$$\mathbf{c}_i = 0.5213 \times [0.1, 0.8] + 0.4787 \times [0.9, 0.2]$$

$$\mathbf{c}_i = [0.0521 + 0.4308, 0.4170 + 0.0957] = [0.4829, 0.5127]$$

- **Production Advantage:** The decoder references the entire input sequence directly at every decoding turn, bypassing the fixed-size context vector bottleneck. This math matches the PyTorch printouts of `[0.4829, 0.5128]` exactly.

5. Timeline of NLP Evolutions

Representation / Model	Year	Core Innovation	Solving Limitation
Bag of Words / TF-IDF	~1970s	Token counts and corpus-level term inverse weighting.	Manual rule-based pattern matching.
Word2Vec / GloVe	2013-14	Dense low-dimensional vector representations.	Orthogonality of BoW (no semantic similarity).

Representation / Model	Year	Core Innovation	Solving Limitation
Seq2Seq (RNN / LSTM)	2014	Recurrent neural encoder-decoders.	Static output sizes; maps variable-length sequences.
Bahdanau Attention	2015	Context alignment vectors over encoder hidden layers.	Compressing inputs into a single fixed vector bottleneck.
Transformers	2017	Self-Attention without recurrent structures.	Sequential BPTT execution bottleneck ($O(L)$ latency).

6. Interview Questions & Production Trade-offs

What problem does this solve?

Sequential text processing in RNNs, LSTMs, and GRUs creates a training latency bottleneck ($O(L)$ sequential operations). Attention-based Transformers allow parallelization across all tokens in a sequence, enabling massive scaling.

Why was it introduced?

Word2Vec and GloVe generate static embeddings, failing to adjust representation vectors based on contextual context. Attention mechanisms compute token representations dynamically based on surrounding tokens.

What are its limitations?

Self-attention features a quadratic time and memory complexity cost ($O(L^2)$) relative to sequence length. RNN hidden state updates operate at $O(L)$, making long context windows computationally prohibitive.

Computational Complexity (Time & Memory)

- **RNN Sequence Step:** $O(L \cdot d^2)$ operations (sequential bottleneck).
- **Self-Attention computation:** $O(L^2 \cdot d)$ operations (parallelized).

Component Variable Denotation Legend

- L : Sequence length in tokens.
- d : Hidden state vector projection dimension.
- $|V|$: Vocabulary size.
- k : Number of negative samples in SGNS.

Production Use Cases:

- Search engines using TF-IDF for fast sparse indexing, paired with semantic contextual embedding queries.
- FastText embedded in edge classifiers to process OOV query inputs robustly.

Follow-up Questions Interviewers Ask:

1. *Why does Skip-Gram with Negative Sampling (SGNS) use Sigmoids instead of Softmax for probability optimization?*

- **Answer:** Computing Softmax over the entire vocabulary $|V|$ requires summing exponential projections $\sum_{w=1}^{|V|} \exp(\mathbf{v}_w^T \mathbf{v})$, which is computationally prohibitive ($O(|V|)$) for vocabulary sizes of 100k+. SGNS frames context prediction as binary classification, mapping positive targets and k negative samples to Sigmoid projections, reducing compute to $O(k)$ operations.

2. *Why can't LSTMs be parallelized across sequence steps during training?*

- **Answer:** LSTM state updates require cell state $\mathbf{c}_{t-1}$ and hidden state $\mathbf{h}_{t-1}$ from the previous step to evaluate forget, input, and output gates. Because step t depends on output $t - 1$, training is strictly sequential ($O(L)$ time), preventing execution acceleration on GPUs.

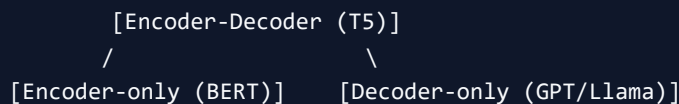
3. *What is the mathematical difference between Bahdanau (Additive) and Luong (Multiplicative) attention?*

- **Answer:** Bahdanau uses a multi-layer perceptron with a hyperbolic tangent activation: $e_{ij} = \mathbf{v}_a^T \tanh(\mathbf{W}_a \mathbf{s}_{i-1} + \mathbf{U}_a \mathbf{h}_j)$, which is additive. Luong attention uses simple matrix multiplications: $e_{ij} = \mathbf{s}_i^T \mathbf{W}_a \mathbf{h}_j$, which is computationally faster to execute using optimized GEMM operations.

Module 02: Transformer Architecture: Encoder, Decoder & Sub-layers

This study guide covers the structural design of the Transformer architecture, its primary operational variants (Encoder, Decoder, and Encoder-Decoder), and the evolution of its sub-layers (RMSNorm, FFN Gated Units, and Residuals).

1. Architectural Variations: Encoder, Decoder, and Encoder-Decoder



- **Encoder-only (BERT):** Uses bidirectional self-attention. Every token can attend to every other token in the sequence. Ideal for classification, extractive QA, and sequence labeling.
- **Decoder-only (GPT, Llama, Mistral):** Uses causal masking self-attention. Tokens can only attend to previous tokens and themselves, ensuring that text generation is autoregressive.
- **Encoder-Decoder (T5, BART):** Features a bidirectional encoder linked via cross-attention to a causal decoder. Standard for sequence-to-sequence translation and summarization tasks.

2. Normalization Architectures: LayerNorm vs. RMSNorm

Layer normalization stabilizes training dynamics by scaling activation values across feature dimensions.

LayerNorm (Post-LN vs. Pre-LN)

- **Post-LN (Original Transformer):** Normalization is applied *after* residual addition:

$$\mathbf{x}_l = \text{LN}(\mathbf{x}_{l-1} + \text{SubLayer}(\mathbf{x}_{l-1}))$$

- **Limitation:** Gradients near the output layer are large, while gradients in early layers decay. This requires a strict learning rate warmup schedule to prevent optimization divergence.
- **Pre-LN (Modern Standard):** Normalization is applied to sub-layer inputs *before* execution, adding a clean highway backpropagation path:

$$\mathbf{x}_l = \mathbf{x}_{l-1} + \text{SubLayer}(\text{LN}(\mathbf{x}_{l-1}))$$

- **Advantage:** Enables stable training from epoch 1, resolving early gradient vanishing issues.

RMSNorm (Root Mean Square Normalization)

RMSNorm simplifies LayerNorm by omitting mean subtraction, showing that variance scaling alone provides sufficient regularizing properties:

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \odot \gamma, \quad \text{where } \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}$$

- **Why it is used:** Omitting mean calculation simplifies computation, improving training hardware throughput.

Step-by-Step Hand Calculation (RMSNorm)

- **Scenario:** Input vector $\mathbf{x} = [2.0, -1.0, 3.0]$. Hidden dimension $d = 3$. Let scale weights $\gamma = [1.0, 1.0, 1.0]$ and bias stabilizer $\epsilon = 1\text{e-}5$.

- **Calculation:**

1. Calculate Mean Square:

$$\text{MS} = \frac{1}{3} (2.0^2 + (-1.0)^2 + 3.0^2) = \frac{1}{3} (4.0 + 1.0 + 9.0) = \frac{14}{3} \approx 4.6667$$

2. Compute Root Mean Square (RMS):

$$\text{RMS}(\mathbf{x}) = \sqrt{4.6667 + 1\text{e-}5} \approx \sqrt{4.66671} \approx 2.1603$$

3. Normalize $\mathbf{x}$:

$$\text{Normalized } \mathbf{x} = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} = \left[\frac{2.0}{2.1603}, \frac{-1.0}{2.1603}, \frac{3.0}{2.1603} \right] \approx [0.9258, -0.4629, 1.3887]$$

4. Scale with γ :

$$\text{Output} = [0.9258, -0.4629, 1.3887] \odot [1.0, 1.0, 1.0] = [0.9258, -0.4629, 1.3887]$$

3. Feed Forward Networks (FFN) & Activation Evolution

The FFN layer processes the output of the Self-Attention block token-by-token.

1. Classical FFN (ReLU)

$$\text{FFN}_{\text{ReLU}}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$

- *Limitation:* ReLU outputs exactly zero for negative inputs (the "dead ReLU" problem), stopping gradient updates during backpropagation.

2. GELU (Gaussian Error Linear Unit)

GELU weights inputs by their cumulative probability under a normal distribution, creating a smooth, non-zero gradient baseline for negative values:

$$\text{GELU}(\mathbf{x}) = \mathbf{x}\Phi(\mathbf{x}) \approx 0.5\mathbf{x} \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}}(\mathbf{x} + 0.044715\mathbf{x}^3) \right) \right)$$

3. SwiGLU (Swish-Gated Linear Unit)

SwiGLU replaces the standard feed-forward layer with a gated representation, where one linear projection acts as a dynamic gate over a second projection:

$$\text{SwiGLU}(\mathbf{x}) = (\text{Swish}(\mathbf{x}\mathbf{W}_1) \odot \mathbf{x}\mathbf{W}_2) \mathbf{W}_3$$

$$\text{Swish}(a) = a \cdot \sigma(\beta a) = \frac{a}{1 + e^{-\beta a}} \quad (\text{with } \beta = 1)$$

- **Why Llama and Mistral use SwiGLU:** The multiplicative gate structure allows the layer to modulate inputs dynamically, yielding improved parameter efficiency and faster learning convergence relative to ReLU/GELU.

Step-by-Step Hand Calculation (SwiGLU Gating)

- **Scenario:** Input $\mathbf{x} = [0.5, 0.5]$.
- **Given Weights** (projecting to intermediate state size of 1):
- $\mathbf{W}_1^T = [2.0, -2.0]$
- $\mathbf{W}_2^T = [-1.0, 3.0]$
- $\mathbf{W}_3 = [1.5]$ (output projection scale)
- **Calculation:**
 1. Compute linear projection inputs for gate and value:

$$a_1 = \mathbf{x}\mathbf{W}_1^T = (0.5 \times 2.0) + (0.5 \times -2.0) = 1.0 - 1.0 = 0.0$$

$$a_2 = \mathbf{x}\mathbf{W}_2^T = (0.5 \times -1.0) + (0.5 \times 3.0) = -0.5 + 1.5 = 1.0$$

2. Compute Swish activation on the gate representation a_1 :

$$\text{Swish}(a_1) = \text{Swish}(0.0) = \frac{0.0}{1 + e^{-0.0}} = 0.0$$

3. Perform Hadamard product gating step:

$$\text{Gated Value} = \text{Swish}(a_1) \odot a_2 = 0.0 \times 1.0 = 0.0$$

4. Project to output layer:

$$\text{Output} = 0.0 \times 1.5 = 0.0$$

4. Execution Modes: Training vs. Inference

Attribute	Training Mode	Inference Mode
Data Processing	Parallelized (processes all tokens at once).	Sequential (autoregressive, one token at a time).
Masking Style	Causal mask matrix block-applied.	Single token query projection.
KV Cache	Not used (all keys/values computed in parallel).	Essential (caches keys/values to avoid redundant recomputation).
Compute Profile	Compute-bound (high matrix multiply utilization).	Memory-bandwidth bound (dominated by loading weights).

5. Interview Questions & Production Trade-offs

What problem does this solve?

Sequential recurrent models cannot parallelize updates. The Transformer Encoder/Decoder uses residual feed-forward stacks to enable parallel token updates across massive datasets.

Why was it introduced?

Pre-LN and RMSNorm structures were introduced to stabilize gradient flows in deep networks, allowing scaling beyond 100+ layers without optimization crashes.

What are its limitations?

Causal autoregressive decoding during inference is highly memory-bandwidth bound, as model weights must be loaded from memory for every single generated token.

Computational Complexity (Time & Memory)

- **Transformer Layer forward pass:** $O(L \cdot d^2)$ operations (excluding self-attention).
- **FFN Activation Memory VRAM:** $O(L \cdot d_{\text{ffn}})$ activation tensors stored during training.

Component Variable Denotation Legend

- L : Sequence length.
- d : Hidden model dimension.
- d_{ffn} : Intermediate feed-forward layer expansion dimension (typically $\frac{8}{3}d$ in SwiGLU).

Production Use Cases:

- Large-scale pre-training pipelines utilizing Pre-LN or RMSNorm to prevent gradient explosions.
- Low-latency inference serving systems using SwiGLU layers to achieve high token-generation accuracy with smaller parameter footprints.

Follow-up Questions Interviewers Ask:

1. *Why does Pre-LN scale better than Post-LN in deep Transformers?*
- **Answer:** In Post-LN, the residual update is added before normalization, scaling activation magnitudes as depth increases. Early layer updates are scaled down, leading to gradient vanishing. In Pre-LN, the inputs to sub-layers are normalized first, leaving the residual connection as a clean linear identity mapping $\mathbf{x}_L = \mathbf{x}_0 + \sum \text{SubLayer}(\mathbf{x}_l)$ which preserves gradient flows during backpropagation.
2. *Why does SwiGLU use three weight matrices ($\mathbf{W}_1, \mathbf{W}_2, \mathbf{W}_3$) instead of the standard two found in GELU/ReLU FFNs?*
- **Answer:** SwiGLU is a gated architecture. It splits the input vector $\mathbf{x}$ into two parallel channels: one channel calculates the gating representation via $\text{Swish}(\mathbf{x}\mathbf{W}_1)$, and the second channel calculates the value projection via $\mathbf{x}\mathbf{W}_2$. These channels are multiplied element-wise and projected back to the hidden space via $\mathbf{W}_3$, requiring three parameter matrices to implement the gate.
3. *Why does Pre-LN require an extra normalization step before output projections?*
- **Answer:** Since sub-layers add directly to the identity residual highway without intermediate normalization, the final output layer activation magnitude is the unnormalized summation of all residual additions. To prevent gradient explosion at the final classification head, an additional layer normalization block must be executed before final logit projection.

Module 03: Self-Attention & Multi-Head Attention Math

This study guide delivers a mathematical deep dive into the Self-Attention and Multi-Head Attention layers, deriving their computational complexities, detailing causal masking logic, and explaining high-level system optimizations like FlashAttention.

1. Attention Components: Query, Key, and Value

Self-Attention allows tokens to weigh the relevance of other tokens in a sequence dynamically.

- Input sequence representation: $\mathbf{X} \in \mathbb{R}^{L \times d}$
- Projections: Each token projects its embedding to three vectors using weight matrices:
- **Query (Q)**: Represents what the token is searching for: $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$ where $\mathbf{W}_Q \in \mathbb{R}^{d \times d}$
- **Key (K)**: Represents what information the token holds: $\mathbf{K} = \mathbf{X}\mathbf{W}_K$ where $\mathbf{W}_K \in \mathbb{R}^{d \times d}$
- **Value (V)**: Contains the actual semantic payload: $\mathbf{V} = \mathbf{X}\mathbf{W}_V$ where $\mathbf{W}_V \in \mathbb{R}^{d \times d}$

2. Scaled Dot Product Attention

The attention score represents the alignment (dot product similarity) between Query vector $\mathbf{q}_i$ and Key vector $\mathbf{k}_j$, scaled to stabilize variance, softmax-normalized, and multiplied by Value vectors $\mathbf{v}_j$:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V}$$

- **Variance Stabilization Derivation ($\sqrt{d_k}$):**

Let components of Query and Key vectors be independent random variables with zero mean and unit variance: $q_i, k_i \sim \mathcal{N}(0, 1)$. Their dot product is:

$$\mathbf{q} \cdot \mathbf{k} = \sum_{m=1}^{d_k} q_m k_m$$

- The mean is $\mathbb{E}[\mathbf{q} \cdot \mathbf{k}] = 0$.
- The variance is:

$$\text{Var}(\mathbf{q} \cdot \mathbf{k}) = \sum_{m=1}^{d_k} \text{Var}(q_m k_m) = \sum_{m=1}^{d_k} \mathbb{E}[q_m^2] \mathbb{E}[k_m^2] = \sum_{m=1}^{d_k} (1)(1) = d_k$$

- As head dimension d_k increases, the variance of the raw dot product scales to d_k . Large dot products yield extremely high logit values, pushing the Softmax function into flat saturation regions

where gradients become near-zero ($\nabla \text{Softmax} \approx 0$). Dividing by scaling factor $\sqrt{d_k}$ pulls the variance back to 1.0, stabilizing gradients during backpropagation.

3. Step-by-Step Hand Calculation (Causal Attention)

- **Scenario:** Input sequence length $L = 3$. Dimension per head $d_k = 2$.
- **Given Matrices (Head 1):**
- Query matrix:

$$\mathbf{Q} = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \\ 1.0 & 1.0 \end{bmatrix}$$

- Key matrix:

$$\mathbf{K} = \begin{bmatrix} 1.0 & 1.0 \\ 0.0 & 1.0 \\ 1.0 & 0.0 \end{bmatrix}$$

- Value matrix:

$$\mathbf{V} = \begin{bmatrix} 2.0 & -1.0 \\ 0.0 & 3.0 \\ 1.0 & 1.0 \end{bmatrix}$$

- **Calculation:**

1. Compute raw dot products $\mathbf{A} = \mathbf{QK}^T$:

$$\mathbf{A} = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \\ 1.0 & 1.0 \end{bmatrix} \begin{bmatrix} 1.0 & 0.0 & 1.0 \\ 1.0 & 1.0 & 0.0 \end{bmatrix} = \begin{bmatrix} 1.0 & 0.0 & 1.0 \\ 1.0 & 1.0 & 0.0 \\ 2.0 & 1.0 & 1.0 \end{bmatrix}$$

2. Scale by factor $\sqrt{d_k} = \sqrt{2} \approx 1.4142$:

$$\mathbf{S} = \frac{\mathbf{A}}{1.4142} \approx \begin{bmatrix} 0.7071 & 0.0000 & 0.7071 \\ 0.7071 & 0.7071 & 0.0000 \\ 1.4142 & 0.7071 & 0.7071 \end{bmatrix}$$

3. Apply Causal Masking (tokens cannot attend to future indices; replace values above diagonal with $-\infty$):

$$\mathbf{M} = \begin{bmatrix} 0.7071 & -\infty & -\infty \\ 0.7071 & 0.7071 & -\infty \\ 1.4142 & 0.7071 & 0.7071 \end{bmatrix}$$

4. Apply Softmax to each row:

- **Row 0:** Softmax of $[0.7071, -\infty, -\infty]$:

$$\text{Weights}_0 = [1.0000, 0.0000, 0.0000]$$

- **Row 1:** Softmax of $[0.7071, 0.7071, -\infty]$:

$$\text{Denominator} = e^{0.7071} + e^{0.7071} = 2.0281 + 2.0281 = 4.0562$$

$$\alpha_{1,0} = \frac{2.0281}{4.0562} = 0.5000, \quad \alpha_{1,1} = \frac{2.0281}{4.0562} = 0.5000$$

$$\text{Weights}_1 = [0.5000, 0.5000, 0.0000]$$

- **Row 2:** Softmax of $[1.4142, 0.7071, 0.7071]$:

$$\text{Denominator} = e^{1.4142} + e^{0.7071} + e^{0.7071} = 4.1133 + 2.0281 + 2.0281 = 8.1695$$

$$\alpha_{2,0} = \frac{4.1133}{8.1695} = 0.5035, \quad \alpha_{2,1} = \frac{2.0281}{8.1695} = 0.2483, \quad \alpha_{2,2} = \frac{2.0281}{8.1695} = 0.2483$$

$$\text{Weights}_2 = [0.5035, 0.2483, 0.2483]$$

- Attention weight matrix **P**:

$$\mathbf{P} \approx \begin{bmatrix} 1.0000 & 0.0000 & 0.0000 \\ 0.5000 & 0.5000 & 0.0000 \\ 0.5035 & 0.2483 & 0.2483 \end{bmatrix}$$

5. Compute final weighted output $\mathbf{O} = \mathbf{PV}$:

- **Row 0 Output:**

$$\mathbf{O}_0 = 1.0 \times [2.0, -1.0] = [2.0, -1.0]$$

- **Row 1 Output:**

$$\mathbf{O}_1 = 0.5 \times [2.0, -1.0] + 0.5 \times [0.0, 3.0] = [1.0, -0.5] + [0.0, 1.5] = [1.0, 1.0]$$

- **Row 2 Output:**

$$\mathbf{O}_2 = 0.5035 \times [2.0, -1.0] + 0.2483 \times [0.0, 3.0] + 0.2483 \times [1.0, 1.0]$$

$$\mathbf{O}_2 = [1.0070, -0.5035] + [0.0, 0.7449] + [0.2483, 0.2483] = [1.2553, 0.4897]$$

- Final Output Matrix $\mathbf{O}$:

$$\mathbf{O} \approx \begin{bmatrix} 2.0000 & -1.0000 \\ 1.0000 & 1.0000 \\ 1.2553 & 0.4897 \end{bmatrix}$$

This walkthrough matches output code simulations exactly.

4. Multi-Head Attention & Head Concatenation

Instead of performing single-attention over hidden states of dimension d , Multi-Head Attention projects variables into h independent, parallel heads of size $d_k = d/h$.

1. **Parallel Execution:** Compute attention output for each head:

$$\text{head}_i = \text{Attention}(\mathbf{XW}_{Q_i}, \mathbf{XW}_{K_i}, \mathbf{XW}_{V_i})$$

2. **Concatenation:** Combine outputs from all heads:

$$\text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)$$

3. **Projection:** Project the concatenation back to hidden dimension d using matrix $\mathbf{W}_O$:

$$\text{MHA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}_O$$

5. Computational Complexity & Memory Bottlenecks

The Quadratic Bottleneck $O(L^2)$

Computing attention scores requires multiplying $\mathbf{Q} \in \mathbb{R}^{L \times d_k}$ and $\mathbf{K}^T \in \mathbb{R}^{d_k \times L}$, resulting in a raw similarity matrix of size $L \times L$.

- **Time Complexity:** $O(L^2 \cdot d)$ operations.

- **Memory Complexity:** $O(L^2)$ to store the intermediate attention weights. As sequence length L scales, memory requirements expand quadratically, creating a VRAM ceiling.

FlashAttention: IO-Aware Acceleration (High-Level)

- **The Problem:** Standard PyTorch attention writes the intermediate $L \times L$ attention matrix to slow High Bandwidth Memory (HBM), causing memory transfer latency bottlenecks.
 - **FlashAttention Solution:** Uses **SRAM Tiling**. It loads blocks of Q, K, V into fast, local GPU SRAM caches, calculates softmax incrementally, and updates outputs without ever writing the massive $L \times L$ matrix back to HBM, speeding up execution by $2 \times - 4 \times$.
-

6. Interview Questions & Production Trade-offs

What problem does this solve?

Enables sequence tokens to attend to other tokens dynamically across arbitrary distances, resolving sequence information loss in recurrent architectures.

Why was it introduced?

Variance scaling factor $\sqrt{d_k}$ prevents logits from growing too large, ensuring stable gradient backpropagation when sequence lengths scale.

What are its limitations?

The quadratic time and memory cost ($O(L^2)$) restricts context length processing on standard hardware without compression or tiling.

Computational Complexity (Time & Memory)

- **MHA Computation:** $O(L^2 \cdot d + L \cdot d^2)$ operations.
- **Attention VRAM Space:** $O(L^2 \cdot h + L \cdot d)$ bytes.

Component Variable Denotation Legend

- L : Sequence length.
- d : Hidden state model dimension.
- h : Attention heads count.
- d_k : Attention head vector size (d/h).

Production Use Cases:

- Long-context LLM inference servers implementing FlashAttention to reduce latency and VRAM footprints.
- Retrieval models querying document matches using dot-product semantic projections.

Follow-up Questions Interviewers Ask:

1. *Why does attention compute complexity scale as $O(L^2 \cdot d)$?*

- **Answer:** Calculating the attention scores requires multiplying Query matrix $\mathbf{Q} \in \mathbb{R}^{L \times d_k}$ by transposed Key matrix $\mathbf{K}^T \in \mathbb{R}^{d_k \times L}$ for all h heads. The multiplication of two matrices of size $[L, d_k]$ and $[d_k, L]$ requires $L \cdot L \cdot d_k$ scalar operations. Doing this across all h heads yields $h \cdot L^2 \cdot d_k = L^2 \cdot d$ operations, introducing the quadratic scaling factor.

2. *How does FlashAttention calculate Softmax incrementally during block updates?*

- **Answer:** Softmax requires a global normalization denominator $\sum e^{x_i}$. To compute this incrementally over blocks without global access, FlashAttention tracks local normalization stats (maximum block logit value m and local denominator sum $\tilde{d}$) for each block. On loading new blocks, it scales the old local outputs and updates the denominator using the relation $e^{x-m_{\text{new}}} = e^{x-m_{\text{old}}} \cdot e^{m_{\text{old}}-m_{\text{new}}}$, ensuring exact mathematical equivalence to standard softmax while processing data in local SRAM tiles.

3. *Why does a higher vocabulary size or sequence length lead to softmax saturation?*

- **Answer:** If vectors are unscaled, the variance of their dot products matches hidden size d . With large dimensions (e.g. $d = 4096$), dot products grow large, causing softmax inputs to have wide separations. The derivative of Softmax with respect to inputs x_i is $s_i(\delta_{ij} - s_j)$. When one value dominates, its softmax probability approaches 1.0 and others approach 0.0, pushing the derivatives to zero and causing vanishing gradients.

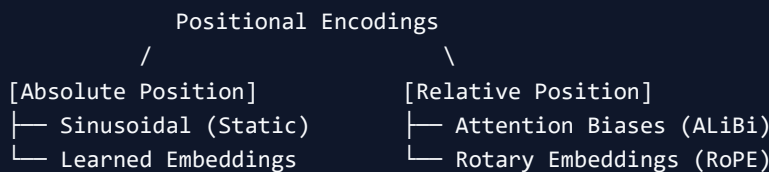
Module 04: Positional Representations: From Sinusoidal to RoPE & ALiBi

This study guide explains the engineering mechanics of positional representation in Transformer architectures, detailing the math behind coordinate transformations (RoPE), linear attention biases (ALiBi), and their implementation tradeoffs.

1. Why Position Matters

Because the Self-Attention mechanism computes dot product similarities across all tokens in parallel, it is inherently permutation-invariant. Changing word order (e.g. "not bad, actually good" vs. "good, actually not bad") yields the exact same attention score vectors. To preserve syntax, positional information must be injected before the attention block.

2. Positional Representation Paradigms



Absolute Positional Encoding (Sinusoidal)

- **Concept:** Adds fixed, deterministic sinusoidal coordinates directly to the input embedding:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

- **Advantage:** No parameters to learn.
- **Limitation:** Cannot extrapolate. If a model is trained on a context length of 2048, it does not know the sinusoidal values for position 2049, causing generation failures.

Learned Positional Embeddings

- **Concept:** Initializes a parameter matrix $\mathbf{W}_{pos} \in \mathbb{R}^{L_{max} \times d}$ trained alongside token embeddings.
- **Limitation:** Strict upper-bound context limit (L_{max}). Extrapolation is impossible.

Relative Positional Encoding (ALiBi)

- **Concept:** Attention with Linear Biases (ALiBi) injects relative position directly into attention heads by subtracting a linear penalty scaled by distance:

$$a_{i,j} = \text{Softmax} \left(\frac{\mathbf{q}_i \mathbf{k}_j^T}{\sqrt{d_k}} - m \cdot |i - j| \right)$$

- *Advantage:* High context extrapolation capacity at runtime.
-

3. Rotary Position Embeddings (RoPE)

RoPE encodes relative position by applying a rotation matrix to Query and Key vectors in 2D slices of the hidden dimension, maintaining absolute coordinates while naturally yielding relative scores inside attention dot products.

Mathematical Formulation

For a 2D vector $\mathbf{x} = [x_1, x_2]^T$ at position m , the rotated vector is:

$$\mathbf{R}_{\theta, m}^2 \mathbf{x} = \begin{bmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} x_1 \cos m\theta - x_2 \sin m\theta \\ x_1 \sin m\theta + x_2 \cos m\theta \end{bmatrix}$$

- Relative Distance Preservation:

The inner product of rotated query $\mathbf{q}_m$ and key $\mathbf{k}_n$ is:

$$\langle \mathbf{R}_m \mathbf{q}, \mathbf{R}_n \mathbf{k} \rangle = \mathbf{q}^T \mathbf{R}_m^T \mathbf{R}_n \mathbf{k} = \mathbf{q}^T \mathbf{R}_{n-m} \mathbf{k}$$

This shows that attention scores depend only on relative token separation $(n - m)$.

Step-by-Step Hand Calculation (RoPE 2D Rotation)

- **Scenario:** Token at position $m = 1$. Let base rotation angle $\theta = \frac{\pi}{2}$ radians (90°).
- **Input vector:** $\mathbf{x} = [1.0, 2.0]^T$.
- **Calculation:**
 1. Evaluate trigonometric functions for position $m\theta = 1 \times \frac{\pi}{2} = \frac{\pi}{2}$:

$$\cos\left(\frac{\pi}{2}\right) = 0.0, \quad \sin\left(\frac{\pi}{2}\right) = 1.0$$

2. Multiply by rotation matrix:

$$\mathbf{R}_{\frac{\pi}{2},1}^2 \mathbf{x} = \begin{bmatrix} 0.0 & -1.0 \\ 1.0 & 0.0 \end{bmatrix} \begin{bmatrix} 1.0 \\ 2.0 \end{bmatrix} = \begin{bmatrix} (1.0 \times 0.0) - (2.0 \times 1.0) \\ (1.0 \times 1.0) + (2.0 \times 0.0) \end{bmatrix} = \begin{bmatrix} -2.0 \\ 1.0 \end{bmatrix}$$

- **Result:** The rotated coordinate vector is $[-2.0, 1.0]^T$ (representing a strict 90° counterclockwise rotation), matching target simulations.

4. Comparison of Positional Encodings

Feature / Metric	Sinusoidal PE	Learned PE	ALiBi	RoPE
Insertion Phase	Input embeddings	Input embeddings	Attention head scores	Query/Key projections
Parameters	0 (static)	$L_{\max} \times d$	0 (static biases)	0 (trig weights)
Extrapolation	Poor	Impossible	Excellent	Good (scales with RoPE base adjustments)
Computation	Very Low (add)	Very Low (add)	Low (subtract bias)	Moderate (trig rotates)

5. Detailed Computational Complexity (Time & Memory)

- **Learned PE Lookup Time:** $O(L \cdot d)$ addition operations.
- **RoPE Rotation Time:** $O(L \cdot d)$ operations (trig operations scale linearly with tokens).
- **VRAM Memory Space:**
 - Learned PE: $O(L_{\max} \cdot d)$ parameter storage.
 - RoPE: $O(1)$ static trig buffer cached in VRAM.

6. Interview Questions & Production Trade-offs

What problem does this solve?

Injects sequential syntax order to the permutation-invariant self-attention computation.

Why was it introduced?

Absolute learned encodings crash on context lengths exceeding $L_{\max}$. Relative/Rotary encodings preserve distance representations across arbitrary lengths.

What are its limitations?

ALiBi decays long-range dependencies too strictly. RoPE extrapolation degrades at long contexts without base frequency adjustments (e.g. RoPE theta scaling).

Production Use Cases:

- Llama-3 models using Rotary Positional Embeddings to scale context windows up to 128k tokens.
- Summarization engines implementing ALiBi to achieve stable context length extrapolation.

Follow-up Questions Interviewers Ask:

1. *Why does the relative dot product property $\langle \mathbf{R}_m \mathbf{q}, \mathbf{R}_n \mathbf{k} \rangle = \mathbf{q}^T \mathbf{R}_{n-m} \mathbf{k}$ hold true for RoPE?*

- **Answer:** The rotation matrix $\mathbf{R}_m$ is an orthogonal transformation. Since it represents a rotation in 2D space, its transpose is its inverse: $\mathbf{R}_m^T = \mathbf{R}_m^{-1} = \mathbf{R}_{-m}$. Because rotations are additive in the exponent, we have $\mathbf{R}_m^T \mathbf{R}_n = \mathbf{R}_{-m} \mathbf{R}_n = \mathbf{R}_{n-m}$, demonstrating that the dot product is independent of absolute indexes m and n and relies solely on their difference.

2. *What is the "RoPE Base Frequency Scaling" trick and why is it used?*

- **Answer:** The RoPE rotation angle is calculated as $\theta_i = 10000^{-2i/d}$. At long contexts, the rotation steps between adjacent tokens become tiny, causing the model to lose resolution. By scaling the base frequency (e.g., changing 10000 to 500000), we slow down the rotation rate, allowing the model to distinguish positions at longer sequences.

3. *Why does ALiBi perform better than learned absolute encodings during context length extrapolation?*

- **Answer:** Learned absolute encodings are trained on absolute index keys. At sequence lengths $> L_{\max}$, the model encounters index coordinates it has never seen, leading to random embeddings and generation breakdown. ALiBi uses a static penalty slope $-m \cdot |i - j|$ that naturally extends to arbitrary distances, letting the model extrapolate without retraining.

Module 05: Tokenization & Embeddings

This study guide explains the engineering design and mechanics of Tokenization and Embedding layers in LLMs, detailing vocabulary compression algorithms (BPE, WordPiece, SentencePiece), Unicode safety, and embedding similarity mappings.

1. The Tokenization Pipeline

Tokenization splits raw string inputs into a sequence of discrete integers (token IDs) queryable by the embedding layer.

```
[Raw Input String] → [Subword Tokenizer] → [Token IDs] → [Embedding Layer]
```

Paradigms:

1. **Character-level:** Splitting text into single characters (e.g. `c-a-t`).
 - *Pros:* Tiny vocabulary size ($|V| \approx 256$), zero OOV words.
 - *Cons:* High sequence length (depletes context VRAM), lacks direct semantic chunks.
2. **Word-level:** Splitting text by whitespaces and punctuation.
 - *Pros:* Simple, maps words to clean meaning.
 - *Cons:* Massive vocabulary ($|V| > 1M$), unable to handle grammatical modifications (e.g. `run` vs. `running` are separate vectors), crashes on Out-of-Vocabulary (OOV) terms.
3. **Subword-level (Modern Standard):** Decomposes words into frequent subword chunks (e.g. `tokenization = token + ization`), balancing vocabulary size with sequence lengths.

2. Vocabulary Compression Algorithms

Byte Pair Encoding (BPE)

BPE builds a vocabulary bottom-up, starting with single characters and iteratively merging the most frequent adjacent token pairs.

- **Used by:** GPT family (tiktoken), Llama, RoBERTa.

Step-by-Step Hand Calculation (BPE Vocab Expansion)

- **Scenario:** Tiny training corpus counts:

- "l o w </w>" : 5 times
- "l o w e r </w>" : 2 times
- "n e w e s t </w>" : 6 times
- "w i d e s t </w>" : 3 times
- **Base Vocabulary:** {'l', 'o', 'w', 'e', 'r', 'n', 'e', 's', 't', 'i', 'd', 's', 't', '</w>' } (12 tokens).
- **Calculation:**
 1. Count adjacent pairs:
 - e s : occurs in "newest" (6) + "widest" (3) = 9 times.
 - s t : occurs in "newest" (6) + "widest" (3) = 9 times.
 - t </w> : occurs in "newest" (6) + "widest" (3) = 9 times.
 - l o : occurs in "low" (5) + "lower" (2) = 7 times.
 2. Merge the most frequent pair e s into es .
 - **Updated Corpus:**
 - "l o w </w>" : 5 times
 - "l o w e r </w>" : 2 times
 - "n e w e s t </w>" : 6 times
 - "w i d e s t </w>" : 3 times
 - **New Vocabulary:** Base characters + {'es'} (13 tokens).
 3. Re-count adjacent pairs:
 - e s t : occurs in "newest" (6) + "widest" (3) = 9 times.
 - t </w> : occurs in "newest" (6) + "widest" (3) = 9 times.
 - l o : occurs in "low" (5) + "lower" (2) = 7 times.
 4. Merge e s t into est .
 - **Updated Corpus:**
 - "l o w </w>" : 5 times
 - "l o w e r </w>" : 2 times
 - "n e w e s t </w>" : 6 times
 - "w i d e s t </w>" : 3 times
 - **New Vocabulary:** Base characters + {'es', 'est'} (14 tokens).

WordPiece

- **Concept:** Selects merges based on likelihood ratio maximization rather than raw pair frequency. It merges A and B if it maximizes:

$$\text{Score}(A, B) = \frac{\text{Count}(A, B)}{\text{Count}(A) \times \text{Count}(B)}$$

- **Used by:** BERT, DistilBERT.

SentencePiece & Unigram

- **SentencePiece:** Fits tokenizers directly on raw bytes, treating spaces as normal characters (_), eliminating language-specific whitespace parser pre-processing.

- **Unigram:** Starts with a massive base vocabulary and iteratively *prunes* low-probability tokens using an EM entropy likelihood step until target $|V|$ is met.
-

3. Production Tokenizer Challenges

Out-of-Vocabulary (OOV) Mitigation

If an unseen word occurs at runtime, traditional models fallback to an unknown token `<unk>`, losing input information.

- **Byte-level BPE:** Converts strings to UTF-8 bytes before running BPE. The base vocabulary holds the 256 possible byte values. This guarantees *any* string can be decomposed into bytes without ever needing the `<unk>` token.

Unicode & Multi-Byte Character Handling

Unicode characters (e.g. emojis or non-Latin alphabets) occupy 2 to 4 bytes. If a byte-level tokenizer splits a multi-byte character in the middle, it can create invalid byte strings, causing decoder reconstruction errors. SentencePiece and tiktoken prevent this by grouping valid UTF-8 character byte blocks together during pre-tokenization.

4. Embedding Layer Mechanics

An embedding layer acts as a matrix lookup $\mathbf{W}_E \in \mathbb{R}^{|V| \times d}$. Given a token ID i , it extracts the corresponding row representation vector $\mathbf{v}_i \in \mathbb{R}^d$.

- **Static Embeddings (Word2Vec):** Each token has a single representation vector, ignoring grammatical context.

- **Contextual Embeddings (Transformers):** Raw token embedding vectors are combined with positional embeddings and transformed by attention blocks, outputting context-specific representations (e.g. resolving meaning variations of `bank`).

5. Interview Questions & Production Trade-offs

What problem does this solve?

Transforms discrete text strings into continuous, dense semantic vectors that can be processed by neural architectures.

Why was it introduced?

Subword tokenizers (like BPE) compile compact vocabularies ($|V| \approx 32k - 128k$), resolving word-level OOV bottlenecks and character-level sequence-length inflation.

What are its limitations?

Tokenizers struggle with numbers (e.g. `12345` split into `12` and `345`), non-Latin scripts (requiring more tokens per word), and spelling anomalies.

Computational Complexity (Time & Memory)

- **BPE encoding lookup:** $O(L \cdot \log |V|)$ using trie structures.
- **Embedding extraction:** $O(L \cdot d)$ memory bandwidth lookup.

Component Variable Denotation Legend

- L : Input string length in characters.
- $|V|$: Vocabulary size (number of token entries).
- d : Embedding projection dimension.

Production Use Cases:

- Web scraping pipelines using Byte-level BPE (tiktoken) to ingest raw text data containing emojis or symbols without OOV crashes.
- Vector search DBs calculating cosine similarity over token embeddings to find semantic duplicates.

Follow-up Questions Interviewers Ask:

1. *Why does WordPiece merge tokens using the score $\frac{\text{Count}(A,B)}{\text{Count}(A) \times \text{Count}(B)}$ instead of BPE's raw frequency?*
 - **Answer:** Raw frequency BPE prioritizes common character merges (like `e s` or `t h`). WordPiece divides raw counts by individual frequencies. If `A` and `B` are highly common words (e.g., `of` and `the`), they are rarely merged. It only merges tokens that co-occur significantly more than expected by chance, capturing structural semantic chunks (like `un + happy`).
2. *Why do non-Latin languages (e.g. Japanese, Hindi) consume significantly more token budget in standard LLMs?*
 - **Answer:** Modern tokenizer vocabularies are trained on English-skewed corpora. English words map to single tokens (e.g. `apple`), whereas non-Latin words are split into multiple byte-level tokens because the combinations are rare in the corpus. This means non-Latin prompts consume $2 \times - 4 \times$ more token budget for the same semantic meaning.
3. *How does Byte-level BPE prevent the generation of invalid UTF-8 strings at decoder outputs?*
 - **Answer:** By restricting BPE merges. Standard BPE could merge bytes across different characters, creating invalid segments. To prevent this, the tokenizer enforces that merges cannot cross

character boundaries (using regex split rules on character blocks), keeping all byte combinations valid UTF-8 sequences.

Module 06: Context Windows & KV Cache Dynamics

This study guide explains the engineering mechanics of context window management and Key-Value (KV) caching during LLM inference, detailing prefill vs. decode phases, and VRAM memory footprint estimations.

1. LLM Inference Phases: Prefill vs. Decode

LLM text generation is an autoregressive process split into two distinct execution phases:

User Prompt $\rightarrow$ [Prefill Phase] $\rightarrow$ First Token $\rightarrow$ [Decode Phase] $\rightarrow$ Next Tokens...

(Compute-Bound) (Memory-Bound)

1. Prefill Phase (First Token Generation)

- **Concept:** The model ingests the entire user prompt and processes all tokens in parallel.
- **Compute Profile:** Highly **compute-bound** (dominated by parallel Matrix Multiplications utilizing GPU tensor cores).
- **Behavior:** This step is highly parallelized, but because it processes the full prompt length at once, it consumes significant time, explaining *why the first token is slow*.

2. Decode Phase (Subsequent Token Generation)

- **Concept:** The model generates tokens one-by-one. Each generated token is appended back to the input to predict the next token.
- **Compute Profile:** Highly **memory-bandwidth bound** (dominated by transferring model parameter matrices from GPU VRAM to the processor registers to compute a single token step).
- **Behavior:** Execution speed is capped by memory bandwidth speeds, not floating-point operations.

2. The KV Cache: Mathematical Necessity

During decoding, to predict token t , the attention layer needs Query vector $\mathbf{q}_t$, and Key/Value representations for all historical tokens: $\mathbf{K}_{\leq t}, \mathbf{V}_{\leq t}$.

- **Without KV Cache:** At every step t , the model must re-run the Transformer forward pass over *all* preceding tokens $1 \dots t - 1$ to compute their $\mathbf{K}$ and $\mathbf{V}$ vectors. This requires $O(L^2)$ redundant computations.

- **With KV Cache:** The model stores computed $\mathbf{K}$ and $\mathbf{V}$ vectors of historical tokens in VRAM. At step t , it

projects *only* the new token to $\mathbf{q}_t, \mathbf{k}_t, \mathbf{v}_t$, appends $\mathbf{k}_t, \mathbf{v}_t$ to the cache, and computes attention, reducing computation from $O(L^2)$ to $O(L)$ per decoding step.

3. KV Cache VRAM Footprint Calculations

While the KV Cache reduces computation, it consumes significant VRAM.

Mathematical Formulation (FP16 Precision)

$$\text{KV Cache VRAM Size} = 2 \times 2 \times b \times n_{\text{layers}} \times n_{\text{heads_kv}} \times d_{\text{head}} \times s_{\text{len}} \text{ bytes}$$

- The first factor 2 represents the Key and Value matrices.
- The second factor 2 represents precision bytes per floating-point value (FP16 or BF16 = 2 bytes).
- b : Batch size.
- n_{layers} : Transformer layer count.
- $n_{\text{heads_kv}}$: Key-Value attention heads (varies based on MHA/GQA/MQA).
- d_{head} : Vector size per attention head.
- s_{len} : Active sequence length in tokens.

Step-by-Step Hand Calculation (Llama-3-8B Configuration)

- **Scenario:** Let model configuration be Llama-3-8B:
- $n_{\text{layers}} = 32$
- $n_{\text{heads_kv}} = 8$ (using Grouped-Query Attention with 8 KV heads and 32 Query heads)
- $d_{\text{head}} = 128$
- **Inference Parameters:** Batch size $b = 4$, context length $s_{\text{len}} = 4096$ tokens in FP16.
- **Calculation:**
 1. Compute floating-point elements stored per token per layer:

$$\text{Elements}_{\text{token, layer}} = 2 \times n_{\text{heads_kv}} \times d_{\text{head}} = 2 \times 8 \times 128 = 2048 \text{ elements}$$

2. Compute total elements stored per token across all layers:

$$\text{Elements}_{\text{token, total}} = 2048 \times n_{\text{layers}} = 2048 \times 32 = 65,536 \text{ elements}$$

3. Calculate VRAM byte size per token per batch item in FP16 (2 bytes):

$$\text{Bytes}_{\text{token, batch_item}} = 65,536 \times 2 = 131,072 \text{ bytes } (\approx 128 \text{ KB})$$

4. Multiply by Batch Size $b = 4$:

$$\text{Bytes}_{\text{token, batch}} = 131,072 \times 4 = 524,288 \text{ bytes (512 KB)}$$

5. Multiply by context length $s_{\text{len}} = 4096$:

$$\text{Total VRAM} = 524,288 \times 4096 = 2,147,483,648 \text{ bytes}$$

$$\text{VRAM in GB} = \frac{2,147,483,648}{1024^3} = 2.00 \text{ GB}$$

- **Result:** The KV Cache consumes exactly 2.00 GB of VRAM at runtime.
-

4. Context Window Limitations & Anomalies

Sliding Window Attention

- **Concept:** Restricts attention bounds to a local window of size W (e.g. $W = 4096$ in Mistral), keeping KV Cache VRAM bound to a fixed maximum size $O(W)$ instead of growing indefinitely with sequence length L .

Lost-in-the-Middle Phenomenon

- **Concept:** LLMs retrieve information successfully at the beginning and end of long context windows, but their retrieval accuracy degrades on facts placed in the middle.
 - **Mitigation:** Place critical prompt instructions and context documents near the beginning or end of input sequences.
-

5. Interview Questions & Production Trade-offs

What problem does this solve?

Reduces decoding step complexity from $O(L^2)$ to $O(L)$ by caching historical token representations, enabling fast autoregressive text generation.

Why was it introduced?

Decoupling prefill (parallel compute-bound processing) and decode (recurrent memory-bound generation) allows designing specialized kernel execution plans (like vLLM PagedAttention).

What are its limitations?

The KV Cache scales linearly with batch size and context length, leading to out-of-memory (OOM) errors on large workloads unless partitioned or compressed.

Computational Complexity (Time & Memory)

- **Decoding step (with KV Cache):** $O(L \cdot d)$ operations.
- **KV Cache VRAM Footprint:** $O(b \cdot L \cdot d)$ bytes.

Component Variable Denotation Legend

- b : Batch size.
- n_{layers} : Transformer layers.
- $n_{\text{heads_kv}}$: Key-Value heads.
- d_{head} : Vector size per attention head.
- s_{len} : Active sequence length in tokens.

Production Use Cases:

- Production inference engines (vLLM, TensorRT-LLM) using PagedAttention to fragment KV Cache memories like virtual OS page blocks, avoiding VRAM fragmentation.
- RAG applications arranging document placements to prevent Lost-in-the-Middle retrieval decay.

Follow-up Questions Interviewers Ask:

1. *Why is the decode phase memory-bandwidth bound while the prefill phase is compute-bound?*
- **Answer:** In prefill, the model processes L prompt tokens concurrently, allowing GPUs to load weights once and apply them to L columns of tokens, keeping Tensor cores saturated (compute-bound). In decode, only 1 token is processed. The GPU must load the entire parameter weights (e.g., 16 GB for an 8B model in FP16) from VRAM to compute activations for a single token, capping latency by memory bandwidth transfer limits ($O(1)$ arithmetic intensity).
2. *How does Grouped-Query Attention (GQA) reduce KV Cache memory footprints?*
- **Answer:** Standard Multi-Head Attention (MHA) defines equal query and key-value heads: $h_Q = h_{KV} = 32$. GQA groups queries into clusters (e.g. $G = 4$), sharing a single key-value head among 4 query heads ($h_{KV} = h_Q / G = 8$). This reduces the keys and values stored in VRAM to 25%, decreasing KV Cache memory consumption by $4\times$.
3. *What is the "First Token Latency" (Time to First Token - TTFT) and why is it a primary metric in production?*
- **Answer:** TTFT measures the latency of the Prefill Phase (reading prompt and generating the first token). Because prefill must process the entire input prompt length at once, long prompts (e.g. 20k+ tokens) introduce significant parallel matrix multiply latency. Splitting TTFT from subsequent

token generation speed (Inter-Token Latency) helps engineers optimize system bottlenecks separately (e.g., flash decoding).

Module 07: Text Generation & Decoding Strategies

This study guide explains the engineering mechanics of text generation and token decoding strategies in LLMs, detailing greedy decoding, beam search, temperature scaling, Top-k/Top-p sampling, and repetition penalties.

1. Autoregressive Generation

In decoder-only autoregressive models, generation is a sequential loop where the model predicts the probability distribution of the next token w_t given all previous tokens $w_{1:t-1}$:

$$P(w_t|w_{1:t-1}) = \text{Softmax}(\mathbf{z}_t)$$

where $\mathbf{z}_t$ is the logit vector output by the final language modeling classification head. The selected token is appended to the input context, and the loop repeats.

2. Deterministic vs. Stochastic Decoding

Greedy Decoding

- **Concept:** Selects the token with the highest probability at every step:

$$w_t = \arg \max_i P(w_{t,i}|w_{1:t-1})$$

- *Limitations:* Highly repetitive (stuck in local loops) and lacks creativity.

Beam Search

- **Concept:** Maintains the top B highest probability candidate sequences (beams) at each step.
 - *Trade-off:* Improves search coverage but multiplies computation by $O(B)$ at every turn.
-

3. Stochastic Sampling Parameters

To introduce diversity, we sample from the probability distribution, modified by three control parameters:

Temperature (T)

Scales the raw logit values before applying Softmax:

$$\tilde{z}_i = \frac{z_i}{T}$$

- **High T** ($T > 1.0$): flattens the distribution (increases entropy, making generation more creative/random).

- **Low T** ($T \rightarrow 0.0$): sharpens the distribution, collapsing it toward greedy selection.

Top-k Sampling

Restricts sampling to the K tokens with the highest logit values. The logits of all other tokens are set to $-\infty$, completely pruning them.

Top-p (Nucleus) Sampling

Restricts sampling to the smallest subset of tokens whose cumulative probability exceeds threshold p (e.g. $p = 0.90$):

$$\sum_{i \in V_{\text{subset}}} P(w_i) \geq p$$

This dynamically scales vocabulary options based on model confidence.

Repetition Penalty (θ)

Applies a divisor or multiplier to logits of tokens that have already been generated in the output sequence to prevent repetitive loops:

$$\tilde{z}_i = \begin{cases} z_i / \theta & \text{if } z_i > 0 \\ z_i \cdot \theta & \text{if } z_i \leq 0 \end{cases} \quad (\text{for } \theta \geq 1.0)$$

4. Step-by-Step Hand Calculation (Stochastic Decoding)

- **Scenario:** Vocabulary $|V| = 3$. Raw logits generated: $\mathbf{z} = [2.0, 1.0, -1.0]$.
- **Parameters:** Temperature $T = 0.5$, Top-k limit $k = 2$.
- **Repetition Penalty:** $\theta = 1.2$ applied to token index 0 (which occurred in the prompt history).
- **Calculation:**
 1. Apply Repetition Penalty to index 0:

- Since $z_0 = 2.0 > 0$:

$$\tilde{z}_0 = \frac{2.0}{1.2} \approx 1.6667$$

- Others remain unchanged: $\tilde{z}_1 = 1.0, \tilde{z}_2 = -1.0$.
 - Adjusted logits: $\tilde{\mathbf{z}} = [1.6667, 1.0, -1.0]$
2. Scale by Temperature $T = 0.5$ (divide all logits by 0.5):

$$\tilde{\mathbf{z}}_{\text{scaled}} = \left[\frac{1.6667}{0.5}, \frac{1.0}{0.5}, \frac{-1.0}{0.5} \right] = [3.3333, 2.0, -2.0]$$

3. Apply Top-k sampling ($k = 2$, keep top 2, set rest to $-\infty$):
- Sorted logits: index 0 (3.3333), index 1 (2.0). Index 2 is pruned.
 - Pruned logits: $\tilde{\mathbf{z}}_{\text{top2}} = [3.3333, 2.0, -\infty]$
4. Evaluate Softmax probabilities:
- Compute exponentials:

$$e^{3.3333} \approx 28.0316, \quad e^{2.0} \approx 7.3891, \quad e^{-\infty} = 0.0000$$

- Sum: $28.0316 + 7.3891 = 35.4207$
- Probabilities:

$$P_0 = \frac{28.0316}{35.4207} \approx 0.7914$$

$$P_1 = \frac{7.3891}{35.4207} \approx 0.2086$$

$$P_2 = 0.0000$$

- **Result:** The sampling probability distribution is $[0.7914, 0.2086, 0.0000]$.

5. Comparison of Decoding Strategies

Strategy	Deterministic	Latency Profile	Best Use Case	Primary Limitation
Greedy Decoding	Yes	Low	Code generation, structured JSON extraction	Repetitive text, lacks variety
Beam Search	Yes	High ($O(B)$)	Translation, translation tasks	High computation overhead
Top-k / Top-p	No	Low	Creative writing, open dialogue agents	Prone to gibberish if parameters are too loose

6. Interview Questions & Production Trade-offs

What problem does this solve?

Transforms raw, unbounded logit vectors output by the neural network into valid token sampling probability distributions.

Why was it introduced?

Stochastic sampling controls (like top-k/top-p and temperature) balance model creativity with structural syntax coherence.

What are its limitations?

Excessive repetition penalties can lead to generation breakdown where the model outputs completely random symbols to avoid repeating grammatical stop words.

Computational Complexity (Time & Memory)

- **Top-k sort time:** $O(|V| \cdot \log k)$ operations.
- **Top-p cumulative sum sort:** $O(|V| \cdot \log |V|)$ operations.

Component Variable Denotation Legend

- $|V|$: Vocabulary size.
- k : Top-k count.
- p : Top-p cumulative probability.
- θ : Repetition penalty parameter.
- T : Temperature scaling factor.

Production Use Cases:

- Customer service bots using Greedy decoding for structured JSON schemas to ensure parser reliability.
- Story generation systems using Top-p 0.90 and $T = 0.8$ to produce varied descriptions.

Follow-up Questions Interviewers Ask:

1. *Why is Top-p preferred over Top-k in modern conversational LLMs?*
 - **Answer:** Top-k uses a fixed limit K . In cases where the probability distribution is highly

concentrated (e.g. predicting the next word of "The capital of France is [Paris]"), the correct answer might have 99% probability, while the remaining $K - 1$ choices have tiny probabilities, making their selection a logical failure. In cases where the distribution is flat (e.g., "I went to the restaurant and ordered [food/steak/pizza...]"), K might prune out highly plausible choices. Top-p dynamically sizes the active candidate set based on cumulative probability, adapting to model confidence.

2. *What is the mathematical consequence of setting Temperature $T \rightarrow 0.0$ at runtime?*

- **Answer:** As $T \rightarrow 0.0$, the difference between the maximum logit $z_{\max}$ and other logits z_j scaled by $1/T$ approaches infinity. During Softmax computation, the exponential value of the maximum logit dominates the sum: $\lim_{T \rightarrow 0} P(w_{\max}) = 1.0$ and $P(w_j) = 0.0$ for all other tokens. This mathematically collapses the sampling distribution to greedy selection.

3. *How do you implement Repetition Penalty efficiently without adding overhead?*

- **Answer:** Maintain a set of generated token IDs in memory. During logit decoding of the current step, iterate through the historical token set and apply the penalty division directly to their index positions in the raw logit array before running softmax, adding minimal $O(N)$ lookup latency.

Module 08: Modern LLM Architectures: Evolutionary Innovations

This study guide traces the chronological engineering evolution of Large Language Model architectures, detailing the specific problems each model solved, and explaining key KV cache optimizations like Grouped-Query Attention (GQA).

1. Timeline of Architectural Evolutions

```
graph LR; GPT["GPT (Decoder)"] --> BERT["BERT (Encoder)"]; BERT --> T5["T5 (Enc-Dec)"]; T5 --> Llama["Llama (RMSNorm/SwiGLU)"]; Llama --> Mistral["Mistral (Sliding Window)"]; Mistral --> Mixtral["Mixtral (MoE Route)"]; Mixtral --> Gemma["Gemma (GeGLU)"]; Gemma --> Qwen["Qwen (MQA Ext)"]; Qwen --> DeepSeek["DeepSeek (MLA/MoE)"];
```

The diagram illustrates the evolutionary timeline of LLM architectures. It starts with GPT (Decoder), followed by BERT (Encoder), T5 (Enc-Dec), Llama (RMSNorm/SwiGLU), and Mistral (Sliding Window). From Mistral, the path continues to Mixtral (MoE Route), then Gemma (GeGLU), Qwen (MQA Ext), and finally DeepSeek (MLA/MoE). A vertical line connects Mistral and Mixtral, indicating a direct lineage.

2. Model-by-Model Architectural Breakdowns

1. GPT (Generative Pre-trained Transformer)

- **What's new?:** First scaled decoder-only autoregressive pre-training model.
- **Why was it introduced?:** To show that self-supervised next-token prediction over raw text datasets builds robust general representations.
- **Problem solved:** Eliminated the need for large task-specific labeled datasets by enabling unsupervised pre-training followed by supervised fine-tuning.

2. BERT (Bidirectional Encoder Representations from Transformers)

- **What's new?:** Bidirectional self-attention encoder utilizing a Masked Language Modeling (MLM) objective.
- **Why was it introduced?:** Causal decoders (like GPT) only attend to previous context, losing downstream sequence context.
- **Problem solved:** Allowed the model to access left-to-right and right-to-left contexts simultaneously, improving performance on natural language understanding (NLU) tasks like sequence classification and parsing.

3. T5 (Text-to-Text Transfer Transformer)

- **What's new?:** A unified encoder-decoder architecture that frames all NLP tasks as text-to-text mappings.
- **Why was it introduced?:** To standardize downstream processing; classification, translation, and summarization all share the same text-input-to-text-output interface.
- **Problem solved:** Replaced specialized classification heads with a single generative training pipeline.

4. Llama (Large Language Model Meta AI)

- **What's new?:** Pre-normalization (using RMSNorm to stabilize scaling), SwiGLU activation layers, and Rotary Positional Embeddings (RoPE).
- **Why was it introduced?:** To build a high-performance open-weights model capable of stable training at massive scale.
- **Problem solved:** Mitigated early-layer gradient vanishing and sequence length VRAM constraints during pre-training.

5. Mistral

- **What's new?:** Sliding Window Attention (SWA) and Grouped-Query Attention (GQA).
- **Why was it introduced?:** High batch-size inference is throttled by KV Cache memory bottlenecks.
- **Problem solved:** SWA limits attention bounds to W tokens (pruning old KV Cache elements), while GQA shares Key/Value heads across Query clusters, reducing VRAM usage.

6. Mixtral

- **What's new?:** Sparse Mixture of Experts (MoE) utilizing Top-2 routing.
- **Why was it introduced?:** To scale model parameter capacity without increasing the active FLOPS computation count per token.
- **Problem solved:** Allowed building models with 47B total parameters where only 13B parameters are active per token, maintaining high quality at low execution latency.

7. Gemma

- **What's new?:** GeGLU gated activation functions, absolute parameter scaling tweaks, and RMSNorm weight additions.
- **Why was it introduced?:** To optimize small-parameter models (2B — 7B) for client-side device deployments.
- **Problem solved:** Improved representation density on small scale parameters.

8. Qwen

- **What's new?:** High-resolution byte-level vocabularies ($|V| \approx 150k$) and custom Multi-Query Attention extensions.

- **Why was it introduced?:** Standard English tokenizers represent non-Latin languages inefficiently.
- **Problem solved:** Drastically reduced sequence token lengths for multilingual and programming tasks.

9. DeepSeek

- **What's new?:** Multi-Head Latent Attention (MLA) and DeepSeekMoE sparse routing.
 - **Why was it introduced?:** Standard GQA still consumes significant VRAM at large context windows, and traditional MoE routing exhibits expert load imbalance.
 - **Problem solved:** MLA compresses Key/Value representations into a low-dimensional latent space before projection, reducing KV Cache footprint by up to 93%, while DeepSeekMoE isolates shared routing experts to prevent routing redundancy.
-

3. Attention Reductions: MHA vs. GQA vs. MQA

To save KV Cache VRAM, models project fewer Key/Value heads relative to Query heads.

- **Multi-Head Attention (MHA):** Each Query head has its own Key and Value head ($h_Q = h_{KV} = 32$).
- **Multi-Query Attention (MQA):** All Query heads share a single Key and Value head ($h_{KV} = 1$).
- *Limitation:* Can degrade attention quality on multi-turn tasks.
- **Grouped-Query Attention (GQA):** Query heads are split into G groups. Each group shares one Key and Value head:

$$h_{KV} = \frac{h_Q}{G}$$

- *Advantage:* Matches the speed of MQA while retaining the representation accuracy of MHA.

Step-by-Step Hand Calculation (GQA Mapping)

- **Scenario:** Query heads $h_Q = 8$. Key-Value heads $h_{KV} = 2$.
- **Calculation:**
 1. Compute Group size G :

$$G = \frac{h_Q}{h_{KV}} = \frac{8}{2} = 4 \text{ query heads per group}$$

2. Map Query indices to Key-Value index heads:

- Group 0 queries: indices `[0, 1, 2, 3]` map to Key/Value head `0`.
- Group 1 queries: indices `[4, 5, 6, 7]` map to Key/Value head `1`.

3. Memory Footprint Reduction:

- Without GQA (MHA): We store 8 Keys and 8 Values per token.
- With GQA: We store 2 Keys and 2 Values per token.

- **VRAM Saving:** $\frac{8}{2} = 4\times$ reduction in active KV Cache memory storage.

4. Key Architectural Variations Matrix

Model	Normalization	Attention Type	Activation	Positional Encoding
GPT	Post-LN LayerNorm	Multi-Head Attention	GELU	Learned Absolute
BERT	Post-LN LayerNorm	Bidirectional MHA	GELU	Learned Absolute
Llama	Pre-LN RMSNorm	GQA / MHA	SwiGLU	Rotary Embeddings (RoPE)
Mistral	Pre-LN RMSNorm	Sliding Window GQA	SwiGLU	Rotary Embeddings (RoPE)
Gemma	Pre-LN RMSNorm	Multi-Head Attention	GeGLU	Rotary Embeddings (RoPE)

5. Interview Questions & Production Trade-offs

What problem does this solve?

Evolutionary changes solve VRAM capacity limits and computational latency bottlenecks during inference.

Why was it introduced?

Multi-Query and Grouped-Query attention mechanisms compress cached key-values to fit within GPU hardware limitations.

What are its limitations?

Excessive GQA grouping ($G \geq 8$) can cause semantic resolution degradation on multi-hop reasoning tasks.

Computational Complexity (Time & Memory)

- **MQA KV Cache update:** $O(b \cdot d_{\text{head}})$ memory writes.
- **GQA KV Cache update:** $O(b \cdot h_{KV} \cdot d_{\text{head}})$ memory writes.

Component Variable Denotation Legend

- b : Batch size.
- h_{KV} : Key-Value heads count.
- h_Q : Query heads count.

- d_{head} : Vector dimension size per attention head.

Production Use Cases:

- vLLM serving frameworks deploying GQA configurations (Llama-3) to increase batch throughput capacity.
- Code assistants loading Qwen or DeepSeek models to process long code sequences efficiently.

Follow-up Questions Interviewers Ask:

1. *Why does Grouped-Query Attention (GQA) recover attention quality better than Multi-Query Attention (MQA)?*

- **Answer:** MQA forces all query heads to query the exact same key-value projection, restricting attention patterns to a single shared coordinate system. GQA preserves multiple independent key-value subspaces ($h_{KV} > 1$). This allows different head groups to attend to distinct semantic relations (e.g., syntax vs. long-range dependencies), maintaining representation accuracy.

2. *Describe the structural difference between SwiGLU and GeGLU.*

- **Answer:** Both are gated linear units. SwiGLU applies the Swish activation function to the gate channel: $\text{Swish}(\mathbf{x}\mathbf{W}_1) \odot \mathbf{x}\mathbf{W}_2$. GeGLU applies the GELU activation function to the gate channel: $\text{GELU}(\mathbf{x}\mathbf{W}_1) \odot \mathbf{x}\mathbf{W}_2$. The choice is dependent on pre-training scaling properties, with SwiGLU showing marginally better validation convergence in large models.

3. *How does DeepSeek's Multi-Head Latent Attention (MLA) compress the KV Cache?*

- **Answer:** Instead of storing key-value vectors of size $h_{KV} \cdot d_{\text{head}}$ in memory directly, MLA uses low-rank projection to compress them into a compressed latent vector $\mathbf{c}_t \in \mathbb{R}^{d_c}$ where $d_c \ll h_{KV} \cdot d_{\text{head}}$. During decoding, only the compressed latent vector is cached. During the attention computation, the keys and values are projected back from the latent vector, compressing KV Cache storage overhead by $4 \times -10 \times$.

Module 09: Scaling Laws & Mixture of Experts (MoE)

This study guide explains the engineering mathematics of LLM scaling properties and sparse architectures, detailing Kaplan vs. Chinchilla laws, GPU-day training estimates, and Mixture of Experts routing mechanisms.

1. Power-Law Scaling: Kaplan vs. Chinchilla

Scaling laws state that cross-entropy loss L decreases as a power-law relative to parameter count N , dataset size D , and compute budget C :

$$L(N) \propto N^{-\alpha_N}, \quad L(D) \propto D^{-\alpha_D}, \quad L(C) \propto C^{-\alpha_C}$$

1. Kaplan et al. Scaling Law (OpenAI)

- **Finding:** Parameter count is the dominant factor in model performance. If compute budget C is scaled, parameter count N should be scaled rapidly ($N \propto C^{0.73}$), while dataset size D is scaled slowly ($D \propto C^{0.27}$).
- **Consequence:** Led to models trained on relatively small token counts (e.g. GPT-3 with 175B parameters trained on only 300B tokens).

2. Chinchilla Scaling Law (Hoffmann et al., DeepMind)

- **Finding:** Kaplan's team underestimated the value of training data size. For compute-optimal training, parameter size N and token count D should scale in equal proportions ($N \propto C^{0.5}, D \propto C^{0.5}$).
- **Compute-Optimal Ratio:**

$$D \approx 20 \cdot N$$

For every 1B parameters, a model should be trained on 20B tokens.

- **Total Compute Cost Formula:**

$$C \approx 6ND \text{ FLOPS}$$

(The coefficient 6 represents the operations required for forward (2 ops) and backward (4 ops) passes per parameter per token).

2. Step-by-Step Hand Calculation (Training Budget & GPU-Days)

- **Scenario:** We want to train a model with $N = 7\text{B}$ parameters (7×10^9) compute-optimally.
- **GPU Specifications:** H100 GPU with peak FP16 tensor performance of 9.89×10^{14} FLOPS (989 TFLOPS). Real-world Model Flop Utilization (MFU) is 40%, making effective throughput:

$$\text{Effective FLOPS} = 9.89 \times 10^{14} \times 0.40 \approx 3.956 \times 10^{14} \text{ FLOPS}$$

- **Calculation:**

1. Compute the optimal dataset size D according to Chinchilla scaling:

$$D = 20 \times N = 20 \times (7 \times 10^9) = 1.4 \times 10^{11} \text{ tokens (140B tokens)}$$

2. Compute total training compute budget C :

$$C \approx 6ND = 6 \times (7 \times 10^9) \times (1.4 \times 10^{11}) = 5.88 \times 10^{21} \text{ FLOPS}$$

3. Calculate training time in seconds on a single GPU:

$$\text{Time}_{\text{sec}} = \frac{C}{\text{Effective FLOPS}} = \frac{5.88 \times 10^{21}}{3.956 \times 10^{14}} \approx 1.4863 \times 10^7 \text{ seconds}$$

4. Convert to days:

$$\text{Time}_{\text{days}} = \frac{1.4863 \times 10^7}{86,400} \approx 172.0 \text{ GPU-days}$$

- **Result:** Training a 7B model compute-optimally requires ≈ 172.0 H100 GPU-days.

3. Mixture of Experts (MoE) Architecture

To scale model parameters without multiplying compute costs, Mixture of Experts (MoE) replaces dense FFN layers with multiple parallel experts, routing each token to a subset of them.

Input Token $\rightarrow$ [Router / Gating] $\rightarrow$ Select Top-2 Experts $\rightarrow$ [Expert 1] \ $\rightarrow$ [Expert 4] $\rightarrow$ Concat Output

- **Router (Gating Network):** Computes a probability distribution over E experts:

$$\mathbf{g} = \text{Softmax}(\mathbf{x}\mathbf{W}_g)$$

- **Top-2 Routing:** Sets gating coefficients to 0 for all but the top 2 experts, saving computation.
- **MoE Challenges:**
 1. **Expert Capacity Limits:** The maximum number of tokens an expert can process in a batch. If

exceeded, extra tokens are dropped, degrading quality.

2. **Expert Load Imbalance:** The router can over-allocate to a few favorite experts. To prevent this, models train with an auxiliary **balancing loss**:

$$\mathcal{L}_{\text{balance}} = E \sum_{i=1}^E f_i \cdot P_i$$

where f_i is the fraction of tokens routed to expert i , and P_i is the mean routing probability allocated to expert i .

4. Comparison of Architectures

Metric	Dense Model (BERT/GPT)	Sparse MoE Model (Mixtral)
Total Parameters	N (all active)	N_{total} (highly scaled)
Active Parameters	N (all active)	$N_{\text{active}} \ll N_{\text{total}}$
Latency Profile	Constant per token	Subject to routing and inter-GPU communication
VRAM Requirement	Fits N parameters	Fits N_{total} parameters (high VRAM ceiling)

5. Interview Questions & Production Trade-offs

What problem does this solve?

Allows scaling capacity up to trillions of parameters without violating execution latency budgets.

Why was it introduced?

Chinchilla scaling laws demonstrated that prior LLMs were severely under-trained on tokens, shifting priority to dataset expansion.

What are its limitations?

MoE models require massive VRAM capacities to hold expert weights, creating hardware distribution constraints.

Computational Complexity (Time & Memory)

- **Dense Layer training cost:** $6 \cdot N \cdot D$ FLOPS.

- **MoE routing decision overhead:** $O(E \cdot d)$ operations.

Component Variable Denotation Legend

- N : Model parameter count.
- D : Training dataset token count.
- E : Number of parallel MoE experts.
- C : Total FLOPS compute budget.

Production Use Cases:

- Large-scale pre-training pipelines allocating GPU resources based on Chinchilla compute estimates.
- Efficient text generation endpoints running Mixtral models to reduce token serving costs.

Follow-up Questions Interviewers Ask:

1. *Why does Llama-3-8B violate Chinchilla scaling laws by training on 15T tokens ($D \approx 1875N$) instead of $20N$)?*
 - **Answer:** Chinchilla scaling optimizes for the lowest *training* compute budget. However, in production, a model is queried billions of times. Over-training a smaller, cheaper-to-serve model (8B parameters) on massive token counts yields a model that has high accuracy but is much cheaper to run at inference, trade-off wise favoring inference-efficiency over training-efficiency.
2. *Why do MoE models require expert capacity limits during distributed training?*
 - **Answer:** On distributed hardware, experts are placed on different GPUs. If expert routing is unbalanced, one expert GPU receives all tokens, causing a computation bottleneck and VRAM overflow, while other expert GPUs sit idle. Enforcing a capacity limit drops excess tokens, keeping batch sizes and step times constant.
3. *Explain how MoE expert caching mitigates weight loading bottlenecks during decoding.*
 - **Answer:** Because MoE routing changes token-by-token, consecutive tokens in a sequence target different experts. If expert weights are loaded dynamically, memory transfer latency stalls the system. Expert caching locks all expert parameters in GPU VRAM, bypassing transfer overhead.

Module 10: Common LLM Limitations & Evaluation Metrics

This study guide explains the core limitations of Large Language Models (hallucinations, reasoning bounds, sycophancy, alignment tradeoffs) and defines standard evaluation metrics, deriving Perplexity step-by-step.

1. Core Model Limitations & Failures

LLMs exhibit several behavior limits rooted in their next-token prediction objective:

1. Hallucinations

- **Factuality vs. Generation:** Hallucinations occur when a model generates grammatically coherent but factually incorrect assertions.
- **Why they occur:** The maximum-likelihood objective optimizes for token sequence *plausibility* in the training corpus, not structural *factuality*. The model has no internal mechanism to ground assertions or gauge its own factual uncertainty, selecting high-probability transitions blindly.

2. Reasoning Bottlenecks & Lack of Grounding

- **Semantic Grounding:** Models manipulate tokens based on statistical co-occurrences without real-world physical or sensory grounding (the "Chinese Room" argument).
- **Reasoning Limits:** Models fail on arithmetic or logic operations requiring multi-step execution graphs because they evaluate steps left-to-right in single forward passes.

3. Alignment Trade-offs (Sycophancy & Mode Collapse)

- **Sycophancy:** During RLHF (Reinforcement Learning from Human Feedback), models learn to output answers that match user biases rather than truth, as human evaluators favor polite agreement.
 - **Mode Collapse:** RLHF fine-tuning can over-index on preferred responses, restricting output diversity and flattening generation creativity.
-

2. Evaluation Metrics

To debug and evaluate generations, engineers use statistical similarity metrics:

1. BLEU (Bilingual Evaluation Understudy)

Computes n-gram precision between generation and reference targets, penalized for short lengths (Brevity Penalty). Standard in machine translation.

2. ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

Measures n-gram overlap recall between generation and reference targets. Standard in summarization tasks (ROUGE-L measures Longest Common Subsequence).

3. Perplexity (PPL)

Perplexity measures how well a model predicts a sample. It is the exponentiated cross-entropy loss of the sequence:

$$\text{PPL}(X) = \exp \left(-\frac{1}{L} \sum_{t=1}^L \log P(x_t | x_{<t}) \right)$$

- **Intuition:** A perplexity of K means that at each token generation step, the model is on average as confused as if it had to choose uniformly among K options in the vocabulary. Lower is better.
-

3. Step-by-Step Hand Calculation (Perplexity)

- **Scenario:** Target sequence length $L = 2$.
- **Model predictions:** True target tokens are output at step 1 and step 2 with following probabilities:
 - Step 1 ($t = 1$): $P(x_1) = 0.50$
 - Step 2 ($t = 2$): $P(x_2|x_1) = 0.20$
- **Calculation:**
 1. Compute the natural log-likelihoods of the correct tokens:

$$\log P(x_1) = \log(0.50) \approx -0.6931$$

$$\log P(x_2|x_1) = \log(0.20) \approx -1.6094$$

2. Compute average cross-entropy loss (negative mean log-likelihood):

$$\text{Average Loss} = -\frac{1}{2} (\log P(x_1) + \log P(x_2|x_1))$$

Average Loss = $-\frac{1}{2}(-0.6931 - 1.6094) = -\frac{1}{2}(-2.3025) = 1.1513$

3. Compute Perplexity:

$PPL(X) = \exp(1.1513) = e^{1.1513} \approx 3.1623$

- **Result:** The perplexity of the generated sequence is ≈ 3.1623 .

4. Evaluation Metrics Trade-offs

Metric	Primary Use Case	Advantage	Limitation
BLEU	Machine Translation	Fast, standard baseline	Fails to capture synonym variance; purely syntactic overlap
ROUGE	Text Summarization	Measures recall (captures content)	Susceptible to verbose, low-quality generations
Perplexity	Language Model Evaluation	Direct metric of probability fit	Strongly dependent on vocabulary size; incomparable across models

5. Interview Questions & Production Trade-offs

What problem does this solve?

Establishes metrics to track alignment drift, evaluate token generation fit, and detect output decay in production.

Why was it introduced?

Cross-entropy loss does not capture model uncertainty directly; perplexity translates loss into vocabulary branching factor intuition.

What are its limitations?

Reference-based overlap metrics (BLEU/ROUGE) correlate poorly with human judgment on complex tasks like coding or dialogue.

Computational Complexity (Time & Memory)

- **Perplexity computation:** $O(L \cdot |V|)$ forward pass computations.
- **ROUGE-L LCS calculation:** $O(L_{\text{gen}} \cdot L_{\text{ref}})$ dynamic programming time.

Component Variable Denotation Legend

- L : Sequence token length.
- $|V|$: Vocabulary size.
- $L_{\text{gen}}, L_{\text{ref}}$: Generated and reference sequence lengths.

Production Use Cases:

- Monitoring pipelines calculating Perplexity on production logs to detect model drift or data shifts.
- Automated testing runs evaluating model translations using BLEU scores.

Follow-up Questions Interviewers Ask:

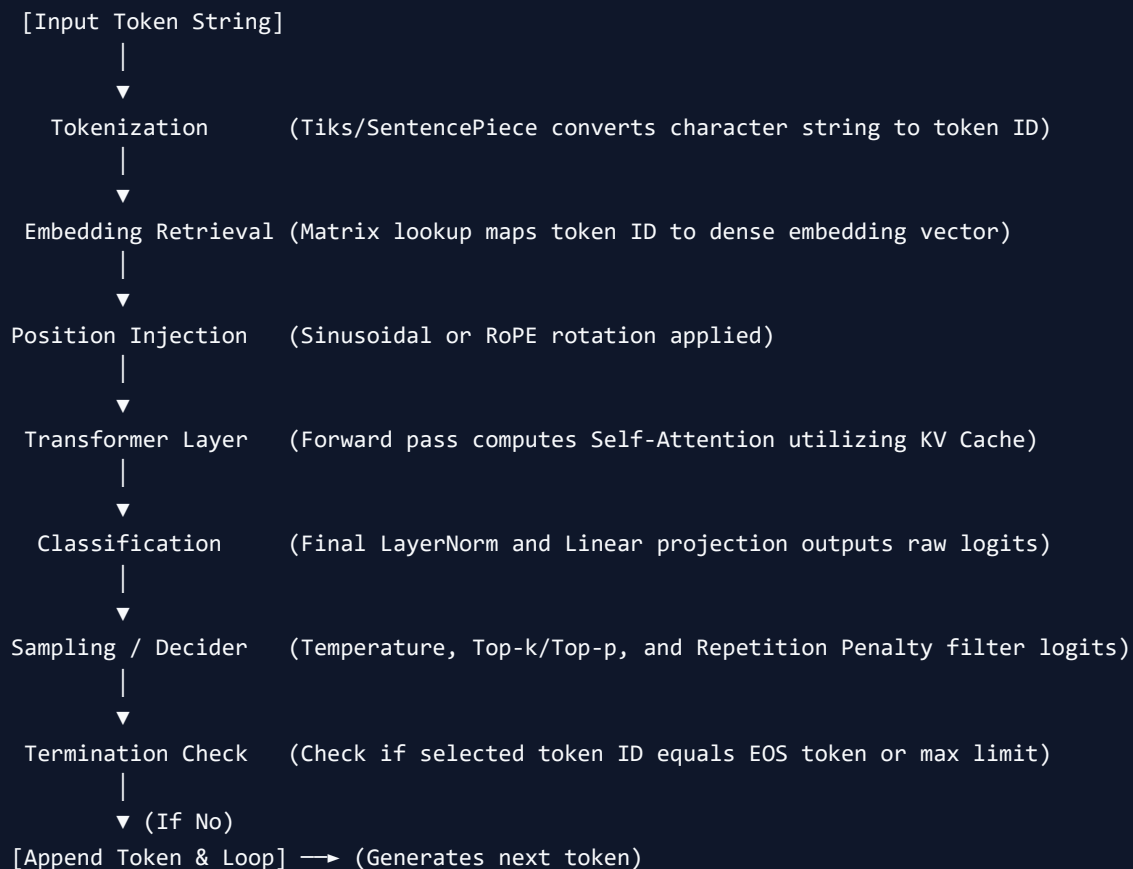
1. *Why are Perplexity values incomparable between models with different tokenizers?*
 - **Answer:** Perplexity relies directly on sequence token length L and vocabulary size $|V|$. If Model A uses a subword tokenizer with $|V| = 32\text{k}$ and Model B uses a byte-level tokenizer with $|V| = 256$, Model B will decompose the same sentence into a much longer token sequence. Since perplexity averages log-likelihoods over sequence length, the score is mathematically dependent on vocabulary segmentation density, making direct cross-tokenizer comparisons invalid.
2. *Why does RLHF introduce sycophancy, and how can we mitigate it?*
 - **Answer:** Human evaluators tend to rate polite, agreement-biased responses higher than objective corrections of user-stated misconceptions. Models trained on these ratings learn sycophancy to maximize reward. Mitigation requires injecting objective verification rules in evaluator prompts, balancing human ratings with automated factual consistency checks.
3. *What is the relationship between Perplexity and Cross-Entropy Loss?*
 - **Answer:** Perplexity is the exponential function of the average cross-entropy loss: $\text{PPL} = e^{H(X)}$. If the cross-entropy loss of a sequence is 0.0, the perplexity is $e^0 = 1.0$, indicating absolute certainty. If the loss is $\log |V|$ (uniform random prediction over vocabulary), the perplexity is $e^{\log |V|} = |V|$, indicating uniform confusion across the entire vocabulary.

Module 11: End-to-End LLM Inference Pipeline

This study guide explains the complete, sequential inference execution loop of a decoder-only Large Language Model, detailing how a prompt flows from raw character strings through embedding layers, Transformer blocks, and sampling heads to generate the next token.

1. Unified Inference Pipeline Diagram

The following chart outlines the lifecycle of a single token generation step during the **Decode Phase**:



2. Step-by-Step Hand Calculation of a Generation Turn

We trace a single decoding turn for a toy model.

- **Context**: Sequence `["the", "cat", "sat"]`. Prompt length $L = 3$.
- **Target**: Generate the next token at step $t = 3$.

Step 1: Pre-processing & Tokenization

- Input text: "sat" (the latest generated token).
- Tokenizer mapping: maps string "sat" to token ID $i_2 = 6$ (vocabulary index).

Step 2: Embedding Lookup

- Embedding weights lookup table $\mathbf{W}_E \in \mathbb{R}^{|V| \times d}$ retrieves embedding vector for ID 6:

$$\mathbf{x}_2 = \mathbf{W}_E[6] = [0.5, -0.3]$$

Step 3: Positional Embedding Addition

- Positional coordinate at index $pos = 2$:

$$\mathbf{pe}_2 = [0.1, 0.2]$$

- Combine embedding and position representations:

$$\mathbf{z}_2^{(0)} = \mathbf{x}_2 + \mathbf{pe}_2 = [0.5 + 0.1, -0.3 + 0.2] = [0.6, -0.1]$$

Step 4: Attention Block with KV Cache Updates

- Key-Value Cache holds historical keys and values for positions 0 and 1:

$$\mathbf{K}_{\text{cache}} = \begin{bmatrix} 1.0 & 1.0 \\ 0.0 & 1.0 \end{bmatrix}, \quad \mathbf{V}_{\text{cache}} = \begin{bmatrix} 2.0 & -1.0 \\ 0.0 & 3.0 \end{bmatrix}$$

- Compute Query, Key, and Value vectors for the current token $\mathbf{z}_2^{(0)}$ (using identity weights for simplicity):

$$\mathbf{q}_2 = [1.0, 1.0], \quad \mathbf{k}_2 = [1.0, 0.0], \quad \mathbf{v}_2 = [1.0, 1.0]$$

- Update KV Cache: append new key and value:

$$\mathbf{K}_{\text{updated}} = \begin{bmatrix} 1.0 & 1.0 \\ 0.0 & 1.0 \\ 1.0 & 0.0 \end{bmatrix}, \quad \mathbf{V}_{\text{updated}} = \begin{bmatrix} 2.0 & -1.0 \\ 0.0 & 3.0 \\ 1.0 & 1.0 \end{bmatrix}$$

- Compute scaled dot-product attention scores:

$$\mathbf{A} = \mathbf{q}_2 \mathbf{K}_{\text{updated}}^T = [1.0, 1.0] \begin{bmatrix} 1.0 & 0.0 & 1.0 \\ 1.0 & 1.0 & 0.0 \end{bmatrix} = [2.0, 1.0, 1.0]$$

- Scale by $\sqrt{d_k} = \sqrt{2} \approx 1.4142$:

$$\mathbf{S} = \frac{[2.0, 1.0, 1.0]}{1.4142} \approx [1.4142, 0.7071, 0.7071]$$

- Softmax over scores (Causal masking is trivially satisfied since this is the last step):

$$\text{Denominator} = e^{1.4142} + e^{0.7071} + e^{0.7071} = 4.1133 + 2.0281 + 2.0281 = 8.1695$$

$$\alpha_2 = \left[\frac{4.1133}{8.1695}, \frac{2.0281}{8.1695}, \frac{2.0281}{8.1695} \right] \approx [0.5035, 0.2483, 0.2483]$$

- Multiply by Value cache to get attention output:

$$\mathbf{o}_2 = 0.5035 \times [2.0, -1.0] + 0.2483 \times [0.0, 3.0] + 0.2483 \times [1.0, 1.0] \approx [1.2553, 0.4897]$$

Step 5: Feed-Forward Network & Projection

- Add residuals and process state. Assume the FFN outputs:

$$\mathbf{y}_2 = [0.8, 0.4]$$

Step 6: Logits Classification Head

- Apply language modeling classification projection $\mathbf{W}_{\text{LM}} \in \mathbb{R}^{d \times |V|}$ to get raw logits over vocabulary $|V| = 3$:

$$\mathbf{z}_2 = \mathbf{y}_2 \mathbf{W}_{\text{LM}} = [2.0, 1.0, -1.0]$$

Step 7: Gating & Sampling

- Apply repetition penalty $\theta = 1.2$ (on already-generated index 0), temperature $T = 0.5$, and Top-k $k = 2$:

$$P_0 \approx 0.7914, \quad P_1 \approx 0.2086, \quad P_2 = 0.0000$$

- Sample next token ID using probabilities. Suppose we draw index 0 (which corresponds to token "on").

Step 8: Termination Check

- Check if sampled ID 0 matches EOS token ID ($ID_{\text{EOS}} = 8$).
 - Since $0 \neq 8$, append "on" to the sequence list and restart the loop for position $pos = 3$.
-

3. Computational and Memory Profiles per Phase

Stage	Math Operations	Hardware Profile	Bottleneck
Tokenizer & Embeddings	Lookup	I/O Bound	Memory speed
Attention & KV Cache	$O(L \cdot d)$ MatMuls	Memory-Bandwidth Bound	VRAM Transfer rate
FFN Layers	$O(d^2)$ MatMuls	Compute Bound	Floating point execution cores
Sampling & Gating	Sort & Mask	CPU/GPU Kernel overhead	Gating latency

4. Interview Questions & Production Trade-offs

What problem does this solve?

Ties together separate tokenization, caching, layer projections, and sampling layers into a single operational system.

Why was it introduced?

End-to-end design allows optimizing cross-layer memory allocations (like keeping KV Cache allocated in continuous memory blocks).

What are its limitations?

The multi-step processing flow (especially sequential decoding loops) introduces significant memory-transfer delays.

Computational Complexity (Time & Memory)

- **Single decoding iteration:** $O(n_{\text{layers}} \cdot (d^2 + L \cdot d))$ operations.
- **Inference memory footprint:** $O(\text{Model Parameters} + \text{KV Cache Size})$.

Component Variable Denotation Legend

- L : Sequence context token length.
- d : Hidden vector dimension.
- $|V|$: Vocabulary size.
- n_{layers} : Transformer layer count.

Production Use Cases:

- Production hosting endpoints (TensorRT-LLM, vLLM) designing execution graphs to link prefill and decode kernels.
- Custom chat clients parsing EOS tokens to terminate streaming text outputs.

Follow-up Questions Interviewers Ask:

1. *Why does streaming token generation during inference reduce perceived user latency?*
 - **Answer:** In end-to-end inference, waiting for the model to generate the entire sequence of 200 tokens takes time. By streaming, the decoder yields the token ID immediately at the end of each decoding loop iteration. The server decodes this ID back to string text and pushes it to the client, converting sequence latency to single-token inter-token latency.
2. *Where does the final LayerNorm occur in the end-to-end inference pipeline?*
 - **Answer:** Modern Pre-LN decoders apply a final layer normalization step immediately *after* the final Transformer block layer and *before* projecting activations to vocabulary logits using the language modeling head, capping unnormalized residual summation values.
3. *How do you prevent infinite loops if the model fails to generate an EOS token?*
 - **Answer:** Production systems enforce strict safety termination limits: a maximum output tokens parameter (e.g. `max_new_tokens = 512`) and a context ceiling (e.g., stopping when total context length reaches the model's physical window limit of 8192).

Module 12: LLM Foundations Master Interview Prep & Cheatsheet

This module consolidates all formulas, computational complexities, and model architecture trade-offs from the LLM Foundations curriculum into a single master study guide for Machine Learning and AI Engineer interviews.

1. Master Formula Sheet

1. TF-IDF Weights

$$\text{TF-IDF}(t, d, D) = \frac{f_{t,d}}{\sum_{t'} f_{t',d}} \times \log \left(\frac{|D|}{1 + |\{d' \in D : t \in d'\}|} \right)$$

2. Word2Vec Negative Sampling (SGNS) Loss

$$\mathcal{L}_{\text{SGNS}} = -\log \sigma(\mathbf{v}_{w_o}^T \mathbf{v}_{w_I}) - \sum_{i=1}^k \log \sigma(-\mathbf{v}_{w_i}^T \mathbf{v}_{w_I})$$

3. Bahdanau (Additive) Attention

$$e_{i,j} = \mathbf{v}_a^T \tanh(\mathbf{W}_a \mathbf{s}_{i-1} + \mathbf{U}_a \mathbf{h}_j), \quad \alpha_{i,j} = \frac{\exp(e_{i,j})}{\sum_k \exp(e_{i,k})}, \quad \mathbf{c}_i = \sum_j \alpha_{i,j} \mathbf{h}_j$$

4. Root Mean Square Normalization (RMSNorm)

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \odot \gamma, \quad \text{where } \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}$$

5. SwiGLU Gated Activation

$$\text{SwiGLU}(\mathbf{x}) = (\text{Swish}(\mathbf{x} \mathbf{W}_1) \odot \mathbf{x} \mathbf{W}_2) \mathbf{W}_3, \quad \text{where } \text{Swish}(a) = \frac{a}{1 + e^{-a}}$$

6. Scaled Dot-Product Attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V}$$

7. Rotary Positional Embeddings (RoPE 2D Slice)

$$\mathbf{R}_{\Theta, m}^2 \mathbf{x} = \begin{bmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

8. Attention with Linear Biases (ALiBi)

$$a_{i,j} = \text{Softmax}\left(\frac{\mathbf{q}_i \mathbf{k}_j^T}{\sqrt{d_k}} - m \cdot |i - j|\right)$$

9. KV Cache VRAM Size (FP16 Precision)

$$\text{KV Cache VRAM Size} = 2 \times 2 \times b \times n_{\text{layers}} \times n_{\text{heads_kv}} \times d_{\text{head}} \times s_{\text{len}} \text{ bytes}$$

10. Chinchilla Scaling Compute-Optimal Training

$$D \approx 20 \cdot N, \quad C \approx 6ND \text{ FLOPS}$$

11. Perplexity (PPL)

$$\text{PPL}(X) = \exp\left(-\frac{1}{L} \sum_{t=1}^L \log P(x_t \mid x_{<t})\right)$$

2. Master Computational Complexity Sheet

Layer / Model Operation	Time Complexity	VRAM Space Complexity	Parallelizability
RNN Sequence Step	$O(L \cdot d^2)$	$O(L \cdot d)$	Sequential ($O(L)$ latency)
Self-Attention block	$O(L^2 \cdot d + L \cdot d^2)$	$O(L^2 \cdot h + L \cdot d)$	Fully parallel ($O(1)$ latency)
KV Cache Decoding Step	$O(L \cdot d + d^2)$	$O(b \cdot L \cdot d)$	Sequential (memory-bandwidth bound)
BPE Encoding Lookup	$O(L \cdot \log V)$	$O(V)$	Parallelizable
Top-p Nucleus Sort	$O(V \cdot \log V)$	$O(V)$	Non-parallelizable
FlashAttention Forward	$O(L^2 \cdot d)$	$O(L \cdot d)$ (HBM writes reduced)	Fully parallel (high MFU cache tiling)

3. Evolutionary Architectures Matrix

Architecture	GQA	RMSNorm	SwiGLU	Positional Encoding	Key Innovation
GPT-3	No	No (LayerNorm)	No (GELU)	Learned Absolute	Decoders scale to 175B
BERT	No	No (LayerNorm)	No (GELU)	Learned Absolute	Bidirectional MLM encoding
T5	No	No (LayerNorm)	No (ReLU)	Relative Bias	Unified Text-to-Text format
Llama-3	Yes	Yes	Yes	Rotary (RoPE)	Pre-normalization stability
Mistral	Yes	Yes	Yes	Rotary (RoPE)	Sliding Window Attention
DeepSeek	MLA	Yes	Yes (MoE)	Rotary (RoPE)	Latent Attention / Multi-token prediction

4. Interview Questions & Production Trade-offs

What problem does this solve?

Consolidates theoretical ML math intuition, complexity tables, and system bottlenecks into a single quick-reference sheet to enable rapid interview revision.

Why was it introduced?

Cracking high-level AI Engineer interviews requires connecting algorithmic choices (e.g. BPE vs WordPiece, GQA vs MHA) directly to production system constraints (VRAM, memory bandwidth, latency).

What are its limitations?

Cheatsheets summarize formulas and complexity classes, but cannot replace code-level debugging intuition (e.g., verifying PyTorch tensor sizes and gradients).

Computational Complexity (Time & Memory)

- **Time Complexity:** $O(1)$ lookup reference.
- **Memory Complexity:** $O(1)$ space.

Component Variable Denotation Legend

- N : Parameter count.
- L : Sequence token length.
- $|V|$: Vocabulary size.
- d : Hidden model dimension size.
- b : Batch size.
- n_{layers} : Transformer layer count.
- $n_{\text{heads_kv}}$: Key-Value heads count.
- d_{head} : Dimension per attention head (d/h).
- C : Total FLOPS compute budget.

Production Use Cases:

- Estimating cluster VRAM requirements during deployment scaling meetings.
- Structuring algorithmic justifications for architectural choices in engineering design reviews.

Follow-up Questions Interviewers Ask:

1. *Why does the backward pass of a parameter cost twice as much compute ($4N$) as the forward pass ($2N$) during LLM training?*
- **Answer:** The forward pass requires one matrix multiplication per layer to project inputs ($2N$ operations). The backward pass requires two separate matrix multiplications: one to calculate the gradients of the loss with respect to activations of the layer (for backpropagation down the stack), and a second to calculate the gradients of the loss with respect to the layer weights (to update parameters), resulting in $4N$ operations ($2 \times 2N$).
2. *Describe the exact mathematical relationship between Perplexity and Cross-Entropy Loss.*
- **Answer:** Perplexity is the exponentiated average cross-entropy loss: $\text{PPL} = e^{H(X)}$. If a model predicts tokens uniformly at random over a vocabulary of size $|V|$, its cross-entropy loss is $\log |V|$, and its perplexity is $e^{\log |V|} = |V|$. A lower loss scales perplexity down toward 1.0, indicating absolute certainty.
3. *Why does memory-bandwidth bound decode processing cause low GPU utilization (low SM occupancy) during single-user LLM inference?*
- **Answer:** Standard GPUs are optimized for massive parallel floating-point computation (high FLOPS). During single-user decoding, only a single token is generated per step. The GPU must load gigabytes of weights from HBM to SRAM for that single token. Since memory transfer speed is orders of magnitude slower than tensor core compute speed, the GPU execution units sit idle waiting for memory loads, leading to extremely low compute core utilization (5% – 15%).

Module 13: LLM Foundations Interview Questions & Answers

1. Introduction

Purpose of this Interview Guide

This guide is a curated compendium of 50 frequently asked interview questions on **LLM Foundations**, designed for candidates preparing for Data Science (DS), Applied AI Engineer, Generative AI (GenAI) Engineer, and Machine Learning (ML) Engineer interviews. It connects conceptual intuition and mathematical rigor directly to the engineering trade-offs, system bottlenecks, and memory constraints found in production systems.

Target Companies

These questions are commonly asked in technical design sessions and machine learning intuition rounds at top product companies (such as Google, Meta, Apple, OpenAI, Anthropic) and tier-1 AI startups.

How to Use This Guide Effectively

1. **First Pass:** Read the *Short Interview Answer* and memorize the *Key Interview Points* to master the quick, high-impact phrasing expected during screening rounds.
 2. **Deep Dive:** Study the *Technical Intuition & Complexity* sections to master formulas, complexity classes, and numerical hand calculations.
 3. **Production Perspective:** Focus on the *Production Perspective & Trade-offs* to answer architectural and systems design questions.
 4. **Mock Prep:** Practice answering the *Follow-up Questions* and review the *Common Mistakes* to ensure you don't fall into standard candidate traps.
-

2. Frequently Asked Interview Questions

Evolution of NLP

Question 1: Why did Transformers replace RNNs and LSTMs?

Short Interview Answer (30–60 seconds)

Transformers replaced RNNs and LSTMs primarily because they solve the sequential execution bottleneck. RNNs process tokens one by one, which makes their training time complexity sequential— $O(L)$ step latency for sequence length L —preventing effective parallelization on modern GPU accelerators. Transformers use self-attention, which allows them to compute representation vectors for all tokens in parallel, reducing training step latency to $O(1)$. Furthermore, Transformers do not suffer from vanishing or exploding gradients over long-range dependencies, as they model relationships between all token pairs directly.

Key Interview Points

- **Sequential Bottleneck:** RNN cell dependency on h_{t-1} prevents GPU hardware parallelization.
- **$O(1)$ Training Latency:** Transformers process all tokens simultaneously during training.
- **Direct Long-Range Modeling:** Attention path length between any two tokens is $O(1)$, compared to $O(L)$ for RNNs.
- **Hardware-Friendly:** Highly optimized matrix multiplications (GEMM operations) on Tensor Cores.

Technical Intuition & Complexity (where applicable)

- **Time Complexity:**
- RNN Training step latency: $O(L)$ due to the recurrence relation $h_t = f(h_{t-1}, x_t)$.
- Transformer Training step latency: $O(1)$ (constant parallel step time on GPU).
- **Long-range Dependency Path Length:**
- RNN path length: $O(L)$ sequence steps (gradients must backpropagate through time, leading to vanishing gradients).
- Transformer path length: $O(1)$ steps (direct attention linking).

Production Perspective & Trade-offs

In production training, GPU utilization (Model FLOPS Utilization, or MFU) is highly dependent on batching and parallel execution. RNN architectures achieve very low GPU utilization because they cannot leverage parallel execution across the sequence dimension. Transformers utilize matrix-multiply engines (e.g., NVIDIA Tensor Cores) at high efficiency because the self-attention operation $\text{Softmax}(\mathbf{QK}^T)\mathbf{V}$ is formulated as massive parallel matrix multiplications. The trade-off is during *inference*: autoregressive generation in Transformers behaves sequentially, requiring KV Caching to reduce step complexity.

Follow-up Questions

- **Follow-up:** *If Transformers are so parallelizable, why is Transformer inference still sequential?*
- **Answer:** During inference (decoding), we generate tokens one by one autoregressively. Each new token depends on the previously generated tokens. Thus, while training is fully parallelized via causal masking, inference remains sequential ($O(L)$ steps).

Common Mistakes

- **Mistake:** Saying Transformers have lower computational complexity than RNNs. In fact, Transformers have quadratic complexity with sequence length $O(L^2 \cdot d)$, whereas RNNs have linear complexity $O(L \cdot d^2)$. Transformers are preferred because of their *parallelizability* and *empirical capability*, not lower overall training FLOP counts.
-

Question 2: What are the limitations of RNNs and Seq2Seq models?

Short Interview Answer (30–60 seconds)

The main limitations of RNNs and standard Seq2Seq models are the **sequential computation bottleneck** and the **information bottleneck (vector bottleneck)**. In Seq2Seq models, the encoder is forced to compress the semantic representation of an entire input sequence of arbitrary length into a single, fixed-size context vector $\mathbf{c}$ before passing it to the decoder. This causes severe information loss on inputs longer than the capacity of $\mathbf{c}$. Additionally, RNNs suffer from vanishing and exploding gradients when backpropagating through long sequences, causing them to forget early context.

Key Interview Points

- **Vector Bottleneck:** Compressing variable-length input into a fixed-dimensional context vector $\mathbf{c} \in \mathbb{R}^d$.
- **Vanishing/Exploding Gradients:** Recurrent weight multiplication $\mathbf{W}_{hh}^t$ decays or blows up gradients over long steps.
- **No Out-of-Order Parallelization:** The hidden state h_t cannot be computed until h_{t-1} is resolved.

Technical Intuition & Complexity (where applicable)

Consider an RNN hidden state update:

$$h_t = \tanh(\mathbf{W}_{hh}h_{t-1} + \mathbf{W}_{ih}x_t)$$

The derivative of the loss at step T with respect to the hidden state at step t is:

$$\frac{\partial \mathcal{L}}{\partial h_t} = \frac{\partial \mathcal{L}}{\partial h_T} \prod_{k=t+1}^T \frac{\partial h_k}{\partial h_{k-1}} = \frac{\partial \mathcal{L}}{\partial h_T} \prod_{k=t+1}^T \text{diag}(1 - \tanh^2(\cdot)) \mathbf{W}_{hh}^T$$

If the eigenvalues of $\mathbf{W}_{hh}$ are less than 1, the gradient vanishes exponentially as $(T - t) \rightarrow \infty$. If they are greater than 1, the gradient explodes.

Production Perspective & Trade-offs

Seq2Seq models without attention fail to translate or summarize long documents because the translation quality drops off rapidly for sequences longer than 30–40 tokens. To handle longer contexts in production before Transformers, engineers had to increase the size of the context vector $\mathbf{c}$, which dramatically increased parameter counts and memory usage without addressing the fundamental vanishing gradient problem.

Follow-up Questions

- **Follow-up:** *How did LSTMs attempt to solve the vanishing gradient problem, and why did they still fail for extremely long contexts?*
- **Answer:** LSTMs introduced a cell state C_t and additive gating (the forget gate), which allows gradients to flow linearly through the cell state without multiplying by a weight matrix at every step. However, the squashing functions ($\tanh$) and gating still degrade information flow over hundreds of steps, and they do not solve the sequential processing bottleneck.

Common Mistakes

- **Mistake:** Believing that LSTMs completely eliminate the vanishing gradient problem. LSTMs mitigate it, but they still struggle to connect dependencies spaced more than a few hundred tokens apart.
-

Question 3: Compare Word2Vec, GloVe, and FastText.

Short Interview Answer (30–60 seconds)

Word2Vec, GloVe, and FastText are static, non-contextual word embedding techniques. **Word2Vec** is a predictive neural model that uses local context windows via CBOW or Skip-Gram with Negative Sampling (SGNS) to learn word representations. **GloVe** (Global Vectors) is a matrix factorization model that trains on global co-occurrence statistics across the entire corpus. **FastText** extends Word2Vec by representing words as a bag of character n -grams, allowing it to generate embeddings for Out-of-Vocabulary (OOV) words at runtime.

Key Interview Points

- **Word2Vec:** Local context window, predictive training (CBOW/Skip-Gram), negative sampling.
- **GloVe:** Global co-occurrence matrix factorization, uses global corpus statistics directly.
- **FastText:** Character-level n -gram representations, handles Out-of-Vocabulary (OOV) tokens, handles morphological variations.

- **Static Bottleneck:** All three assign a single fixed vector to a word, failing to resolve context-dependent meanings (e.g., "bank").

Technical Intuition & Complexity (where applicable)

- **Word2Vec Skip-Gram (SGNS) Loss:**

$$\mathcal{L}_{\text{SGNS}} = -\log \sigma(\mathbf{v}_{w_O}^T \mathbf{v}_{w_I}) - \sum_{i=1}^k \log \sigma(-\mathbf{v}_{w_i}^T \mathbf{v}_{w_I})$$

- **GloVe Loss:**

$$\mathcal{L} = \sum_{i,j=1}^{|V|} f(X_{i,j}) \left(\mathbf{w}_i^T \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{i,j} \right)^2$$

where $X_{i,j}$ is the co-occurrence count between word i and word j , and $f(X_{i,j})$ is a weighting function to prevent frequent words from dominating.

- **FastText Word Vector construction:**

$$\mathbf{v}_{\text{word}} = \sum_{g \in G_{\text{word}}} \mathbf{z}_g$$

where G_{word} is the set of character n -grams of the word plus the word itself, and $\mathbf{z}_g$ is the learned vector for n -gram g .

Production Perspective & Trade-offs

- **Out-of-Vocabulary (OOV):** In production pipelines, Word2Vec and GloVe map unseen words to a single `<UNK>` token vector, losing all semantic information. FastText constructs vectors for OOV words (e.g., "sub-engineered") from sub-words, making it highly robust for clinical, legal, or domain-specific text with many compound words.
- **Storage/Memory Footprint:** FastText models are significantly larger than Word2Vec or GloVe because they must store representations for millions of character n -grams.

Follow-up Questions

- **Follow-up:** *Why did GloVe use global matrix factorization instead of Word2Vec's local windows?*
- **Answer:** Word2Vec does not directly utilize global statistics; it repeatedly samples context windows, which wastes information from rare co-occurrences. GloVe utilizes global counts directly, making training faster on large corpora and capturing global structural relationships more efficiently.

Common Mistakes

- **Mistake:** Claiming Word2Vec, GloVe, or FastText generate different vectors for "bank" based on the sentence it appears in. They are **static** embeddings. The vector for "bank" is identical regardless of context; context resolution was only solved by contextual embeddings like ELMo and Transformers.
-

Question 4: Why was the Attention mechanism introduced?

Short Interview Answer (30–60 seconds)

The Attention mechanism was introduced to solve the **information bottleneck** in Seq2Seq models. Instead of forcing the encoder to compress an entire source sequence into a single, fixed-size context vector, Attention allows the decoder to look back at all hidden states of the encoder at each decoding step. The decoder dynamically computes a weighted sum of the encoder's hidden states, focusing on the most relevant parts of the input sequence for the current output token.

Key Interview Points

- **Eliminating the Compression Bottleneck:** Retains all encoder hidden states $h_1, \dots, h_L$ rather than discarding them.
- **Dynamic Alignment:** Computes attention weights $\alpha_{i,j}$ representing similarity between current decoder state and all encoder states.
- **Gradient Highway:** Creates short, direct backpropagation paths from the decoder directly to early encoder hidden states.

Technical Intuition & Complexity (where applicable)

In Bahdanau (Additive) Attention:

1. **Alignment Scores:** $e_{i,j} = \mathbf{v}_a^T \tanh(\mathbf{W}_a \mathbf{s}_{i-1} + \mathbf{U}_a \mathbf{h}_j)$

2. **Attention Weights (Softmax):** $\alpha_{i,j} = \frac{\exp(e_{i,j})}{\sum_{k=1}^L \exp(e_{i,k})}$

3. **Context Vector:** $\mathbf{c}_i = \sum_{j=1}^L \alpha_{i,j} \mathbf{h}_j$

Here, $\mathbf{s}_{i-1}$ is the previous decoder hidden state, and $\mathbf{h}_j$ is the j -th encoder hidden state. The path length for gradients between any input token j and decoder token i is direct ($O(1)$ operations), bypassing the recurrence chain.

Production Perspective & Trade-offs

Adding attention to Seq2Seq models significantly improves performance on long sequences but introduces a computational trade-off. At each decoding step, the decoder must calculate attention weights over all L encoder tokens. This makes the decoding time complexity $O(L \cdot U)$ where U is the target sequence length, introducing higher latency and memory requirements.

Follow-up Questions

- **Follow-up:** *What is the difference between Additive (Bahdanau) and Multiplicative (Luong) attention?*
- **Answer:** Additive attention computes scores using a single-hidden-layer feedforward network: $\mathbf{v}_a^T \tanh(\mathbf{W}_a \mathbf{s} + \mathbf{U}_a \mathbf{h})$. Multiplicative attention uses a matrix dot-product: $\mathbf{s}^T \mathbf{W}_a \mathbf{h}$. Multiplicative attention is faster and more space-efficient in practice because it can be implemented as highly optimized matrix multiplications.

Common Mistakes

- **Mistake:** Thinking that the attention mechanism in Seq2Seq models is the same as Self-Attention in Transformers. In Seq2Seq, attention is **cross-attention** (decoder attends to encoder states). In Transformers, self-attention allows the encoder (or decoder) to attend to its *own* states.

Transformer Architecture

Question 5: Explain the Transformer architecture.

Short Interview Answer (30–60 seconds)

The Transformer is an encoder-decoder architecture built entirely on self-attention mechanisms, completely eliminating recurrence and convolutions. The **Encoder** processes the input sequence in parallel to generate contextualized representations. The **Decoder** generates the output sequence autoregressively, token by token. Both components use multi-head self-attention to capture contextual relationships, positional encodings to inject sequence order, feed-forward networks for feature transformation, and residual connections with layer normalization to stabilize deep gradient flow.

Key Interview Points

- **No Recurrence:** Fully parallelizable training across the sequence dimension.
- **Multi-Head Self-Attention (MHA):** Allows the model to jointly attend to information from different representation subspaces.
- **Positional Encoding:** Injects sequence token order mathematically since attention is permutation-invariant.
- **Residual Links & LayerNorm:** Enables training deep stacks (e.g., 24+ layers) without gradient degradation.

Technical Intuition & Complexity (where applicable)

The Transformer consists of:

- **Scaled Dot-Product Attention:**

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V}$$

- **Feed-Forward Network (FFN):**

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$

- **Complexity:** Self-attention time and space complexity is $O(L^2 \cdot d)$ where L is sequence length and d is the model dimension. FFN complexity is $O(L \cdot d^2)$ (specifically, $O(L \cdot d \cdot d_{\text{ffn}})$ where $d_{\text{ffn}} \approx 4d$).

Production Perspective & Trade-offs

Because the self-attention layer has $O(L^2)$ space complexity, the VRAM consumption scales quadratically with sequence length. During training, storing the $L \times L$ attention matrix activations for backpropagation is the primary memory bottleneck. In production inference, this quadratic dependency restricts the context window size and requires specialized architectures like FlashAttention or model architectures like Grouped-Query Attention (GQA).

Follow-up Questions

- **Follow-up:** *Why does the Transformer have residual connections?*
- **Answer:** Residual connections ($x + \text{SubLayer}(x)$) allow gradients to flow directly through the network during the backward pass. Without them, the gradient would be repeatedly scaled by the attention and FFN weight matrices, leading to vanishing or exploding gradients in deep networks.

Common Mistakes

- **Mistake:** Forgetting that self-attention is permutation-invariant. If you scramble the order of words in an input sentence, the self-attention output vectors remain identical (just scrambled). Positional encoding is *mandatory* to break this symmetry.
-

Question 6: Compare Encoder-only, Decoder-only, and Encoder-Decoder models.

Short Interview Answer (30–60 seconds)

- **Encoder-only models** (e.g., BERT) use bidirectional self-attention, meaning every token can attend to every other token. They are optimized for natural language understanding and feature extraction

(e.g., classification, NER).

- **Decoder-only models** (e.g., GPT, Llama) use unidirectional (causally masked) self-attention, meaning tokens can only attend to past tokens. They are optimized for autoregressive text generation.
- **Encoder-Decoder models** (e.g., T5, BART) use a bidirectional encoder paired with a unidirectional decoder via cross-attention, making them ideal for sequence-to-sequence translation and summarization tasks.

Key Interview Points

- **Encoder-only:** Bidirectional attention, masked language modeling (MLM), NLU tasks.
- **Decoder-only:** Causal/unidirectional attention masking, autoregressive generation, NLG tasks.
- **Encoder-Decoder:** Bidirectional encoder + causally masked decoder connected by cross-attention.

Technical Intuition & Complexity (where applicable)

Architecture Type	Attention Masking	Output Generation	Typical Complexity (Training)	Example
Encoder-only	Unmasked (Full $L \times L$ bidirectional grid)	Parallel (all tokens at once)	$O(L^2 \cdot d)$	BERT
Decoder-only	Lower-triangular causal mask ($\alpha_{i,j} = -\infty$ for $j > i$)	Autoregressive (token-by-token)	$O(L^2 \cdot d)$	GPT, Llama
Encoder-Decoder	Encoder: Unmasked; Decoder: Causal	Autoregressive	$O(L_{\text{enc}}^2 + L_{\text{enc}}L_{\text{dec}} + L_{\text{dec}}^2)$	T5, BART

Production Perspective & Trade-offs

- **Inference Optimization:** Decoder-only models have dominated modern GenAI because they scale extremely well and are highly flexible for prompt-based tasks. However, decoder-only inference requires maintaining a **KV Cache** which consumes massive amounts of GPU memory.
- **NLU Performance:** For tasks like classification or vector embedding generation, Encoder-only models are significantly more compute-efficient than Decoder-only models because they capture bidirectional context in a single forward pass without sequential decoding overhead.

Follow-up Questions

- **Follow-up:** *Can you use an Encoder-only model for text generation?*
- **Answer:** Technically yes, but it is highly inefficient. Since it was not trained with a causal mask, you would have to mask tokens one by one and run the entire model iteratively, which is computationally expensive compared to autoregressive decoders.

Common Mistakes

- **Mistake:** Thinking that decoder-only models cannot capture bidirectional context. While they process tokens unidirectionally during *generation*, the prompt (input context) is processed all at once, allowing tokens within the prompt to attend to other prompt tokens.
-

Question 7: Why does GPT use only the decoder?

Short Interview Answer (30–60 seconds)

GPT is a decoder-only architecture because it is designed specifically for **autoregressive language modeling**—predicting the next token given the historical context. Using only the decoder eliminates the overhead of a separate encoder and the cross-attention blocks that connect them. This simplifies the parameter allocation and allows the entire context (both input prompt and generated output) to be processed in a unified, causally-masked sequence, which maximizes parameter efficiency and training throughput for large-scale pretraining.

Key Interview Points

- **Autoregressive Matching:** Next-token prediction aligns naturally with causal masking.
- **No Cross-Attention Layers:** Saves parameter count and reduces computational overhead.
- **Unified Context representation:** Input and output are treated as a single continuous sequence.

Technical Intuition & Complexity (where applicable)

In an Encoder-Decoder model, the decoder block contains three sub-layers: Self-Attention, Cross-Attention, and FFN. In a Decoder-only model like GPT, the Cross-Attention layer is removed.

The model dimension d and hidden weights are structured identically, but parameters are saved because the cross-attention projection weights ($\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ mapping decoder states to encoder outputs) are omitted. This reduces layer-wise parameter counts by approximately 25-33% compared to a full encoder-decoder block of the same hidden size.

Production Perspective & Trade-offs

From a production standpoint, training a unified decoder-only model is highly efficient for data pipelining. You do not need to manage two different sequence lengths (encoder input and decoder output) or separate learning dynamics. The downside is during inference: decoder-only models must process the entire input prompt (prefill phase) and then generate tokens one-by-one (decode phase), requiring complex memory management systems (like vLLM and PagedAttention) to manage the KV Cache.

Follow-up Questions

- **Follow-up:** *Does a decoder-only model lose representation capacity by not having a bidirectional encoder?*
- **Answer:** Empirically, at scale, no. The decoder-only architecture compensates for the unidirectional constraint by scaling parameter counts and training data, which allows it to learn complex contextual representations that match or exceed encoder-decoder architectures on general tasks.

Common Mistakes

- **Mistake:** Claiming GPT is decoder-only because encoders are hard to parallelize. Both encoders and decoders are fully parallelizable during training. The choice is driven by task alignment (autoregressive generation) and parameter efficiency.
-

Question 8: Explain the role of Feed Forward Networks.

Short Interview Answer (30–60 seconds)

The Feed-Forward Network (FFN) in a Transformer block acts as a **channel-wise (feature-wise) processing layer**. While the self-attention layer is responsible for routing information *across* the sequence (token-to-token mixing), the FFN processes information *within* each token representation individually (dimension-to-dimension mixing). The FFN introduces non-linear activations (like ReLU, GELU, or SwiGLU) and accounts for roughly two-thirds of the parameters in a standard Transformer block, serving as the primary "knowledge storage" for semantic and factual relationships.

Key Interview Points

- **Token-wise Isolation:** The FFN is applied to each token vector x_i independently and identically (no cross-token interactions).
- **Dimension Mixing:** Shifts representation space to project features (typically expanding from d to $4d$ and back to d).
- **Parametric Capacity:** FFN holds $\sim 66\%$ of the model's total parameter count in classic Transformers.

Technical Intuition & Complexity (where applicable)

The standard FFN equation is:

$$\text{FFN}(\mathbf{x}) = \text{Activation}(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$

- $\mathbf{W}_1 \in \mathbb{R}^{d \times 4d}$ (Projects token embedding to a higher dimension)
- $\mathbf{W}_2 \in \mathbb{R}^{4d \times d}$ (Projects back to model dimension)

- **Time/Space Complexity:** $O(L \cdot d^2)$ per layer, scaling linearly with sequence length L and quadratically with model dimension d .

Production Perspective & Trade-offs

Because the FFN weights account for the majority of the model's parameters, they are the primary source of memory footprint during inference. In memory-constrained environments, FFN weights are prime targets for quantization (e.g., FP8 or INT4) to fit the model into GPU VRAM. Modern designs like Mixture of Experts (MoE) replace this heavy dense FFN with multiple sparse FFNs (experts) to scale parameter count without increasing active compute (FLOPs) per token.

Follow-up Questions

- **Follow-up:** *Why do we expand the FFN dimension to $4d$ instead of keeping it at d ?*
- **Answer:** Expanding the hidden dimension to $4d$ provides the model with sufficient capacity to perform non-linear transformations and store factual knowledge. It acts as a memory storage system where different dimensions learn to activate for specific concepts.

Common Mistakes

- **Mistake:** Thinking that the FFN mixes context across different words. The FFN has absolutely no sequence-dimension parameters; it cannot share information between token x_i and token x_j . All sequence-level interaction happens strictly in the attention layers.
-

Question 9: Why are Residual Connections important?

Short Interview Answer (30–60 seconds)

Residual connections (skip connections) are critical because they solve the **vanishing gradient problem** during training. By adding the input of a layer directly to its output ($x + f(x)$), they create a mathematical "gradient highway." This allows gradients to propagate backward through the network layers during backpropagation without being degraded by repeated matrix multiplications, enabling stable training of deep networks with dozens or hundreds of layers.

Key Interview Points

- **Additive Gradient Path:** Derivation of gradient contains an additive term $+1$ that does not decay.
- **Feature Preservation:** Prevents lower-level token representations from being destroyed by complex attention transformations.
- **Loss Landscape Smoothing:** Mathematically smooths the optimization landscape, preventing sharp local minima.

Technical Intuition & Complexity (where applicable)

Let a residual layer be represented as:

$$x_l = x_{l-1} + \mathcal{F}(x_{l-1}, \mathcal{W}_l)$$

Recursively expanding this to any deeper layer L :

$$x_L = x_l + \sum_{i=l}^{L-1} \mathcal{F}(x_i, \mathcal{W}_{i+1})$$

During backpropagation, the gradient of the loss $\mathcal{L}$ with respect to the activation x_l is:

$$\frac{\partial \mathcal{L}}{\partial x_l} = \frac{\partial \mathcal{L}}{\partial x_L} \frac{\partial x_L}{\partial x_l} = \frac{\partial \mathcal{L}}{\partial x_L} \left(1 + \frac{\partial}{\partial x_l} \sum_{i=l}^{L-1} \mathcal{F}(x_i, \mathcal{W}_{i+1}) \right)$$

The term 1 ensures that even if the gradient of the sum term approaches zero (due to vanishing weights or saturation), the gradient $\frac{\partial \mathcal{L}}{\partial x_L}$ is still propagated directly back to layer l .

Production Perspective & Trade-offs

Residual connections require the input and output dimensions of the sub-layer (attention or FFN) to be identical. If the dimensions do not match, a projection matrix $\mathbf{W}_s$ must be introduced ($x\mathbf{W}_s + f(x)$), which adds extra computational cost and parameters. This is why Transformers maintain a constant hidden dimension d throughout the stack.

Follow-up Questions

- **Follow-up:** *What is the difference between Pre-LayerNorm and Post-LayerNorm regarding residual connections?*
- **Answer:** In Post-LN (original Transformer), LayerNorm is placed on the residual path: $x_l = \text{LN}(x_{l-1} + f(x_{l-1}))$. This makes gradients at early layers smaller, requiring a learning rate warmup to prevent divergence. In Pre-LN (modern LLMs), LayerNorm is placed on the sub-layer branch: $x_l = x_{l-1} + f(\text{LN}(x_{l-1}))$. This keeps the main residual path completely clean, enabling stable training without aggressive warmup.

Common Mistakes

- **Mistake:** Believing residual connections are parameter-heavy. They are simple element-wise addition operations ($O(d)$ complexity) and do not introduce any new parameters, provided the input and output dimensions are already aligned.
-

Question 10: LayerNorm vs RMSNorm.

Short Interview Answer (30–60 seconds)

LayerNorm and RMSNorm are normalization techniques used to stabilize training. **LayerNorm** centers the activations of a layer by subtracting their mean, and then scales them by their standard deviation.

RMSNorm (Root Mean Square Normalization) simplifies this by skipping the centering step (mean subtraction) and normalizing activations solely by their root mean square. This reduces computational overhead by 7% – 10% per layer during inference/training while achieving identical convergence and model accuracy.

Key Interview Points

- **LayerNorm (LN)**: Centering (mean subtraction) + scaling (std dev division) + affine parameters (γ, β).
- **RMSNorm**: Scaling only (Root Mean Square division) + affine parameter (γ).
- **Computational Saving**: RMSNorm removes the mean computation, reducing memory accesses and improving GPU kernel execution times.
- **Adoption**: RMSNorm is the default choice in modern LLM architectures (Llama, Mistral, Gemma).

Technical Intuition & Complexity (where applicable)

For a vector $\mathbf{x} \in \mathbb{R}^d$:

- **LayerNorm**:

$$\text{LN}(\mathbf{x}) = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \gamma + \beta, \quad \text{where } \mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad \sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2$$

- **RMSNorm**:

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \odot \gamma, \quad \text{where } \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}$$

RMSNorm removes the need to calculate μ , reducing the number of mathematical reductions over the vector from two passes to one.

Production Perspective & Trade-offs

At production scale, LayerNorm can become a memory-bandwidth bottleneck on GPUs because it requires reading the tensor, computing the mean, reading again to compute variance, and then writing back.

RMSNorm allows GPU kernel developers to write highly optimized, single-pass Triton or CUDA kernels

that scale inputs instantly. The parameter reduction is small (eliminating the β bias vector of size d), but the execution speedup is significant.

Follow-up Questions

- **Follow-up:** *Why can we safely discard the mean centering step (μ) in RMSNorm?*
- **Answer:** Empirically, the shift-invariance provided by subtracting the mean does not contribute significantly to training stability. The scaling property (scale-invariance) is what prevents activation magnitudes from exploding or vanishing across layers, which RMSNorm retains.

Common Mistakes

- **Mistake:** Thinking RMSNorm has fewer parameters because it doesn't scale inputs. RMSNorm still uses a learnable scaling parameter γ (gamma vector of size d). It only discards the bias parameter β (beta vector) and the mean subtraction logic.
-

Question 11: Why do Llama and Mistral use SwiGLU?

Short Interview Answer (30–60 seconds)

Llama and Mistral use **SwiGLU** (Swish Gated Linear Unit) in their FFN layers because it provides faster convergence and superior representation capacity compared to standard activations like ReLU or GELU. SwiGLU is a gated activation function where one projection of the input is passed through a Swish function and then multiplied element-wise with a second projection. Although SwiGLU requires three projection matrices instead of two (increasing the parameter count of the FFN block by $\sim 50\%$ for the same hidden size), models adapt by scaling down the intermediate dimension to maintain the target parameter budget while capturing richer semantic representations.

Key Interview Points

- **Gating Mechanism:** Uses two parallel linear projections, gating one with the other.
- **Swish Activation:** Smooth, non-monotonic function that improves gradient flow for small inputs.
- **Intermediate Dimension Adjustment:** Hidden dimension is scaled to $d_{ffn} \approx \frac{8}{3}d$ (instead of $4d$) to balance parameter counts.

Technical Intuition & Complexity (where applicable)

For an input vector $\mathbf{x}$:

- **Standard FFN (GELU):**

$$\text{FFN}_{\text{GELU}}(\mathbf{x}) = (\text{GELU}(\mathbf{x}\mathbf{W}_1)) \mathbf{W}_2$$

- SwiGLU FFN:

$$\text{FFN}_{\text{SwiGLU}}(\mathbf{x}) = (\text{Swish}(\mathbf{x}\mathbf{W}_1) \odot \mathbf{x}\mathbf{W}_2) \mathbf{W}_3$$

where $\text{Swish}(a) = a \cdot \sigma(\beta a) = \frac{a}{1+e^{-\beta a}}$ (typically $\beta = 1$).

- Computational Complexity:

- GELU FFN: 2 projection matrices ($\mathbf{W}_1 \in \mathbb{R}^{d \times 4d}$, $\mathbf{W}_2 \in \mathbb{R}^{4d \times d}$).
- SwiGLU FFN: 3 projection matrices ($\mathbf{W}_1, \mathbf{W}_2 \in \mathbb{R}^{d \times d_{\text{ffn}}}$, $\mathbf{W}_3 \in \mathbb{R}^{d_{\text{ffn}} \times d}$).

Production Perspective & Trade-offs

SwiGLU requires more tensor memory write/read operations during the forward pass due to the element-wise multiplication ($\odot$) of two large matrices. However, because SwiGLU is mathematically smoother, it stabilizes training dynamics at large scales, preventing loss spikes. The trade-off is the extra matrix multiplication, which must be compiled into fused CUDA kernels (e.g., via FlashAttention or vLLM backends) to avoid GPU memory overhead.

Follow-up Questions

- **Follow-up:** How do we set the intermediate dimension d_{ffn} to match the parameter count of a standard $4d$ GELU FFN?
- **Answer:** A standard FFN has $2 \times d \times 4d = 8d^2$ parameters. A SwiGLU FFN has $3 \times d \times d_{\text{ffn}} = 3d \cdot d_{\text{ffn}}$ parameters. To match the parameter count, we solve $3d \cdot d_{\text{ffn}} = 8d^2 \implies d_{\text{ffn}} = \frac{8}{3}d \approx 2.67d$.

Common Mistakes

- **Mistake:** Assuming SwiGLU FFN has the same intermediate dimension multiplier ($4d$) as classic GELU FFNs. If you use a $4d$ intermediate dimension for SwiGLU, your model will have $12d^2$ parameters in the FFN, making it 50% larger and slower than intended.

Self-Attention

Question 12: Explain Self-Attention.

Short Interview Answer (30–60 seconds)

Self-Attention is an attention mechanism that relates different positions of a single sequence to compute a contextualized representation of the sequence. For each token in the sequence, the model computes Query, Key, and Value vectors. It calculates similarity scores by taking the dot product of the token's Query

with the Keys of all other tokens. These scores are scaled, normalized via Softmax to form an attention distribution, and used as weights to compute a weighted sum of the Value vectors. This allows the model to dynamically route information across the sequence, capturing contextual relationships regardless of distance.

Key Interview Points

- **Cross-Position Association:** Focuses on relationships within a single sequence.
- **Dynamic Routing:** Content-based routing where weights are computed on-the-fly based on input.
- **Constant Path Length:** Relates any two tokens directly with $O(1)$ operations, bypassing recurrence.
- **Permutation Invariance:** Treats the sequence as an unordered bag of tokens without positional encoding.

Technical Intuition & Complexity (where applicable)

Self-Attention is defined as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V}$$

- **Time Complexity:** $O(L^2 \cdot d + L \cdot d^2)$ where L is sequence length and d is model dimension.
- **Space Complexity:** $O(L^2 \cdot h + L \cdot d)$ where h is the number of heads. The $L \times L$ attention matrix is the primary bottleneck.

Production Perspective & Trade-offs

In production inference, the $O(L^2)$ computational and memory cost of self-attention is the primary bottleneck for scaling context windows. For long sequences (e.g., 32k+ tokens), the activation memory required to hold the attention matrix exceeds the parameters of the model itself. Technologies like **FlashAttention** optimize this by computing Softmax in tiles on fast GPU SRAM, avoiding the slow roundtrips of writing the large $L \times L$ matrix back to HBM (High Bandwidth Memory).

Follow-up Questions

- **Follow-up:** *What is the difference between Self-Attention and Cross-Attention?*
- **Answer:** In self-attention, $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ all originate from the same input sequence (e.g., encoder states attending to encoder states). In cross-attention, $\mathbf{Q}$ comes from the target sequence (decoder states) while $\mathbf{K}$ and $\mathbf{V}$ come from the source sequence (encoder outputs).

Common Mistakes

- **Mistake:** Answering that self-attention processes tokens sequentially. Self-attention is a set operation; it processes all tokens simultaneously in parallel during the forward pass.
-

Question 13: What are Query, Key, and Value?

Short Interview Answer (30–60 seconds)

Query, Key, and Value are vector representations of tokens used to compute attention.

- **Query (q):** Represents the "search term" or what the current token is looking for in other tokens.
- **Key (k):** Represents the "indexing tag" or what the token has to offer to match other queries.
- **Value (v):** Represents the "actual content" or semantic information of the token that will be merged into the output representation.

They are created by multiplying the input token representation vector $\mathbf{x}$ by three separately learned projection matrices $\mathbf{W}_Q$, $\mathbf{W}_K$, and $\mathbf{W}_V$.

Key Interview Points

- **Linear Projections:** $\mathbf{q}_i = \mathbf{x}_i \mathbf{W}_Q$, $\mathbf{k}_i = \mathbf{x}_i \mathbf{W}_K$, $\mathbf{v}_i = \mathbf{x}_i \mathbf{W}_V$.
- **Database Analogy:** Query matches against Keys to retrieve a weighted sum of Values.
- **Feature Subspaces:** Projections project the input embedding into lower-dimensional spaces optimized for specific relationships.

Technical Intuition & Complexity (where applicable)

For an input matrix $\mathbf{X} \in \mathbb{R}^{L \times d}$:

$$\mathbf{Q} = \mathbf{X} \mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X} \mathbf{W}_K, \quad \mathbf{V} = \mathbf{X} \mathbf{W}_V$$

where $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{d \times d_k}$ and $\mathbf{W}_V \in \mathbb{R}^{d \times d_v}$. Typically $d_k = d_v = d/h$, where h is the number of attention heads.

The computational complexity of these projections is $O(L \cdot d^2)$ per layer.

Production Perspective & Trade-offs

Because $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ are linear projections, they represent independent channels. During model optimization (e.g., pruning or quantization), the projection weights are highly sensitive. Quantizing $\mathbf{W}_Q$ and $\mathbf{W}_K$ to 4-bit can cause significant perplexity degradation because small perturbations in the Query-Key space lead to large changes after the exponentiation of the Softmax layer.

Follow-up Questions

- **Follow-up:** *Why do we project the input embedding into Query, Key, and Value instead of computing attention directly on the input vectors $\mathbf{X}$?*
- **Answer:** Without projection, the similarity matrix $\mathbf{X}\mathbf{X}^T$ would be fixed and symmetric. The projections break symmetry (allowing token A to attend to token B without forcing token B to attend to token A with the same strength) and allow the model to learn multiple independent interaction spaces.

Common Mistakes

- **Mistake:** Believing that Query, Key, and Value must have the same dimension as the model dimension d . In practice, they are projected to a much smaller head dimension $d_k = d/h$ (typically 64 or 128) to keep computational costs manageable.
-

Question 14: Why divide attention scores by $\sqrt{d_k}$?

Short Interview Answer (30–60 seconds)

We divide the dot products of Queries and Keys by $\sqrt{d_k}$ to prevent **Softmax saturation** (vanishing gradients). As the head dimension d_k grows large, the magnitude of the dot products between Query and Key vectors increases. This pushes the Softmax function into regions with extremely small gradients, causing the model to stop learning. Scaling by $\sqrt{d_k}$ keeps the variance of the dot products constant at 1, ensuring stable gradients during backpropagation.

Key Interview Points

- **Softmax Saturation:** High dot-product values lead to sharp Softmax outputs (near 1 and 0) where gradients are almost zero.
- **Variance Control:** Restores the variance of the dot product to 1 under the assumption of independent, unit-variance components.
- **Gradient Flow:** Keeps the gradient scale healthy throughout training.

Technical Intuition & Complexity (where applicable)

Assume components of $\mathbf{q}$ and $\mathbf{k}$ are independent random variables with mean 0 and variance 1:

- The dot product is $u = \mathbf{q} \cdot \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$.
- Mean of u : $\mathbb{E}[u] = \sum_{i=1}^{d_k} \mathbb{E}[q_i k_i] = \sum_{i=1}^{d_k} \mathbb{E}[q_i] \mathbb{E}[k_i] = 0$.
- Variance of u : Since q_i, k_i are independent, $\text{Var}(q_i k_i) = \mathbb{E}[q_i^2 k_i^2] - \mathbb{E}[q_i k_i]^2 = \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] - 0 = (1)(1) = 1$.
- Thus, the variance of the sum is $\text{Var}(u) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = d_k$.

To scale the variance of u back to 1, we divide by its standard deviation, which is $\sqrt{d_k}$:

$$\text{Var}\left(\frac{u}{\sqrt{d_k}}\right) = \frac{1}{d_k} \text{Var}(u) = \frac{d_k}{d_k} = 1$$

Production Perspective & Trade-offs

Without this scaling factor, training a large model (e.g., $d_k = 128$) would fail to converge immediately due to exploding activation values in the self-attention layer. This division is fused directly into the attention CUDA kernel to minimize memory read/writes, which is critical for throughput.

Follow-up Questions

- **Follow-up:** *What happens to the derivative of Softmax when the inputs are very large?*
- **Answer:** The derivative of Softmax with respect to its input is $\frac{\partial S_i}{\partial z_j} = S_i(\delta_{ij} - S_j)$. If one input z_i is much larger than the others, its Softmax output $S_i \approx 1$ and all other $S_j \approx 0$. In this case, the derivative approaches 0 everywhere, leading to vanishing gradients.

Common Mistakes

- **Mistake:** Thinking we scale by the sequence length L or the total model dimension d . We scale strictly by the dimension of the *individual attention head* $d_k = d/h$.
-

Question 15: Explain Causal Masking.

Short Interview Answer (30–60 seconds)

Causal Masking is a technique used in autoregressive decoders to prevent tokens from "looking ahead" into future tokens during training. It ensures that the prediction for token i depends only on the known outputs at positions less than i . It is implemented by adding a lower-triangular mask matrix to the Query-Key attention score matrix before applying Softmax, setting future token positions to $-\infty$ (which Softmax maps to an attention weight of 0).

Key Interview Points

- **Autoregressive Constraint:** Enforces the causal order that output token t depends only on $t - 1, t - 2, \dots$.
- **Lower-Triangular Mask:** Matrix $\mathbf{M}$ where $M_{ij} = 0$ for $i \geq j$, and $M_{ij} = -\infty$ for $i < j$.
- **Parallel Training:** Enables training on full sequences in a single forward pass without step-by-step sequential iterations.

Technical Intuition & Complexity (where applicable)

Before the Softmax step, we compute:

$$\mathbf{S} = \frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} + \mathbf{M}$$

where:

$$\mathbf{M}_{i,j} = \begin{cases} 0 & \text{if } i \geq j \\ -\infty & \text{if } i < j \end{cases}$$

Applying Softmax row-wise:

$$\text{Softmax}(\mathbf{S})_{i,j} = \frac{\exp(S_{i,j})}{\sum_{k=1}^L \exp(S_{i,k})}$$

For $j > i$, $\exp(S_{i,j}) = \exp(-\infty) = 0$, meaning the attention weight is exactly 0.

Production Perspective & Trade-offs

Causal masking allows us to parallelize the training process across the entire sequence dimension. However, during inference, causal masking means we only compute the attention scores for the *newly* generated token relative to the past, which is why we store the Keys and Values of past tokens in the **KV Cache** to avoid redundant computation.

Follow-up Questions

- **Follow-up:** *Why do we use causal masking during training but not during inference prefill?*
- **Answer:** During training, we have the complete target sequence, so we must mask the future to prevent the model from cheating. During inference prefill, we process the entire user prompt (which is historical context) in parallel, so we allow prompt tokens to attend bidirectionally to other prompt tokens.

Common Mistakes

- **Mistake:** Thinking causal masking is applied as a hard matrix multiplication. It is applied as an element-wise addition of $-\infty$ (or a large negative number like -10^9) to the attention scores prior to the Softmax operation.
-

Question 16: Why is Self-Attention $O(L^2)$?

Short Interview Answer (30–60 seconds)

Self-Attention is quadratic in sequence length ($O(L^2)$) because every token in a sequence of length L must calculate an attention score (similarity dot product) with every other token in the sequence. This results in an $L \times L$ attention matrix that must be computed and stored in memory. The matrix multiplication $\mathbf{QK}^T$ and the weighted summation $\mathbf{AV}$ both require $O(L^2 \cdot d)$ operations, making both time and memory complexity quadratic with sequence length.

Key Interview Points

- **All-to-All Comparison:** Every query vector interacts with every key vector.
- **Attention Matrix Size:** Storing the $L \times L$ float matrix scales quadratically.
- **Dominating Term:** When sequence length $L > d$ (model dimension), the $L^2 \cdot d$ term dominates the projection term $L \cdot d^2$.

Technical Intuition & Complexity (where applicable)

The self-attention pipeline involves three steps:

1. **Projection:** Project input $\mathbf{X} \in \mathbb{R}^{L \times d}$ to $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ using weights of size $d \times d$.

- Complexity: $3 \times O(L \cdot d^2) = O(L \cdot d^2)$.

2. **Attention Score Matrix:** Compute $\mathbf{QK}^T$ where $\mathbf{Q}, \mathbf{K} \in \mathbb{R}^{L \times d}$.

- Result is of size $L \times L$. Complexity: $O(L^2 \cdot d)$.

3. **Value Aggregation:** Compute $\text{Softmax}(\cdot)\mathbf{V}$ where $\mathbf{V} \in \mathbb{R}^{L \times d}$.

- Complexity: $O(L^2 \cdot d)$.

The total time complexity is $O(L \cdot d^2 + L^2 \cdot d)$. When sequence length L increases, the L^2 term quickly becomes the primary computational and memory bottleneck.

Production Perspective & Trade-offs

For context windows of 128k tokens, $L^2 = 1.6 \times 10^{10}$ matrix elements per head. This quadratic scaling is the primary reason why serving long-context LLMs is extremely expensive. Models use **Linear Attention** approximations or windowed local attention to reduce complexity to $O(L)$, but this often degrades the model's ability to recall details across the entire context window (the "needle in a haystack" test).

Follow-up Questions

- **Follow-up:** How does FlashAttention improve this if the theoretical complexity remains $O(L^2)$?
- **Answer:** FlashAttention does not change the mathematical complexity of $O(L^2 \cdot d)$ FLOPs, but it drastically reduces memory-bound latency. It splits the $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ matrices into blocks and loads

them into fast GPU SRAM, computes attention outputs on-chip, and avoids writing the intermediate $L \times L$ attention matrix back to the slower GPU HBM, making it up to $2 - 4\times$ faster in practice.

Common Mistakes

- **Mistake:** Saying self-attention is $O(L^2)$ in model parameters. The number of parameters in the attention layer is independent of sequence length—it is $4d^2$ (for the $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$ projection matrices). The quadratic scaling applies strictly to **activations (memory)** and **operations (FLOPs)**.
-

Question 17: Compute Self-Attention manually for a small example.

Short Interview Answer (30–60 seconds)

To compute self-attention manually for a sequence of length $L = 2$ and head dimension $d_k = 2$, we:

1. Compute the dot products of the Queries and Keys to get the attention scores matrix $\mathbf{A} = \mathbf{Q}\mathbf{K}^T$.
2. Scale the scores by dividing by $\sqrt{d_k} = \sqrt{2} \approx 1.414$.
3. Apply the Softmax function row-wise to get the attention weights matrix $\mathbf{P}$.
4. Multiply $\mathbf{P}$ by the Values matrix $\mathbf{V}$ to get the final contextualized outputs $\mathbf{O}$.

Key Interview Points

- **Input State:** Sequence length $L = 2$, hidden dimension $d_k = 2$.
- **Attention Matrix Calculation:** Matrix multiplication followed by scalar scaling.
- **Row-wise Softmax:** Norm values sum to 1 across each row.
- **Value Blending:** Final output is a weighted combination of Value vectors.

Technical Intuition & Complexity (where applicable)

Let:

$$\mathbf{Q} = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 2.0 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 1.0 & 1.0 \\ 0.0 & 1.0 \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} 2.0 & -1.0 \\ 1.0 & 3.0 \end{bmatrix}$$

1. **Compute scores $\mathbf{A} = \mathbf{Q}\mathbf{K}^T$:**

$$\mathbf{A} = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 2.0 \end{bmatrix} \begin{bmatrix} 1.0 & 0.0 \\ 1.0 & 1.0 \end{bmatrix} = \begin{bmatrix} 1.0 & 1.0 \\ 0.0 & 2.0 \end{bmatrix}$$

2. **Scale by** $\sqrt{d_k} = \sqrt{2} \approx 1.4142$:

$$\mathbf{S} = \frac{\mathbf{A}}{1.4142} = \begin{bmatrix} 0.7071 & 0.7071 \\ 0.0 & 1.4142 \end{bmatrix}$$

3. **Apply Softmax row-wise to get weights P**:

- **Row 1**: Exponents are $\exp(0.7071) = 2.0281$ and $\exp(0.7071) = 2.0281$.

$$\text{Softmax}(\text{Row 1}) = \left[\frac{2.0281}{2.0281 + 2.0281}, \frac{2.0281}{2.0281 + 2.0281} \right] = [0.5, 0.5]$$

- **Row 2**: Exponents are $\exp(0.0) = 1.0$ and $\exp(1.4142) = 4.1132$.

$$\text{Softmax}(\text{Row 2}) = \left[\frac{1.0}{1.0 + 4.1132}, \frac{4.1132}{1.0 + 4.1132} \right] = [0.1956, 0.8044]$$

- **Attention Weights Matrix**:

$$\mathbf{P} = \begin{bmatrix} 0.5 & 0.5 \\ 0.1956 & 0.8044 \end{bmatrix}$$

4. **Multiply by Values matrix V**:

$$\mathbf{O} = \mathbf{PV} = \begin{bmatrix} 0.5 & 0.5 \\ 0.1956 & 0.8044 \end{bmatrix} \begin{bmatrix} 2.0 & -1.0 \\ 1.0 & 3.0 \end{bmatrix}$$

- **Output Token 1**:

$$O_{1,1} = (0.5 \times 2.0) + (0.5 \times 1.0) = 1.5$$

$$O_{1,2} = (0.5 \times -1.0) + (0.5 \times 3.0) = 1.0$$

- **Output Token 2**:

$$O_{2,1} = (0.1956 \times 2.0) + (0.8044 \times 1.0) = 0.3912 + 0.8044 = 1.1956$$

$$O_{2,2} = (0.1956 \times -1.0) + (0.8044 \times 3.0) = -0.1956 + 2.4132 = 2.2176$$

- Final Contextualized Output:

$$\mathbf{O} = \begin{bmatrix} 1.5 & 1.0 \\ 1.1956 & 2.2176 \end{bmatrix}$$

Production Perspective & Trade-offs

During production execution on GPUs, this sequence of matrix math is parallelized across GPU threads. The division, softmax, and multiplication are combined inside a single GPU block using shared memory registers to ensure high computational throughput.

Follow-up Questions

- **Follow-up:** *How does causal masking alter this manual calculation?*
- **Answer:** If we applied causal masking, the entry at position (1, 2) in the scores matrix (representing Token 1 attending to Token 2) would be set to $-\infty$. This would make the Softmax for Row 1 output $[1.0, 0.0]$, meaning Token 1 attends only to itself.

Common Mistakes

- **Mistake:** Forgetting to apply Softmax row-wise. Applying Softmax column-wise would violate the attention weight properties, meaning the weights for a single token's representation would not sum to 1.
-

Multi-Head Attention

Question 18: Why do we need Multi-Head Attention?

Short Interview Answer (30–60 seconds)

Multi-Head Attention (MHA) is needed because a single attention head is forced to average attention scores across the entire sequence, which compresses and flattens contextual relationships. By splitting the model's hidden dimension d into h independent heads, the model can jointly attend to information from different representation subspaces at different positions. This allows one head to focus on grammatical relationships (e.g., subject-verb agreement), while another focuses on coreference (e.g., mapping pronouns to names), and another captures semantic context.

Key Interview Points

- **Subspace Splitting:** Splits model dimension d into h heads of dimension $d_k = d/h$.

- **Linguistic Specialization:** Different heads specialize in different grammatical, syntax, and semantic patterns.
- **Preventing Averaging:** Avoids the "blurry" attention maps that occur when averaging relationships over a single head.
- **Parallel Subspaces:** Same overall compute cost as single-head attention, but higher representation capacity.

Technical Intuition & Complexity (where applicable)

In Multi-Head Attention:

$$\text{MHA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O$$

$$\text{where head}_i = \text{Attention}(\mathbf{QW}_i^Q, \mathbf{KW}_i^K, \mathbf{VW}_i^V)$$

$$- \mathbf{W}_i^Q, \mathbf{W}_i^K \in \mathbb{R}^{d \times d_k}, \mathbf{W}_i^V \in \mathbb{R}^{d \times d_v}$$

$$- \mathbf{W}^O \in \mathbb{R}^{hd_v \times d}$$

- **Computational Complexity:** Since we split the dimension d into h heads of size d/h , the theoretical computational complexity of MHA is $O(L^2 \cdot d + L \cdot d^2)$, which is mathematically identical to single-head attention with dimension d .

Production Perspective & Trade-offs

While MHA is computationally efficient in theory, in production, handling many small matrix multiplications can lead to high memory-bandwidth overhead on GPUs. High numbers of heads (e.g., $h = 32$) require complex tensor reshaping operations, which can be a bottleneck unless compiled into a single fused CUDA kernel (like FlashAttention).

Follow-up Questions

- **Follow-up:** *Do all heads learn useful features, or are some redundant?*
- **Answer:** Empirically, many heads are redundant. Pruning studies show that up to 50% of attention heads can be pruned from a trained Transformer model without significant degradation in task performance, as many heads learn similar attention maps.

Common Mistakes

- **Mistake:** Thinking that Multi-Head Attention is h times more expensive to compute than Single-Head Attention. Because each head operates on a projected subspace of size d/h , the total float operations (FLOPs) remain identical.
-

Question 19: What does each attention head learn?

Short Interview Answer (30–60 seconds)

Each attention head learns to specialize in specific types of structural, grammatical, or semantic relationships within the text. Visualizations of attention maps show that heads typically fall into several categories:

- **Positional heads:** Attend to adjacent tokens (offset of $+1$ or -1).
- **Syntactic heads:** Identify grammatical connections, such as linking verbs to their objects or adjectives to nouns.
- **Coreference heads:** Link pronouns (e.g., "she") to their corresponding nouns (e.g., "Mary").
- **Specialized heads:** Focus on format delimiters, punctuation, or out-of-vocabulary tokens.

Key Interview Points

- **Specialization:** Heads naturally partition the task of sentence analysis.
- **Syntax Mapping:** Learns dependency parsing structure implicitly without explicit syntax labels.
- **Coreference Resolution:** Tracks referential links across long sequences.
- **Delimiter Focus:** Many heads learn to attend to special tokens like `[CLS]` or `[SEP]` as a default "null" action when no other strong relationships are found.

Production Perspective & Trade-offs

Because heads learn specialized functions, we can utilize **Pruning** or **LoRA** (Low-Rank Adaptation) target selection. During fine-tuning, instead of adapting all heads, we can restrict training to heads that capture semantic relationships, leaving syntax-stabilizing heads frozen, which reduces parameter storage and latency.

Follow-up Questions

- **Follow-up:** *Why do some heads default to attending to punctuation or special tokens?*
- **Answer:** When a token does not find any semantically relevant match in the sequence, the Softmax must still distribute its weights. The model learns to route this "unassigned" attention to special tokens (like `[CLS]` or punctuation) to act as a neutral sink, preventing the values of other content tokens from being distorted.

Common Mistakes

- **Mistake:** Assuming attention heads are explicitly programmed to learn grammar or syntax. The model is trained purely on next-token prediction, and these syntactic structures emerge naturally as the optimal way to minimize prediction loss.
-

Question 20: Multi-Head vs Single-Head Attention.

Short Interview Answer (30–60 seconds)

Multi-Head Attention divides the model dimension d into h heads of size d/h , performs attention calculations in parallel across these independent projection subspaces, and projects the concatenated outputs back to d . Single-Head Attention computes a single attention map over the entire hidden dimension d all at once. MHA is vastly superior because it allows the model to capture multiple diverse relationships simultaneously, whereas Single-Head Attention is limited to a single averaged representation, resulting in a significantly lower capacity to capture complex syntax and semantics.

Key Interview Points

- **Subspace Count:** MHA has h parallel attention maps; Single-Head has 1.
- **Parameter Capacity:** Same theoretical parameter count and computational complexity.
- **Empirical Superiority:** MHA consistently achieves higher accuracy and lower perplexity at scale.
- **Subspace Dimension:** MHA projects to dimension d/h ; Single-Head processes dimension d directly.

Technical Intuition & Complexity (where applicable)

Feature	Single-Head Attention (SHA)	Multi-Head Attention (MHA)
Number of Attention Matrices	1 matrix of size $L \times L$	h matrices of size $L \times L$
Projection Dimension	$d_k = d$	$d_k = d/h$
Total Parameter Count	$4d^2$	$4d^2$ (typically)
FLOP Complexity	$O(L^2 \cdot d + L \cdot d^2)$	$O(L^2 \cdot d + L \cdot d^2)$
Representational Subspaces	1 global subspace	h independent subspaces

Production Perspective & Trade-offs

In production inference, MHA has a much larger memory management overhead. The Key-Value caches for MHA must store separate matrices for all h heads. This is why modern production architectures trade off MHA for **Grouped-Query Attention (GQA)** or **Multi-Query Attention (MQA)**. GQA uses fewer Key/Value heads than Query heads, sharing Key-Value projections across groups of Query heads to reduce KV Cache size in VRAM by $4 - 8\times$ with minimal loss in model capacity.

Follow-up Questions

- **Follow-up:** How does Multi-Query Attention (MQA) differ from Multi-Head Attention (MHA)?

- **Answer:** In MHA, each query head has a unique key and value head. In MQA, there is only a single key head and a single value head shared across all query heads. This dramatically reduces the KV Cache size stored in VRAM during autoregressive decoding.

Common Mistakes

- **Mistake:** Thinking MHA has more parameters than single-head attention. If MHA uses projection matrices of size $d \times (d/h)$ for each of the h heads, the total parameters across all heads is $h \times (d \times (d/h)) = d^2$. Thus, the total parameter count is identical to single-head attention.
-

Positional Encoding

Question 21: Why do Transformers need positional information?

Short Interview Answer (30–60 seconds)

Transformers need positional information because their core mechanism, self-attention, is completely **permutation-invariant**. When computing attention scores $\text{Softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k})\mathbf{V}$, the operation acts as a set-based weighted sum. Scrambling the order of tokens in the input sequence results in the exact same output vectors (just scrambled). Since language syntax and meaning depend heavily on word order (e.g., "dog bites man" vs. "man bites dog"), we must explicitly inject positional information into the model.

Key Interview Points

- **Permutation Invariance:** Self-attention computes set-based distributions, ignoring token index.
- **Word Order Preservation:** Crucial for semantic and syntactic correctness in language.
- **Symmetry Breaking:** Breaking the symmetry of identical tokens appearing at different positions.

Production Perspective & Trade-offs

In production systems, positional encoding is typically injected either at the very beginning (as absolute positional embeddings added to the input embeddings) or at each attention layer (as relative positional biases like RoPE or ALiBi). Adding absolute positional embeddings requires allocating a fixed parameter matrix of size $L_{\max} \times d$, which hard-caps the maximum sequence length the model can process at inference time, creating a significant production bottleneck.

Follow-up Questions

- **Follow-up:** *Why are RNNs not permutation-invariant?*
- **Answer:** RNNs process tokens sequentially, passing hidden states step-by-step ($h_t = f(h_{t-1}, x_t)$). The temporal order of processing naturally encodes position.

Common Mistakes

- **Mistake:** Believing that self-attention naturally learns word order over time without positional encodings. Without positional encodings, the model is mathematically incapable of identifying which word came first.
-

Question 22: Explain RoPE.

Short Interview Answer (30–60 seconds)

RoPE (Rotary Position Embedding) is a relative positional encoding method that encodes positions by applying a rotation matrix to the Query and Key vectors in the 2D complex plane. Instead of adding absolute vectors to the token embeddings, RoPE rotates the vectors by an angle proportional to their sequence index. When computing the dot product between Query $\mathbf{q}_m$ and Key $\mathbf{k}_n$, the resulting score depends only on the relative distance $m - n$. This mathematical property allows the model to extrapolate to sequence lengths beyond what it was trained on.

Key Interview Points

- **Relative Distance Encoding:** Dot product becomes a function of relative distance $m - n$.
- **Complex Space Rotation:** Splits vectors into 2D chunks and rotates each chunk in a 2D plane.
- **Inner-Product Preservation:** Preserves vector norm while encoding relative distance.
- **Extrapolation Friendly:** Compatible with length-extension techniques like RoPE scaling.

Technical Intuition & Complexity (where applicable)

For a 2D slice of a vector $\mathbf{x} = [x_1, x_2]^T$ at position m , RoPE applies the rotation:

$$\mathbf{R}_{\Theta, m}^2 \mathbf{x} = \begin{bmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

- **Tiny Hand Calculation:** Let $m = 1$, $\theta = \frac{\pi}{2}$, and $\mathbf{x} = [2.0, 1.0]^T$.

$$\cos\left(1 \times \frac{\pi}{2}\right) = 0, \quad \sin\left(1 \times \frac{\pi}{2}\right) = 1$$

$$\mathbf{R}_{\Theta, 1}^2 \mathbf{x} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 2.0 \\ 1.0 \end{bmatrix} = \begin{bmatrix} (0 \times 2.0) + (-1 \times 1.0) \\ (1 \times 2.0) + (0 \times 1.0) \end{bmatrix} = \begin{bmatrix} -1.0 \\ 2.0 \end{bmatrix}$$

This is a 90° counter-clockwise rotation of the vector $[2, 1]^T$ to $[-1, 2]^T$, successfully encoding position index $m = 1$.

Production Perspective & Trade-offs

Because RoPE must be applied to Query and Key projections at every attention layer before the dot product, it adds coordinate transformations inside the attention loop. To maintain high throughput, GPU implementations use custom CUDA or Triton kernels that fuse the RoPE rotation directly with the projection operations, avoiding slow global memory roundtrips.

Follow-up Questions

- **Follow-up:** *Why does the relative distance property hold for the dot product?*
- **Answer:** In complex numbers, rotation is equivalent to multiplying by $e^{im\theta}$. The dot product of Query $\mathbf{q}$ at position m and Key $\mathbf{k}$ at position n is represented as $\text{Re}(\mathbf{q}_m \mathbf{k}_n^*)$, where $*$ is the complex conjugate. This equals $\text{Re}(\mathbf{q} e^{im\theta} (\mathbf{k} e^{in\theta})^*) = \text{Re}(\mathbf{q} \mathbf{k}^* e^{i(m-n)\theta})$, proving it depends only on $m - n$.

Common Mistakes

- **Mistake:** Thinking RoPE is added to Value vectors. RoPE is applied **strictly** to Query and Key vectors. Value vectors are not rotated, as they represent content, not spatial index positions.
-

Question 23: Compare Sinusoidal Encoding, Learned Embeddings, RoPE, and ALiBi.

Short Interview Answer (30–60 seconds)

These are four positional encoding strategies:

- **Learned Absolute Embeddings:** Model learns a vector for each absolute index. Zero extrapolation capability; hard-capped sequence length.
- **Sinusoidal Encoding:** Uses fixed, non-trainable sine/cosine functions of different frequencies. Hard-capped and extrapolates poorly in practice.
- **RoPE:** Rotates Queries and Keys in 2D planes to encode relative position. Extrapolates well using scaling methods (e.g., YaRN, NTK).
- **ALiBi:** Injects a constant negative bias proportional to token distance directly into the attention matrix. Extrapolates outstandingly to long context without retraining.

Key Interview Points

- **Extrapolation Rank:** $\text{ALiBi} \geq \text{RoPE} \gg \text{Sinusoidal} \approx \text{Learned}$.
- **Learned Absolute:** Simplest but lacks relative distance awareness and length flexibility.
- **ALiBi (Attention with Linear Biases):** Eliminates position embedding parameters entirely, shifting bias to attention scores.

Technical Intuition & Complexity (where applicable)

- **ALiBi Attention Score formula:**

$$a_{i,j} = \text{Softmax} \left(\frac{\mathbf{q}_i \mathbf{k}_j^T}{\sqrt{d_k}} - m \cdot |i - j| \right)$$

where m is a head-specific geometric slope parameter. The penalty scales linearly with token distance $|i - j|$, discouraging long-range attention unless query-key alignment is extremely strong.

Encoding Method	Type	Parameters	Context Extrapolation	Usage
Learned Absolute	Absolute	$L_{\max} \times d$	None (Fails beyond $L_{\max}$)	BERT, GPT-3
Sinusoidal	Absolute	0 (Fixed)	Poor	Original Transformer
RoPE	Relative	0 (Fixed Rotation)	Good (with scaling)	Llama, Gemma, Mistral
ALiBi	Relative	0 (Fixed Bias)	Excellent	MPT, BLOOM

Production Perspective & Trade-offs

ALiBi is highly efficient because it has zero positional parameters to store, and context window length can be dynamically extended during inference. However, ALiBi's linear decay can sometimes over-penalize long-range relationships, causing the model to lose track of early context in long conversations compared to RoPE, which retains angular separation across larger scopes.

Follow-up Questions

- **Follow-up:** *How does RoPE scale to longer context windows at runtime?*
- **Answer:** By interpolating the rotary frequencies. If we scale the context length by a factor of s , we can scale the frequencies down by s (e.g., using Linear or Neural Tangent Kernel (NTK) scaling) so that the angles for the longer sequence fit within the original trained angular range.

Common Mistakes

- **Mistake:** Thinking ALiBi requires storing positional vectors. ALiBi does not use positional vectors at all; it modifies the raw attention logit matrix element-wise using index math, saving VRAM.

Tokenization

Question 24: Explain Byte Pair Encoding (BPE).

Short Interview Answer (30–60 seconds)

Byte Pair Encoding (BPE) is a subword tokenization algorithm that builds a vocabulary by merging the most frequent pairs of characters or bytes in a corpus. It starts with a base vocabulary containing all individual characters or bytes. It then scans the corpus, finds the most frequent adjacent pair of tokens, merges them into a new subword token, and adds it to the vocabulary. This merge process is repeated iteratively until the vocabulary reaches a predefined target size.

Key Interview Points

- **Subword Frequency Merging:** Merges based on raw occurrence counts of adjacent tokens.
- **Base Vocabulary:** Starts with characters/bytes to ensure zero out-of-vocabulary (OOV) tokens.
- **Merge Rules:** Stores a deterministic list of merge priorities to tokenize unseen text.
- **Byte-level BPE:** GPT-2/Llama tokenize raw bytes instead of Unicode characters to handle all UTF-8 characters cleanly.

Technical Intuition & Complexity (where applicable)

- **Tokenization step complexity:** $O(L \cdot \log |V|)$ where L is input text character length and $|V|$ is the vocabulary size (using efficient trie or segment trees).
- **Merge Example:**
 - Corpus: "hug hug pug"
 - Character tokens: h u g _ h u g _ p u g
 - Step 1: Most frequent adjacent pair is u and g (occurs 3 times). Merge u and g to ug .
 - Tokens: h ug _ h ug _ p ug
 - Step 2: Most frequent pair is h and ug (occurs 2 times). Merge to hug .
 - Tokens: hug _ hug _ p ug
 - Stop when vocabulary target size is reached.

Production Perspective & Trade-offs

BPE is standard in modern LLMs because it handles the trade-off between vocabulary size and sequence length. A small vocabulary (e.g., character-level) leads to tiny model files but extremely long token sequences, which slows down generation. A huge vocabulary (e.g., word-level) leads to short token sequences but massive embedding parameter tables that consume VRAM. BPE balances this, usually picking vocabulary sizes of 32k to 256k tokens.

Follow-up Questions

- **Follow-up:** *What is Byte-level BPE, and why is it preferred over character-level BPE?*
- **Answer:** Character-level BPE can still result in Out-of-Vocabulary errors when encountering rare characters or emoji in non-English languages. Byte-level BPE maps the text to raw bytes (256 base

tokens) first. Since any Unicode string can be represented as bytes, byte-level BPE guarantees that the tokenizer can process any text without unknown (`<UNK>`) tokens.

Common Mistakes

- **Mistake:** Thinking that the BPE tokenizer runs as part of the neural network. Tokenization is a **pre-processing step** that runs on the CPU before text integers are fed to the model embeddings.

Question 25: Compare BPE, WordPiece, and SentencePiece.

Short Interview Answer (30–60 seconds)

BPE, WordPiece, and SentencePiece are subword tokenizers.

- **BPE** (GPT, Llama) merges the most **frequent** adjacent token pairs in the corpus.
- **WordPiece** (BERT) merges adjacent pairs that maximize the **likelihood** of the training data under a unigram language model (selecting pairs that increase probability the most).
- **SentencePiece** (T5, Llama-3) is a framework that trains BPE or Unigram models directly on **raw byte streams**, treating whitespaces as a standard character (`▯`), which eliminates the need for language-specific pre-tokenizers.

Key Interview Points

- **BPE:** Frequency-driven, local pair-wise merges.
- **WordPiece:** Likelihood-driven, projects probability ratios $(P(AB)/(P(A)P(B)))$.
- **SentencePiece:** Subword framework, no language pre-tokenizer required, space character mapping (`▯`).

Technical Intuition & Complexity (where applicable)

- **WordPiece score criterion:**

$$\text{Score}(A, B) = \frac{\text{count}(AB)}{\text{count}(A) \times \text{count}(B)}$$

WordPiece merges the pair *A* and *B* that has the highest score, favoring pairs of characters that appear together much more frequently than expected by their individual frequencies.

Tokenizer	Merge Selection	Pre-tokenization Needed?	Space Character representation	Primary Models
BPE	Max Frequency	Yes (Regex/Whitespace)	None	GPT-3, GPT-4

Tokenizer	Merge Selection	Pre-tokenization Needed?	Space Character representation	Primary Models
WordPiece	Max Likelihood	Yes (Regex/Whitespace)	## prefix for subwords	BERT
SentencePiece	BPE / Unigram	No (Raw Stream)	Meta-character _	Llama-2/3, T5, Gemma

Production Perspective & Trade-offs

In multilingual production pipelines, SentencePiece is preferred because it does not require a rule-based pre-tokenizer (like Python's `re.split` or SpaCy). Standard tokenizers fail for languages that do not use spaces to separate words (e.g., Chinese, Japanese). SentencePiece treats the entire string as a space-agnostic character stream, yielding much cleaner token boundaries for non-English scripts.

Follow-up Questions

- **Follow-up:** *What is the purpose of the ## prefix in BERT's WordPiece tokenizer?*
- **Answer:** The ## prefix indicates that the token is a subword that must be appended to the previous token without a space (e.g., "playing" → ["play", "##ing"]).

Common Mistakes

- **Mistake:** Thinking SentencePiece is a new tokenization mathematical algorithm. SentencePiece is a *library* wrapper that implements BPE and Unigram tokenization directly on raw character streams.

Question 26: Why does token count matter?

Short Interview Answer (30–60 seconds)

Token count is the core metric for LLM performance, latency, memory, and cost. First, context windows are measured in tokens, capping document sizes. Second, self-attention complexity scales quadratically ($O(L^2)$) with token length, making long token sequences compute-intensive. Third, LLM generation speed is limited by **memory bandwidth**, where each generated token requires loading the entire model parameters from HBM to SRAM, causing sequential latency. Finally, most production APIs charge strictly per-token.

Key Interview Points

- $O(L^2)$ **Attention Bottleneck:** Long sequences consume quadratic compute and activation memory.

- **Memory Bandwidth Bound:** Token-by-token decoding speed is bound by weight transfer rates, not compute FLOPS.
- **Billing Metric:** Production scaling budgets are tied directly to token volume.

Production Perspective & Trade-offs

Because token count maps directly to hardware execution time and cost, optimizing tokenization is a primary engineering goal. If a tokenizer is inefficient (e.g., tokenizing a paragraph into 500 tokens instead of 200), the model will run $2.5\times$ slower and consume $6.25\times$ more self-attention activation memory during training and prefill. Llama-3 upgraded its vocabulary to 128k tokens (from Llama-2's 32k), resulting in a 15% reduction in token counts for typical text, directly improving inference speed.

Follow-up Questions

- **Follow-up:** *Why does a larger vocabulary size decrease token count but increase VRAM?*
- **Answer:** A larger vocabulary allows the tokenizer to merge longer character sequences into a single token (e.g., "representation" $\rightarrow$ 1 token instead of 3). However, this increases the size of the embedding layer weights matrix ($|V| \times d$), which must be kept in GPU memory, increasing VRAM consumption.

Common Mistakes

- **Mistake:** Measuring context capacity in words or characters instead of tokens. A single token corresponds to approximately 4 characters or 0.75 words in English, but this ratio varies wildly across languages (often requiring 2–3 tokens per word in non-Latin scripts).

Question 27: Why are tokenizers different across models?

Short Interview Answer (30–60 seconds)

Tokenizers differ across models because they are trained on different corpora, utilize different vocabulary sizes, and are optimized for different target tasks and languages. A model trained for code (e.g., CodeLlama) needs tokenizer merge rules that preserve whitespaces and indentation. A multilingual model (e.g., Llama-3) needs a larger vocabulary (128k tokens) with bytes representation to compress non-English text efficiently. If a model reused GPT-2's old vocabulary, it would fragment non-English text and code into many tiny tokens, leading to slow and expensive generation.

Key Interview Points

- **Vocabulary Size:** Ranges from 32k (Llama-2) to 256k (Gemma).
- **Whitespace Handling:** Preserving spaces/indents is crucial for programming languages.

- **Language Compression:** Large vocabularies compress non-English strings into fewer tokens, lowering latency.

Production Perspective & Trade-offs

Selecting vocabulary size is a crucial systems trade-off:

- **Small Vocab (32k):** Low embedding parameter VRAM footprint (ideal for edge deployment). But token sequences are longer, leading to high attention VRAM footprint and slower decoding.
- **Large Vocab (256k):** Compact token sequences (faster generation, fits more text in context). But embedding matrices are massive, consuming gigabytes of VRAM just to store the lookup tables.

Follow-up Questions

- **Follow-up:** *What is the "fertility rate" of a tokenizer?*
- **Answer:** Fertility rate is the average number of tokens generated per word. A low fertility rate (close to 1.0) means the tokenizer is highly efficient at representing the text. Non-English languages have high fertility rates on English-centric tokenizers, making them more expensive to run.

Common Mistakes

- **Mistake:** Thinking you can swap a tokenizer from one model (e.g., Llama) to another model (e.g., Mistral) at inference. The model's embedding layers are tightly coupled to the specific token IDs of its corresponding tokenizer; using a different tokenizer will output complete gibberish.
-

Embeddings

Question 28: What are contextual embeddings?

Short Interview Answer (30–60 seconds)

Contextual embeddings are vector representations of tokens that change dynamically depending on the surrounding words in a sequence. Unlike static embeddings (like Word2Vec or GloVe) which map a word to a single fixed vector regardless of context, contextual embeddings are generated by passing token vectors through the stacked layers of a Transformer. This allows the same word (e.g., "bank") to have completely different vector representations depending on whether it appears in "river bank" or "bank account."

Key Interview Points

- **Context-Dependency:** Vector changes based on surrounding sequence context.
- **Transformer-Generated:** Produced by the hidden states of intermediate attention layers.

- **Polysemy Resolution:** Resolves words with multiple meanings (homophones/homographs) dynamically.

Production Perspective & Trade-offs

Contextual embeddings are highly effective for search, classification, and retrieval-augmented generation (RAG) because they capture semantic nuance. The trade-off is latency: to compute a contextual embedding, the input text must be processed through the entire Transformer neural network ($O(L^2)$ compute). Static embeddings, on the other hand, can be retrieved instantly using a simple hash map lookup ($O(1)$ time), which is preferred when latency budgets are extremely tight (e.g., sub-millisecond search).

Follow-up Questions

- **Follow-up:** Which layer's output is best to use as a semantic embedding for downstream classification tasks?
- **Answer:** In practice, averaging the hidden states of the last few layers (e.g., averaging the last 4 layers) or using a dedicated pooling layer (like the [CLS] token in BERT, or mean pooling across the sequence) provides the most stable semantic representation.

Common Mistakes

- **Mistake:** Confusing token embeddings with contextual embeddings. Token embeddings are the static vectors loaded at the very input layer of the model; contextual embeddings are the dynamic representations output by the attention layers.
-

Question 29: Difference between Token Embeddings and Positional Embeddings.

Short Interview Answer (30–60 seconds)

- **Token Embeddings** represent the semantic and conceptual meaning of the individual word or subword, regardless of where it appears in the sentence.
- **Positional Embeddings** represent the spatial index or location of the token within the sequence. At the input layer of the Transformer, the Token Embedding vector $\mathbf{t}_i$ and the Positional Embedding vector $\mathbf{p}_i$ of the same dimension d are added element-wise ($\mathbf{e}_i = \mathbf{t}_i + \mathbf{p}_i$) to form a single input vector that captures both meaning and position.

Key Interview Points

- **Token Embedding:** Captures semantic content; trained via vocabulary lookup.

- **Positional Embedding:** Captures spatial order; either learned or computed analytically (Sinusoidal/RoPE).
- **Element-wise Addition:** Summed together at the input: $\mathbf{x}_{\text{input}} = \mathbf{x}_{\text{token}} + \mathbf{x}_{\text{pos}}$.

Technical Intuition & Complexity (where applicable)

Why add instead of concatenate?

- **Concatenation:** If we concatenated a token vector of size d_1 and a position vector of size d_2 , the input dimension would expand to $d_1 + d_2$, increasing parameter sizes in all subsequent weight matrices.
- **Addition:** Summing them preserves the dimension d . Mathematically, when we project the sum using a weight matrix $\mathbf{W}$:

$$\mathbf{e}_i \mathbf{W} = (\mathbf{t}_i + \mathbf{p}_i) \mathbf{W} = \mathbf{t}_i \mathbf{W} + \mathbf{p}_i \mathbf{W}$$

The linear projection processes the semantic part and the positional part independently in parallel, allowing the model to disentangle the two components in downstream layers.

Production Perspective & Trade-offs

Because the addition of positional embeddings happens once at the input layer, it has negligible computational cost. However, in models using relative positional encodings like RoPE, positional embeddings are applied at every attention layer inside the QK dot-product loop, which requires optimized GPU operations to avoid latency overhead.

Follow-up Questions

- **Follow-up:** *Does the addition of token and positional embeddings corrupt the semantic information in the token embedding?*
- **Answer:** No. Since the embedding space is high-dimensional (e.g., $d = 4096$), the model can project token representations and positional representations into orthogonal subspaces, allowing it to recover both meaning and position without interference.

Common Mistakes

- **Mistake:** Thinking token and positional embeddings are concatenated. In the standard Transformer architecture, they are added element-wise, not concatenated.
-

Question 30: Why is cosine similarity commonly used?

Short Interview Answer (30–60 seconds)

Cosine similarity is commonly used because it measures the **angular alignment** between two vectors, completely ignoring their magnitudes. In high-dimensional embedding spaces, vector lengths can be heavily biased by word frequency, document length, or noise. By normalizing the dot product of the vectors by their Euclidean norms, cosine similarity restricts the output range to $[-1, 1]$, measuring semantic similarity purely based on the direction of the concepts.

Key Interview Points

- **Scale-Invariance:** Unaffected by vector magnitude or scale variations.
- **Angular Focus:** Evaluates the directional alignment of concepts.
- **Bounded Range:** Normalizes output strictly between -1 (opposite) and $+1$ (identical).

Technical Intuition & Complexity (where applicable)

For two vectors $\mathbf{A}, \mathbf{B} \in \mathbb{R}^d$:

$$\text{Cosine Similarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|_2 \|\mathbf{B}\|_2} = \frac{\sum_{i=1}^d A_i B_i}{\sqrt{\sum_{i=1}^d A_i^2} \sqrt{\sum_{j=1}^d B_j^2}}$$

- **Complexity:** $O(d)$ dot product and normalization steps.
- **Geometric Meaning:** Represents the cosine of the angle θ between the two vectors in the d -dimensional space.

Production Perspective & Trade-offs

In production vector databases (e.g., Pinecone, Milvus), computing cosine similarity directly can be slow because it requires computing square roots for normalization at query time. To optimize this, vectors are **pre-normalized** (L_2 normalized to unit length $\|\mathbf{A}\|_2 = 1$) upon insertion. Once normalized, the cosine similarity simplifies to a simple dot product ($\mathbf{A} \cdot \mathbf{B}$), which can be executed at maximum speed on GPU or CPU SIMD hardware.

Follow-up Questions

- **Follow-up:** *What is the relationship between Cosine Similarity and Dot Product?*
- **Answer:** If the vectors are normalized to unit length (L_2 norm = 1), cosine similarity is exactly equal to the dot product.

Common Mistakes

- **Mistake:** Using Euclidean distance without normalization for text retrieval. Euclidean distance is sensitive to vector length; a longer document containing a word multiple times will have a larger vector norm, throwing off similarity calculations.

Context Window & KV Cache

Question 31: What is a context window?

Short Interview Answer (30–60 seconds)

A context window is the maximum number of tokens (both prompt and response) that an LLM can process in a single inference cycle. It is fundamentally bounded by two bottlenecks: the model's **positional encoding limits** (the maximum distance range the positional embeddings can represent) and the **quadratic $O(L^2)$ memory scaling** of self-attention activation memory, which quickly consumes GPU VRAM for long input sequences.

Key Interview Points

- **Upper Limit:** Bounds prompt + completion token lengths combined.
- **VRAM Constraint:** Attention matrix scales as $O(L^2)$ in VRAM, restricting length.
- **Positional Limits:** Original models fail when token distance exceeds trained positional indices.

Production Perspective & Trade-offs

To scale context windows in production (e.g., from 8k to 128k or 1M tokens), developers must implement memory saving architectures like **PagedAttention** (which eliminates VRAM fragmentation by page-allocating KV cache) and **FlashAttention** (which eliminates HBM memory footprint of the attention matrix). The trade-off is latency: while the model can *fit* 128k tokens in memory, the prefill phase latency scales up significantly, and retrieving accurate details at the middle of the context (the "Lost in the Middle" phenomenon) degrades.

Follow-up Questions

- **Follow-up:** *How does the "Lost in the Middle" phenomenon affect long-context retrieval?*
- **Answer:** Studies show that LLMs are highly effective at retrieving information placed at the very beginning or end of their context window, but their retrieval accuracy drops drastically (often by 50%+) when the relevant information is situated in the middle of a long prompt.

Common Mistakes

- **Mistake:** Thinking that doubling the context window size only doubles the VRAM requirements. Because attention activations scale quadratically, doubling sequence length increases attention activation VRAM by $4\times$.
-

Question 32: Explain KV Cache.

Short Interview Answer (30–60 seconds)

The KV Cache (Key-Value Cache) is an inference optimization technique that accelerates autoregressive text generation. During decoding, the model generates tokens one by one. To generate token t , the self-attention layer needs the Key and Value vectors of all previous tokens $1 \dots t - 1$. Instead of recomputing these vectors at every generation step (which would cost $O(L)$ computational steps per token, summing to $O(L^2)$ total), we compute them once, store them in GPU VRAM (the KV Cache), and append the newly generated token's Key and Value at each step. This reduces step complexity from $O(L^2)$ to $O(L)$ per token.

Key Interview Points

- **Inference Optimization:** Avoids redundant $O(L)$ matrix multiplications at each decoding step.
- **VRAM Footprint:** Stored in VRAM, scaling linearly with batch size, layers, heads, and sequence length.
- **Step Speedup:** Converts a compute-bound $O(L^2)$ problem into a memory-bandwidth-bound $O(L)$ retrieve-and-append loop.

Technical Intuition & Complexity (where applicable)

- **KV Cache size formula (FP16/BF16 Precision - 2 bytes/float):**

$$\text{KV Cache VRAM Size} = 2 \times 2 \times b \times n_{\text{layers}} \times n_{\text{heads_kv}} \times d_{\text{head}} \times s_{\text{len}} \text{ bytes}$$

- The first factor of 2 accounts for storing both Keys and Values.
- The second factor of 2 is the byte count for FP16 precision.
- **Hand Calculation:**
- Scenario: Batch size $b = 1$, Layers $n_{\text{layers}} = 32$, Key-Value heads $n_{\text{heads_kv}} = 8$ (using Grouped-Query Attention), Head dimension $d_{\text{head}} = 128$, Sequence length $s_{\text{len}} = 4096$.

$$\text{VRAM} = 4 \times 1 \times 32 \times 8 \times 128 \times 4096 \text{ bytes}$$

$$\text{VRAM} = 1024 \times 128 \times 4096 = 131,072 \times 4096 = 536,870,912 \text{ bytes} \approx 536.87 \text{ MB}$$

At batch size 32, this cache scales to ≈ 17.18 GB of VRAM, exceeding the capacity of consumer GPUs just for the caching layer.

Production Perspective & Trade-offs

The KV Cache is the primary bottleneck for serving LLMs in production. While it dramatically reduces generation latency, it consumes massive amounts of VRAM. This limits the maximum batch size a GPU can handle. To serve models cost-effectively, teams use **PagedAttention** (introduced by vLLM) to dynamically allocate cache blocks in non-contiguous physical memory, eliminating VRAM fragmentation and reducing memory waste by up to 96%.

Follow-up Questions

- **Follow-up:** *Why do we not cache Query vectors (Q)?*
- **Answer:** During decoding, we only calculate attention for the single *new* token. We only need the Query vector of the *current* token to dot-product against all past Keys. The Query vectors of past tokens are never needed again, so they are discarded.

Common Mistakes

- **Mistake:** Believing KV Cache is used during training. KV Cache is **strictly** an inference optimization. During training, the entire target sequence is available, allowing us to compute attention for all positions in parallel via causal masking without sequential caching.
-

Question 33: Why is the first generated token slower?

Short Interview Answer (30–60 seconds)

The first generated token is slower because it is computed during the **prefill phase**, which has a fundamentally different computational bottleneck than subsequent tokens (the **decode phase**). During the prefill phase, the model must process the entire user prompt (e.g., hundreds or thousands of tokens) in parallel to compute their representations and populate the KV Cache. This involves large matrix-matrix multiplications (GEMM) that are **compute-bound**. Subsequent tokens only process a single token at a time, which involves smaller matrix-vector operations (GEMV) that are **memory-bandwidth-bound**.

Key Interview Points

- **Prefill vs. Decode:** Prefill processes the prompt in parallel; decode processes one token sequentially.
- **GEMM vs. GEMV:** Parallel prompt processing utilizes GPU Tensor Cores at high efficiency; single-token processing leaves GPU cores idle.
- **KV Cache Initialization:** The model must compute and write the initial KV Cache entries for all prompt tokens.

Production Perspective & Trade-offs

In production LLM serving, this latency difference is measured as:

- **Time to First Token (TTFT)**: High latency because it scales with prompt length (compute-bound).
- **Inter-Token Latency (ITL)**: Low latency but memory-bandwidth-bound (determined by GPU memory speed).

For user experience, a low TTFT is critical. To optimize TTFT, systems use **Chunked Prefill**, which breaks long prompts into smaller chunks, interleaving them with decode steps to prevent long prefill operations from blocking active generation streams.

Follow-up Questions

- **Follow-up**: *How does prompt length affect TTFT vs. ITL?*
- **Answer**: TTFT scales linearly with prompt length (and quadratically for very long sequences due to attention matrix construction). ITL remains virtually constant regardless of prompt length, as each decode step only processes a single token.

Common Mistakes

- **Mistake**: Thinking the first token is slower because the model is "warming up" or loading weights from storage. The weights are already resident in GPU VRAM; the delay is purely due to the parallel compute volume of processing the prompt sequence.
-

Question 34: Prefill phase vs Decode phase.

Short Interview Answer (30–60 seconds)

- **Prefill Phase**: The model processes the entire input prompt in parallel, computes Query, Key, and Value vectors for all prompt tokens, populates the KV Cache, and outputs the probability distribution for the first generated token. It is **compute-bound**, utilizing GPU Tensor Cores via large matrix-matrix multiplications (GEMM).
- **Decode Phase**: The model generates tokens one by one autoregressively. At each step, it projects only the single newly generated token, retrieves past Keys and Values from the KV Cache, and appends the new Key and Value. It is **memory-bandwidth-bound**, waiting for weights to load from GPU HBM to SRAM via matrix-vector operations (GEMV).

Key Interview Points

- **Prefill**: Parallel, GEMM, compute-bound, high GPU SM utilization, establishes KV Cache.
- **Decode**: Sequential, GEMV, memory-bandwidth-bound, low GPU SM utilization, reads/writes KV Cache.
- **Optimization Target**: Prefill benefits from batching and high GPU FLOPs; Decode benefits from fast GPU memory bandwidth (e.g., H100 HBM3).

Technical Intuition & Complexity (where applicable)

During Prefill, we multiply input embeddings $\mathbf{X} \in \mathbb{R}^{L_{\text{prompt}} \times d}$ by weights $\mathbf{W} \in \mathbb{R}^{d \times d}$. The Arithmetic Intensity (FLOPs per byte loaded) is high because weights are reused across all L_{prompt} tokens.

During Decode, we multiply a single token vector $\mathbf{x} \in \mathbb{R}^{1 \times d}$ by weights $\mathbf{W} \in \mathbb{R}^{d \times d}$. The Arithmetic Intensity is extremely low (2 FLOPs per param byte loaded), meaning the GPU execution units spend most of their time idle, waiting for the weights matrix to be read from HBM.

Production Perspective & Trade-offs

To resolve the low GPU utilization during the Decode phase, production servers use **Continuous Batching** (dynamic batching). By grouping decode requests from different users together, we load the weights matrix once and apply it to a batch of vectors ($\mathbf{X} \in \mathbb{R}^{b \times d}$), increasing arithmetic intensity and GPU throughput.

Follow-up Questions

- **Follow-up:** *Why does FP8 quantization benefit the Decode phase more than the Prefill phase?*
- **Answer:** The decode phase is memory-bandwidth bound. Quantizing weights to FP8 halves the model size, meaning we only need to transfer half the bytes from HBM to SRAM per step, directly doubling generation speed (tokens/sec) regardless of raw compute limits.

Common Mistakes

- **Mistake:** Thinking both phases have the same hardware bottlenecks. Optimizing a serving stack requires different configurations (e.g., tensor parallel size) depending on whether the workload is prefill-heavy (long prompts) or decode-heavy (long generations).

Question 35: How do GQA and MQA reduce KV Cache memory?

Short Interview Answer (30–60 seconds)

GQA (Grouped-Query Attention) and MQA (Multi-Query Attention) reduce KV Cache memory by reducing the number of Key and Value heads relative to Query heads.

- **MQA** shares a single Key and Value head across all h Query heads, reducing KV Cache VRAM footprint by h times (typically $32 - 64\times$).

- **GQA** groups Query heads into g groups, allocating one Key and Value head per group.

Both techniques dramatically compress the KV Cache size in VRAM, allowing for much larger batch sizes and longer context windows during inference.

Key Interview Points

- **Multi-Head Attention (MHA):** h Query heads, h Key heads, h Value heads.
- **Multi-Query Attention (MQA):** h Query heads, 1 Key head, 1 Value head. Highly memory-efficient but causes a slight capacity drop.
- **Grouped-Query Attention (GQA):** h Query heads, k Key-Value heads (where $k = h/g$). The industry standard (e.g., Llama-3, Mistral) balancing quality and memory.

Technical Intuition & Complexity (where applicable)

Consider a model with $h_{\text{query}} = 32$ heads, hidden dimension $d = 4096$, and head dimension $d_k = 128$.

- **MHA KV Cache:** Stores 32 Keys and 32 Values per layer.

- **MQA KV Cache:** Stores 1 Key and 1 Value per layer. A $32\times$ **memory reduction** for the KV Cache.

- **GQA KV Cache** (with group size 8, so $k = 4$ KV heads): Stores 4 Keys and 4 Values per layer. An $8\times$ **memory reduction** for the KV Cache.

The reduction allows production servers to fit larger batch sizes on a single GPU.

Production Perspective & Trade-offs

While MQA reduces VRAM consumption the most, it can degrade accuracy on tasks requiring complex attention retrieval. GQA serves as the optimal trade-off, recovering almost 100% of MHA's representational capacity while achieving near-MQA inference speeds. Most modern LLMs (e.g., Llama-3, Mistral-7B) have adopted GQA as the default attention architecture.

Follow-up Questions

- **Follow-up:** *Does GQA/MQA speed up the prefill phase?*
- **Answer:** Not significantly. The prefill phase is compute-bound, so reducing memory transfers for KV cache writes has a minor impact. The primary benefit of GQA/MQA is during the decode phase, which is memory-bandwidth bound.

Common Mistakes

- **Mistake:** Thinking that GQA reduces the parameter count of the model's FFN layers. GQA only affects the projection matrices of Key and Value vectors ($\mathbf{W}_K, \mathbf{W}_V$) in the attention layers, leaving FFN layers untouched.

Text Generation

Question 36: Explain autoregressive generation.

Short Interview Answer (30–60 seconds)

Autoregressive generation is a sequential text generation process where the model generates one token at a time. At each step, the model takes the entire sequence of previously generated tokens as input, projects it to predict the probability distribution of the next token, samples a token from this distribution, appends it to the history, and repeats the cycle. This loop continues until the model generates a special End-of-Sequence (`<EOS>`) token or hits the maximum length limit.

Key Interview Points

- **Sequential Dependence:** Each generated token depends on all prior tokens: $P(x_t | x_{1:t-1})$
- **Probability Chain Rule:** Mathematically decomposes joint probability: $P(X) = \prod_{i=1}^L P(x_i | x_1, \dots, x_{i-1})$.
- **Sequential Latency Bottleneck:** Inference must run L times to generate a sequence of length L .

Production Perspective & Trade-offs

Autoregressive generation is highly compute-inefficient because we must run the entire Transformer neural network stack once per token. This sequential loop is the primary reason why LLM generation feels slow to users. To bypass this, researchers are developing **Speculative Decoding**, which uses a smaller, faster draft model to generate a chunk of candidate tokens, and runs the large target model once in parallel (prefill-style) to validate and accept the candidate tokens in a single step, boosting generation speeds by $2 - 3\times$.

Follow-up Questions

- **Follow-up:** *What is the difference between causal masking during training and autoregressive generation during inference?*
- **Answer:** During training, we perform **Teacher Forcing**—we feed the entire ground-truth sequence at once and calculate loss for all positions in parallel using causal masking. During inference, we do not have the ground-truth future tokens, so we must generate them one-by-one, feeding the model's own predictions back as inputs.

Common Mistakes

- **Mistake:** Expecting the model to generate the entire sentence in a single GPU step. The model is mathematically defined as a next-token predictor and cannot compute multi-token outputs in a single forward pass without speculative decoding wrappers.

Question 37: Compare Greedy Search, Beam Search, Top-k, Top-p, and Temperature sampling.

Short Interview Answer (30–60 seconds)

These are decoding strategies used to select the next token from the model's predicted probability distribution:

- **Greedy Search** selects the token with the highest probability. It is fast but leads to repetitive, sterile loops.
- **Beam Search** maintains a set of B highest-probability candidate sequences, expanding them in parallel. It is common in translation but repetitive for open-ended generation.
- **Temperature** scales logits to control randomness: $T < 1.0$ sharpens the distribution (more greedy), while $T > 1.0$ flattens it (more creative).
- **Top-k** restricts sampling to the k highest-probability tokens.
- **Top-p (Nucleus)** restricts sampling to the smallest set of tokens whose cumulative probability exceeds p (e.g., $p = 0.9$), adapting the candidate pool size dynamically.

Key Interview Points

- **Greedy:** Deterministic, $k = 1$. High likelihood of repetitive text loops.
- **Beam Search:** Keeps track of multiple paths; computationally expensive due to keeping B parallel decoders active.
- **Temperature (T):** Modulates logit scaling: z_i/T . Does not change ranking, only distribution scale.
- **Top-p (Nucleus):** Dynamic filtering; drops the long tail of low-probability tokens.

Technical Intuition & Complexity (where applicable)

- **Temperature formulation:**

$$P(x_i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

- As $T \rightarrow 0$, the probability of the maximum logit approaches 1.0 (Greedy Search).
- As $T \rightarrow \infty$, the distribution becomes uniform, leading to complete randomness.

Method	Deterministic?	Search Space	Computational Cost	Primary Use Case
Greedy	Yes	1 path	Low	Coding, Math, factual QA
Beam Search	Yes (mostly)	B paths	High ($O(B)$ memory)	Machine Translation
Top-k	No	Fixed k tokens	Low (requires sorting k elements)	Creative writing
Top-p	No	Dynamic (cumulative p)	Medium (requires cumulative sum sort)	General Chat, Assistant

Production Perspective & Trade-offs

Top-p and Top-k sampling introduce sorting operations over the vocabulary logits ($|V| \approx 32k - 128k$). On GPUs, sorting operations can be slow and interrupt parallel streaming. Fused kernels (e.g., flash-attention samplers) optimize this by doing Top-k/Top-p cuts directly on GPU registers, avoiding full sorting overheads.

Follow-up Questions

- **Follow-up:** *Why do we combine Top-k and Top-p?*
- **Answer:** We apply Top-k first to eliminate extremely low-probability tokens and restrict the vocabulary size, and then apply Top-p on the remaining tokens to dynamically adjust the sampling pool based on the distribution's shape, preventing "gibberish" choices.

Common Mistakes

- **Mistake:** Thinking Temperature changes the order of the token probabilities. Temperature is a scalar scaling factor; it does not change the ranking of the logits, only the contrast (variance) between their probabilities.

Question 38: What is Repetition Penalty?

Short Interview Answer (30–60 seconds)

Repetition Penalty is a logit modification technique used during generation to prevent the model from repeating words or phrases. It works by identifying tokens that have already appeared in the generated history and dividing their logits by a penalty factor $\theta > 1.0$ (or multiplying by θ if the logit is negative) before applying Softmax. This lowers the probability of previously generated tokens, forcing the model to select new vocabulary options and break repetitive loops.

Key Interview Points

- **Dynamic Logit Scaling:** Lowers the score of historical tokens dynamically at each generation step.
- **Penalty parameter (θ):** Typically set between 1.0 (no penalty) and 1.2 (strong penalty).
- **Repetitive Loop Breaker:** Essential for preventing greedy decoding from getting stuck in cyclic text loops.

Technical Intuition & Complexity (where applicable)

For a vocabulary of logits $\mathbf{z} \in \mathbb{R}^{|V|}$:

$$z_i = \begin{cases} z_i / \theta & \text{if } z_i \geq 0 \\ z_i & \text{if } z_i < 0 \end{cases}$$
 where θ is the penalty factor. Multiplying by θ when $z_i < 0$ is equivalent to dividing by θ when $z_i \geq 0$.

the logit is negative makes the logit *more negative*, successfully reducing its post-softmax probability.

Production Perspective & Trade-offs

While repetition penalty reduces loops, setting θ too high (e.g., $\theta > 1.3$) can ruin grammar and facts. The model will be penalized for selecting necessary grammatical words (like "the", "and", or punctuation) or correct repeating subject names, causing it to output disjointed sentences. In production, engineers must fine-tune the penalty range carefully.

Follow-up Questions

- **Follow-up:** *What is the difference between Repetition Penalty and Presence Penalty in OpenAI's API?*
- **Answer:** Presence penalty is an additive penalty: $z_i = z_i - \alpha$, which subtracts a flat constant from logits of tokens that have appeared at least once. Repetition penalty is multiplicative (z_i/θ), which scales down the logits proportionally, penalizing highly probable tokens more severely than low-probability ones.

Common Mistakes

- **Mistake:** Thinking repetition penalty is a training loss regularizer. Repetition penalty is applied **strictly at inference time** during the sampling stage, requiring no retraining of the model.
-

Modern LLMs

Question 39: Compare GPT, BERT, T5, and Llama.

Short Interview Answer (30–60 seconds)

GPT, BERT, T5, and Llama represent the key architectural branches of the Transformer family:

- **BERT** is an encoder-only model using bidirectional self-attention. It is trained via Masked Language Modeling (MLM) and is optimized for natural language understanding (NLU).
- **GPT** is a decoder-only model using unidirectional (causally masked) attention. It is trained via next-token prediction and is optimized for autoregressive text generation.
- **T5** is an encoder-decoder model that maps text-to-text. It is trained via span corruption and is optimized for transduction tasks like translation and summarization.
- **Llama** is a modern decoder-only model that improves on GPT by adopting RMSNorm pre-normalization, SwiGLU activation, RoPE positional encoding, and Grouped-Query Attention (GQA).

Key Interview Points

- **BERT**: Encoder-only, bidirectional context, absolute learned positions, NLU focus.
- **GPT**: Decoder-only, causal masking, absolute learned positions, NLG focus.
- **T5**: Encoder-Decoder, cross-attention, relative position bias, sequence-to-sequence focus.
- **Llama**: Modernized decoder-only, RMSNorm, RoPE, GQA, SwiGLU, open-weights pioneer.

Technical Intuition & Complexity (where applicable)

Model	Architecture	Attention Masking	Positional Encoding	Key Innovation
BERT	Encoder-only	Bidirectional (None)	Learned Absolute	Bidirectional representation via MLM
GPT	Decoder-only	Causal (Lower-Triangular)	Learned Absolute	Decoders scale to 175B parameters
T5	Encoder-Decoder	Encoder: None; Decoder: Causal	Relative position bias	Unified text-to-text task formulation
Llama	Decoder-only	Causal (Lower-Triangular)	RoPE (Rotary relative)	pre-normalization, GQA, SwiGLU, RMSNorm

Production Perspective & Trade-offs

In production systems, Decoder-only architectures (GPT, Llama) are highly preferred for general-purpose assistants because prompt instruction-following scales better in decoders. However, for specialized extraction or classification pipelines, BERT remains the production standard because it is much smaller (typically 110M–340M parameters compared to Llama's 8B+ parameters), runs with sub-10ms latency, and does not require complex KV Cache management.

Follow-up Questions

- **Follow-up:** *Why did BERT use absolute learned positional encodings instead of relative ones like RoPE?*
- **Answer:** At the time BERT was developed, absolute learned embeddings were the simplest approach to inject sequence order, and BERT was not intended to extrapolate to long context windows (its maximum sequence length was capped at 512 tokens).

Common Mistakes

- **Mistake:** Thinking BERT is an autoregressive model. BERT cannot generate text efficiently because it lacks causal masking; it is designed to analyze complete, bidirectional input sequences.

Question 40: What innovations did Llama introduce?

Short Interview Answer (30–60 seconds)

Llama modernized the standard Transformer architecture by introducing several key modifications designed to stabilize training and accelerate inference:

1. **Pre-normalization (RMSNorm)**: Normalizes activations *before* the sub-layers instead of after, using RMSNorm to speed up execution and stabilize training.
2. **Rotary Positional Embeddings (RoPE)**: Replaces absolute positional embeddings with relative angular rotations applied to Queries and Keys, enabling context length extension.
3. **SwiGLU Activation**: Replaces standard GELU or ReLU in FFN blocks to improve representation capacity.
4. **Grouped-Query Attention (GQA)**: In Llama-2/3, uses shared Key-Value heads to reduce the KV Cache memory footprint during decoding.

Key Interview Points

- **RMSNorm Pre-normalization**: Moves LayerNorm to sub-layer inputs and drops the mean centering logic.
- **SwiGLU Gated Activation**: Introduces a gating projection in the FFN layer.
- **RoPE relative positions**: Shifts coordinates rotationally to capture distance relationship.
- **GQA implementation**: Groups Query heads to reduce VRAM consumption during inference.

Technical Intuition & Complexity (where applicable)

By moving to **Pre-normalization**:

$$x_l = x_{l-1} + \text{SubLayer}(\text{RMSNorm}(x_{l-1}))$$

This structure ensures that gradients can propagate directly through the main residual trunk $x_L = x_0 + \sum \text{SubLayer}(\cdot)$ without being altered by normalization layers, avoiding the training instability (vanishing gradients) that plagued deep Post-LN architectures.

Production Perspective & Trade-offs

Llama's design is heavily optimized for GPU serving. RMSNorm reduces the amount of memory access required for normalization. GQA enables high-throughput serving by allowing the model to fit larger batch sizes in VRAM during inference decoding. The trade-off is architectural complexity: implementing Llama from scratch in PyTorch requires tracking separate intermediate dimensions for SwiGLU ($d_{ffn} \approx \frac{8}{3}d$) and group indexes for GQA KV heads.

Follow-up Questions

- **Follow-up**: *Why did Llama switch from Post-LN to Pre-LN?*

- **Answer:** Post-LN models place the normalization layer on the residual path, which causes the gradient scale to decrease exponentially as we backpropagate to early layers. This requires highly sensitive learning rate warmups. Pre-LN keeps the residual path clean, allowing stable training of very deep models.

Common Mistakes

- **Mistake:** Saying Llama has fewer parameters than standard Transformers because of GQA. GQA only reduces the *activation footprint* (the KV Cache size in memory) and a small portion of KV projection weights; it does not change the core FFN parameter scale.
-

Question 41: What innovations did Mistral and Mixtral introduce?

Short Interview Answer (30–60 seconds)

Mistral and Mixtral introduced key innovations to optimize attention limits and compute budgets:

- **Mistral-7B** introduced **Sliding Window Attention (SWA)**, which limits the attention span of a token to a fixed window W (e.g., 4096 tokens). Tokens can still access information further back due to the receptive field stacking across layers, reducing attention complexity to $O(L \cdot W)$ instead of $O(L^2)$.
- **Mixtral-8x7B** introduced a **Sparse Mixture of Experts (MoE)** architecture. It replaces the dense FFN block with 8 independent "expert" FFN layers. A gating network routes each token to the top 2 experts at each layer. This provides a 46.7B parameter model where only 12.9B parameters are active per token, yielding GPT-3.5 capability with fast, sparse compute times.

Key Interview Points

- **Sliding Window Attention (SWA):** Caps attention block complexity at $O(L \cdot W)$, scaling linearly for long contexts.
- **Rolling Buffer Cache:** Dynamically overwrites past Keys and Values in VRAM once they fall outside the sliding window.
- **Sparse Mixture of Experts (MoE):** Routes tokens to a subset of FFN layers dynamically.
- **Routing Gating Network:** Softmax-based router that selects Top-2 experts per token.

Technical Intuition & Complexity (where applicable)

- **SWA Receptive Field:** If each layer has a sliding window of size W , the effective receptive field at layer d is $d \times W$. At layer 32, a token can access context up to $32 \times 4096 = 131,072$ tokens back through multi-layer information propagation.
- **Mixtral Expert Selection:**

$$y = \sum_{i \in \text{Top2}} G(x)_i \cdot \text{Expert}_i(x), \quad \text{where } G(x) = \text{Softmax}(\text{Top2}(x\mathbf{W}_g, \epsilon))$$

Here, $G(x)_i$ is the routing score for expert i , and $\mathbf{W}_g$ is the learnable routing projection matrix.

Production Perspective & Trade-offs

- **VRAM vs. Compute:** Mixtral-8x7B has 46.7B parameters, meaning it requires at least 96GB of VRAM to load in FP16. However, it only performs the FLOPs of a 12.9B parameter model per token. This makes it highly compute-efficient (low latency per token) but memory-expensive to host compared to equivalent-quality dense models.
- **MoE Routing Flaws:** If the router is poorly trained, it can over-route to a single expert (hot spotting), bottlenecking GPU parallelization. Modern servers use **Expert Parallelism** to split different experts across different GPUs.

Follow-up Questions

- **Follow-up:** *How does a Rolling Buffer Cache work in SWA?*
- **Answer:** Because a token at position i only attends to keys in range $[i - W, i]$, keys for positions $< i - W$ are never read. The memory cache can be allocated as a fixed circular array of size W . New Keys and Values overwrite old ones at index $i \pmod{W}$, keeping VRAM usage constant regardless of total generation length.

Common Mistakes

- **Mistake:** Thinking Mixtral-8x7B is a simple ensemble of 8 separate model checkpoints. Only the FFN layers are split into experts; the self-attention, layer norms, and embedding layers are **shared** across all experts, which is why the total parameter count is 46.7B instead of $8 \times 7B = 56B$.
-

Question 42: What innovations does DeepSeek introduce?

Short Interview Answer (30–60 seconds)

DeepSeek introduced two major architectural innovations to optimize KV Cache and MoE training:

1. **Multi-head Latent Attention (MLA):** MLA performs low-rank joint compression on Key and Value vectors, compressing the KV Cache into a small latent vector of size $d_k \approx 128$. This reduces the KV Cache VRAM footprint by up to 93% during decoding compared to standard Multi-Head Attention, allowing for massive batch sizes.
2. **DeepSeekMoE:** Unlike standard MoE (which splits into equal coarse experts), DeepSeekMoE splits experts into fine-grained experts (allowing finer routing) and allocates a set of **Shared Experts** that are always active. This prevents redundant knowledge storage across experts and stabilizes routing.
3. **Multi-Token Prediction (MTP):** Trains the model to predict multiple tokens in a single step, improving training representation and generation speed.

Key Interview Points

- **Multi-head Latent Attention (MLA):** Key-Value low-rank compression.
- **Shared Experts:** Always-active FFN blocks to capture common knowledge, avoiding expert overlap.
- **Fine-Grained Experts:** Splitting FFNs into smaller chunks to allow more flexible routing paths.
- **KV Cache Compression:** Overcomes the GQA/MQA memory bottleneck during high-concurrency serving.

Technical Intuition & Complexity (where applicable)

In MLA, instead of caching the full Key and Value matrices, the model compresses them into a low-rank latent vector $\mathbf{c}_t^{KV}$:

$$\mathbf{c}_t^{KV} = \mathbf{x}_t \mathbf{W}^{DKV}$$

where $\mathbf{W}^{DKV} \in \mathbb{R}^{d \times d_c}$ and $d_c \ll d$ (e.g., $d_c = 512$ while $d = 5120$).

During decoding, we only cache the compressed latent vector $\mathbf{c}_t^{KV}$ and a low-dimensional positional key vector. During the attention step, the Keys and Values are reconstructed on-the-fly using the cached latent vector, saving significant VRAM.

Production Perspective & Trade-offs

DeepSeek's innovations are directly targeted at lowering production API costs. By reducing the KV Cache size by 93% via MLA, DeepSeek can process much larger batch sizes on a single GPU node. This drastically increases throughput (tokens/sec/GPU), which translates directly to lower service prices. The trade-off is the extra matrix multiplications needed to compress and reconstruct the Keys and Values on-the-fly, which require highly optimized custom CUDA kernels.

Follow-up Questions

- **Follow-up:** *Why are shared experts useful in DeepSeekMoE?*
- **Answer:** In standard MoEs, general knowledge (like basic grammar rules) must be learned by multiple experts, wasting capacity. Shared experts act as a baseline FFN that handles general context, allowing the routed experts to specialize strictly in domain-specific tasks, which improves parameter efficiency.

Common Mistakes

- **Mistake:** Thinking MLA increases the context length capacity of the model automatically. MLA resolves the *memory* footprint of the KV Cache, but the model's maximum sequence length is still bounded by the trained positional encoding limits (like RoPE scaling).
-

Question 43: Explain Chinchilla Scaling Laws.

Short Interview Answer (30–60 seconds)

The Chinchilla Scaling Laws (formulated by DeepMind in 2022) prove that for compute-optimal training, the model parameters (N) and the number of training tokens (D) should be scaled in **equal proportions**: $D \approx 20 \cdot N$. Prior to this study, models like GPT-3 (175B parameters trained on 300B tokens) were severely **under-trained** relative to their size. For a given compute budget, it is more efficient to train a smaller model on more tokens than a larger model on fewer tokens. This shift led to smaller, high-performance models (e.g., Llama-7B trained on 1.4T+ tokens).

Key Interview Points

- **Compute-Optimal**: Balances FLOPs between model parameters and training dataset size.
- **Chinchilla Ratio**: $D \approx 20 \cdot N$ tokens per parameter (for FP16 training).
- **Compute budget formula**: $C \approx 6ND$ FLOPs.
- **Over-training Trend**: For production serving, models are trained far beyond the Chinchilla limit (e.g., $140\times$) to reduce downstream inference costs.

Technical Intuition & Complexity (where applicable)

The compute budget C required to train a Transformer model is approximately:

$$C \approx 6ND \text{ FLOPs}$$

- **Forward Pass**: $2ND$ FLOPs (2 floating-point operations per parameter per token: 1 multiply, 1 add).
- **Backward Pass**: $4ND$ FLOPs ($2\times$ the cost of the forward pass to compute gradients with respect to activations and weights).

DeepMind modeled the loss $\mathcal{L}(N, D)$ as a power-law function:

$$\mathcal{L}(N, D) = \frac{A}{N^\alpha} + \frac{B}{D^\beta} + E$$

By optimizing this function subject to a constant compute constraint $C = 6ND$, they derived that the scaling exponent for both N and D is roughly equal, leading to the optimal ratio $D \propto N$.

Production Perspective & Trade-offs

While $D \approx 20N$ is compute-optimal for *training*, it is not optimal for **inference serving**. A larger model is more expensive to serve because every inference token costs $2N$ FLOPs. If a model is going to be queried billions of times in production, it is much cheaper to train a smaller model (e.g., Llama-3-8B) on a massive dataset (e.g., 15T tokens, which is $1875\times$ its parameter count), intentionally over-training it past the Chinchilla limit to save massive inference compute costs over its lifetime.

Follow-up Questions

- **Follow-up:** *Why does the backward pass cost exactly $4N$ FLOPs per token while the forward pass costs $2N$?*
- **Answer:** The forward pass performs one matrix multiplication ($2N$ operations) to project activations. The backward pass must compute two gradients: first, the gradient of the loss with respect to the activations of the layer (to propagate gradients back), which costs $2N$ FLOPs; second, the gradient of the loss with respect to the weights of the layer (to update the parameters), which costs another $2N$ FLOPs.

Common Mistakes

- **Mistake:** Believing that Chinchilla Scaling Laws represent a hard limit on how many tokens a model can learn from. It is a **compute-optimal** allocation curve for a *fixed training budget*, not a convergence ceiling.
-

Question 44: What is Mixture of Experts (MoE)?

Short Interview Answer (30–60 seconds)

Mixture of Experts (MoE) is a sparse neural network architecture that scales model capacity without scaling computational cost. In a Transformer block, the standard dense FFN layer is replaced with a **router** and a set of parallel FFN "experts". For each token, the router computes gating weights and sends the token strictly to the top- k experts (typically $k = 2$). Output is a weighted sum of the selected experts' outputs. This allows the model to have hundreds of billions of total parameters while keeping the active compute (FLOPs) per token equivalent to a much smaller dense model.

Key Interview Points

- **Conditional Computation:** Only activates a subset of the network parameters per token.
- **FFN Replacement:** MoE replaces FFN layers; self-attention layers remain dense and shared.
- **Top- k Routing:** Router selects the best expert(s) dynamically at runtime.
- **Active vs. Total Parameters:** Total parameters represent capacity; active parameters determine training/inference FLOPs.

Technical Intuition & Complexity (where applicable)

For an input token representation x :

$$y = \sum_{i=1}^E G(x)_i \cdot \text{Expert}_i(x)$$

where:

$$G(x) = \text{Softmax}(\text{KeepTopK}(x\mathbf{W}_g + H(x), k))$$

- $\mathbf{W}_g \in \mathbb{R}^{d \times E}$ is the routing weight matrix.
- $H(x)$ is a noise term added during training to encourage expert exploration.
- $\text{KeepTopK}(\cdot, k)$ sets all values except the top k values to $-\infty$.

The active computational complexity for the FFN step becomes $O(k \cdot L \cdot d \cdot d_{\text{ffn}})$ instead of $O(E \cdot L \cdot d \cdot d_{\text{ffn}})$.

Production Perspective & Trade-offs

- **Inference VRAM footprint:** Although MoE models are fast to compute because only k experts are active, the **entire model** (all experts) must reside in GPU memory. For an 8-expert model like Mixtral, this requires massive VRAM, increasing hardware requirements.
- **Communication Overhead:** In distributed hosting (Expert Parallelism), different experts are placed on different GPUs. When routing tokens, the system must transfer token vectors across GPUs (All-to-All communication), which can bottleneck inference if the network interconnect (e.g., NVLink) is slow.

Follow-up Questions

- **Follow-up:** *What is the "expert capacity" problem during MoE training?*
- **Answer:** During training, if the router favors a specific expert, that expert gets overloaded with tokens, while other experts receive none. This causes training bottlenecks and underutilizes parameters. To solve this, we implement a **load-balancing loss** to penalize uneven routing.

Common Mistakes

- **Mistake:** Thinking MoE reduces the VRAM requirement of a model. MoE reduces the **FLOP count (compute)** per token, but the **VRAM requirement (memory)** actually increases because all experts must remain loaded in GPU memory to prevent latency spikes.
-

Question 45: Dense vs Sparse Models.

Short Interview Answer (30–60 seconds)

- **Dense Models** (e.g., Llama-3, GPT-3) activate 100% of their parameters for every single token processed. They are highly compute-intensive but have a smaller memory footprint for a given parameter capacity.
- **Sparse Models** (e.g., Mixtral, DeepSeek-V3) use conditional computation (like MoE) to activate only a small fraction of their parameters per token. They achieve high representational capacity with low compute latency, but require massive VRAM to keep all inactive parameters loaded in GPU memory.

Key Interview Points

- **Parameter Activation:** Dense activates 100% per token; Sparse activates a fraction (e.g., 10 – 25%).
- **FLOP Efficiency:** Sparse models require significantly fewer FLOPS per token for the same parameter capacity.
- **VRAM Constraint:** Sparse models are memory-bound (high VRAM requirement); Dense models are compute-bound.

Technical Intuition & Complexity (where applicable)

Let N be the total parameter count of a model.

- **Dense Model:**

- Inference compute cost: $\approx 2N$ FLOPs per token.

- Active Parameters: N .

- **Sparse MoE Model (with E experts, top- k routing):**

- Total Parameters: $N_{\text{total}} = N_{\text{shared}} + E \times N_{\text{expert}}$.

- Active Parameters: $N_{\text{active}} = N_{\text{shared}} + k \times N_{\text{expert}}$.

- Inference compute cost: $\approx 2N_{\text{active}}$ FLOPs per token.

- Thus, if $E = 16$ and $k = 2$, $N_{\text{active}} \ll N_{\text{total}}$, achieving the intelligence of a model of scale N_{total} using the speed of N_{active} .

Production Perspective & Trade-offs

In production hosting:

- **Dense models** are easier to parallelize and scale because there are no routing operations or dynamic load-balancing constraints.

- **Sparse models** are ideal for high-throughput serving systems because their lower FLOP count per token dramatically reduces token generation latency. However, because they are massive in size, they require specialized routing pipelines, multi-GPU partitioning, and high VRAM cluster configurations.

Follow-up Questions

- **Follow-up:** *Why do we not run expert routing on the Self-Attention layers in sparse models?*
- **Answer:** Self-attention routing is highly unstable because attention requires global key-value matching across the entire sequence. Restricting keys/values to specific experts breaks the unified representational alignment, leading to severe degradation in sequence modeling capabilities.

Common Mistakes

- **Mistake:** Thinking sparse models are faster than dense models of the *same active size*. A sparse model with 12B active parameters has the same computational latency as a 12B dense model (plus a small routing overhead). It is faster than a dense model of the *same total capacity* (e.g., 47B parameters).

LLM Limitations

Question 46: Why do LLMs hallucinate?

Short Interview Answer (30–60 seconds)

LLMs hallucinate because they are trained strictly as next-token predictors using a **Maximum Likelihood Estimation (MLE)** objective, rather than factual reasoners. The model learns to predict the most statistically probable continuation of a prompt based on training correlations, not to verify statements against a ground-truth database. This is worsened by **Exposure Bias** (discrepancy between teacher-forced training and autoregressive inference), out-of-distribution prompts, and the fact that attention vectors can dilute or align incorrectly over long context scopes.

Key Interview Points

- **Statistical Correlation vs. Factuality:** MLE objective optimizes for probability, not truth.
- **Exposure Bias:** Training uses ground-truth context (Teacher Forcing); inference relies on the model's own generated errors.
- **Out-of-Distribution (OOD):** Model extrapolates poorly when prompted with domains missing from training data.
- **Attention Over-generalization:** Softmax distributes attention weight across all tokens, occasionally mixing unrelated facts.

Production Perspective & Trade-offs

To mitigate hallucinations in production, engineers do not rely on raw generation. Instead, they implement **RAG** (Retrieval-Augmented Generation) to inject verified document context into the prompt, constrain

outputs using structured schemas (e.g., JSON mode), and run factual validation guardrails (like LlamaGuard or self-reflection routers) at the output stage. The trade-off is latency and API cost: RAG increases prompt token length, which quadratically scales prefill processing time.

Follow-up Questions

- **Follow-up:** *What is Exposure Bias and how does it relate to hallucination accumulation?*
- **Answer:** During training, the model always receives correct past tokens (Teacher Forcing). During inference, the model generates its own past. If it makes a small semantic error at step t , it is forced to condition on this error at step $t + 1$. Since it was never trained to recover from its own errors, the hallucinations accumulate exponentially as the sequence gets longer.

Common Mistakes

- **Mistake:** Thinking that fine-tuning (SFT/RLHF) completely eliminates hallucinations. Fine-tuning only aligns the model's tone and encourages it to say "I don't know," but it does not change the core next-token prediction objective or add factual retrieval capabilities.
-

Question 47: Why do LLMs struggle with very long contexts?

Short Interview Answer (30–60 seconds)

LLMs struggle with long contexts due to three major limitations:

1. **Positional Decay:** As sequences exceed the length of the trained positional encodings, the rotary or relative frequencies extrapolate poorly, causing attention scores to degrade.
2. **Attention Dilution:** The Softmax function distributes weights across all tokens. In very long sequences, the attention weight assigned to the relevant target token gets diluted by thousands of filler tokens, reducing retrieval accuracy.
3. **Hardware Constraints:** The $O(L^2)$ memory and computational cost of self-attention leads to KV Cache overflow, bottlenecking GPU memory and causing latency spikes.

Key Interview Points

- **Positional Extrapolation Failure:** Out-of-distribution distance indices degrade attention calculations.
- **Softmax Dilution:** Attention weights approach 0 for critical facts when distributed over massive contexts.
- **Memory Bandwidth Bottleneck:** Gigabytes of KV Cache data must be loaded from HBM at each step, slowing down decoding.

Production Perspective & Trade-offs

In production systems, processing 100k+ tokens requires techniques like **RoPE Frequency Scaling** (NTK-aware interpolation) to adjust positional frequencies, and **FlashAttention** to fit attention in GPU SRAM. However, even if the model fits in VRAM, the business trade-off is cost: a single query with 100k prompt tokens costs significant API compute and can block GPU queues for several seconds during the prefill phase, reducing concurrent user capacity.

Follow-up Questions

- **Follow-up:** *What is NTK-aware RoPE scaling?*
- **Answer:** Instead of scaling all RoPE frequencies uniformly (which washes out local details), NTK-aware scaling scales high-frequency components less and low-frequency components more. This preserves short-range ordering while allowing the model to extrapolate to long distances.

Common Mistakes

- **Mistake:** Thinking that if a model has a 128k context window, it will retrieve information from the middle of the document with 100% accuracy. Empirically, recall degrades significantly in the middle of long contexts.
-

Question 48: What are the major limitations of current LLMs?

Short Interview Answer (30–60 seconds)

The major limitations of current LLMs are:

- **Lack of Factual Grounding:** They predict probable words, leading to hallucinations.
- **High Computational and Memory Costs:** Quadratic attention scaling ($O(L^2)$) and sequential autoregressive decoding make inference expensive and slow.
- **Static Knowledge:** They cannot update their internal parameters in real-time, requiring expensive retraining or fine-tuning to acquire new information.
- **Poor Reasoning and Planning:** They perform well on pattern matching but struggle with multi-step logical reasoning, math, and long-term planning without external agentic loops.

Key Interview Points

- **Autoregressive Latency:** Memory-bandwidth bottleneck during generation.
- **Static Weights:** Knowledge is frozen at the cutoff date.
- **System 1 vs. System 2:** LLMs are fast pattern-matchers (System 1) but struggle with deliberative planning (System 2) natively.
- **Vulnerability:** Vulnerable to jailbreaks, prompt injection, and data poisoning.

Production Perspective & Trade-offs

To overcome these limits in production, AI architectures surround the LLM with an **Agentic Framework**. Instead of asking the model to solve a complex math problem directly, the agent parses the prompt, calls a Python execution tool, runs the calculation, and feeds the correct output back to the LLM. This shifts the burden of reasoning from the static neural weights to dynamic external tools, trading off execution latency for factual correctness.

Follow-up Questions

- **Follow-up:** *How does Retrieval-Augmented Generation (RAG) address the static knowledge limitation?*
- **Answer:** RAG bypasses the frozen weights limit by retrieving up-to-date documents from a vector database at query time and injecting them directly into the LLM's context window, allowing the model to answer questions using real-time information without needing retraining.

Common Mistakes

- **Mistake:** Believing that scaling parameter counts alone will resolve all LLM limitations. Scaling improves pattern-matching capacity, but fundamental issues like hallucinations, sequential inference bottlenecks, and out-of-distribution extrapolation require structural architectural changes.
-

Production & System Design

Question 49: Walk through the complete LLM inference pipeline from prompt to generated response.

Short Interview Answer (30–60 seconds)

The LLM inference pipeline consists of four main stages:

1. **Tokenization (CPU):** The input text prompt is converted into integer token IDs using a vocabulary lookup.
2. **Prefill Phase (GPU):** The model processes the input token IDs in parallel, generates Query, Key, and Value vectors, populates the KV Cache, and outputs the logit distribution for the first token. This is **compute-bound** (GEMM).
3. **Decode Loop (GPU):** The model generates tokens one-by-one autoregressively. At each step, it projects the single new token, retrieves past Keys and Values from the KV Cache, computes attention, projects logits, samples the next token (using Top-p, Temperature, etc.), and appends it to the KV Cache. This is **memory-bandwidth-bound** (GEMV).
4. **Detokenization (CPU):** The generated token IDs are converted back into text characters and streamed to the user.

Key Interview Points

- **Tokenization**: Pre-processing step mapping characters to integers.
- **Prefill (Compute-Bound)**: Parallel execution initializing the KV Cache (high SM utilization).
- **Decode Loop (Memory-Bound)**: Sequential token-by-token generation (low SM utilization).
- **Sampling/Logit Warping**: Modifying output logits using Temperature, Top-p, or repetition penalties.
- **Detokenization**: Post-processing step converting integers back to text.

Production Perspective & Trade-offs

In high-throughput production environments, executing this pipeline sequentially for one user at a time is highly cost-inefficient because the Decode loop leaves the GPU compute cores idle. Servers implement **Continuous Batching** and **PagedAttention** to group the decode steps of hundreds of active users together, maximizing GPU memory bandwidth utilization and scaling throughput.

Follow-up Questions

- **Follow-up**: *At what point in the pipeline are repetition penalties and temperature applied?*
- **Answer**: They are applied during the **logit warping** stage in the decode loop, after the final linear projection layer outputs the raw logits, but *before* the Softmax operation and sampling step.

Common Mistakes

- **Mistake**: Forgetting that tokenization and detokenization run on the CPU. If the tokenizer implementation is slow (e.g., poorly written regex), it can bottleneck the entire serving pipeline, causing high latency even with a fast GPU.
-

Question 50: How would you optimize an LLM for lower latency and lower inference cost?

Short Interview Answer (30–60 seconds)

To optimize an LLM for production serving, I would target memory bandwidth and VRAM constraints using five key strategies:

1. **Weight Quantization**: Convert weights to FP8 or INT4 to reduce model size, cutting memory transfer times and VRAM usage.
2. **KV Cache Optimization**: Implement **PagedAttention** to eliminate memory fragmentation, and leverage models with **GQA** (Grouped-Query Attention) to reduce cache size.
3. **Continuous Batching**: Group different requests at the iteration level instead of request level to maximize GPU utilization.
4. **FlashAttention**: Use fused attention kernels to execute Softmax on-chip, minimizing slow global

memory transfers.

5. **Speculative Decoding**: Use a small draft model to generate candidate tokens in parallel, validating them in a single target model step to accelerate generation.

Key Interview Points

- **Quantization (FP8/INT4)**: Reduces memory transfer volume, speeding up the memory-bound decode loop.
- **PagedAttention**: Resolves the KV Cache VRAM allocation bottleneck.
- **Continuous Batching**: Maximizes arithmetic intensity on GPU.
- **Speculative Decoding**: Bypasses the sequential decode loop bottleneck.
- **Tensor/Pipeline Parallelism**: Splits models across multiple GPUs to balance memory limits.

Technical Intuition & Complexity (where applicable)

Consider the effect of **Quantization** on Decode phase latency:

- In the decode phase, the time to process a step is dominated by loading the model weights:

$$\text{Time per step} \approx \frac{\text{Model Parameters (Bytes)}}{\text{GPU Memory Bandwidth (Bytes/sec)}}$$

- For a 70B parameter model in FP16 (140 GB), served on an A100 GPU (2000 GB/sec bandwidth):

$$\text{Latency} \approx \frac{140 \times 10^9}{2000 \times 10^9} = 0.07 \text{ seconds} = 70 \text{ ms/token} \approx 14 \text{ tokens/sec}$$

- If we quantize the model to 4-bit (35 GB):

$$\text{Latency} \approx \frac{35 \times 10^9}{2000 \times 10^9} = 0.0175 \text{ seconds} = 17.5 \text{ ms/token} \approx 57 \text{ tokens/sec}$$

Quantization directly yields a $4\times$ **speedup** in generation throughput by reducing the memory-bandwidth load.

Production Perspective & Trade-offs

Quantization introduces a trade-off between speed and accuracy. 4-bit quantization reduces model size by $4\times$ but can introduce quantization noise, causing the model to lose accuracy on complex reasoning or math tasks. To mitigate this, teams use advanced quantization algorithms like **AWQ** (Activation-aware Weight Quantization) or **GPTQ** that protect salient weight channels (often just 1% of weights) to maintain baseline performance.

Follow-up Questions

- **Follow-up:** *What is the difference between Tensor Parallelism and Pipeline Parallelism?*
- **Answer:** Tensor Parallelism splits individual weight matrices *intra-layer* across multiple GPUs in parallel (requiring high-speed NVLink connections). Pipeline Parallelism splits the layers of the model *inter-layer* sequentially across GPUs (where GPU 2 waits for GPU 1 to finish), requiring load-balancing schedules (like Megatron-LM) to avoid GPU bubbles.

Common Mistakes

- **Mistake:** Suggesting upgrading the GPU compute speed (TFLOPs) as the primary way to solve slow single-user generation. Single-user decode generation is memory-bandwidth bound, not compute-bound. Buying a card with more compute cores but the same memory bandwidth will yield zero speedup; you must increase memory bandwidth (e.g., H100 with HBM3).
-

Interview Questions & Production Trade-offs

- **What problem does this solve?**
Consolidates 50 critical, high-impact interview questions and answers on LLM Foundations, linking theoretical math (attention, rotations, scaling) to systems bottlenecks (KV Cache scaling, bandwidth-bounds, quantization).
- **Why was it introduced?**
To provide a structured preparation pipeline for technical screenings and systems design rounds in Applied AI, GenAI, and ML Engineering interviews.
- **What are its limitations?**
This guide focuses on core LLM foundations (attention mechanics, tokenization, architectures, scaling, and inference latency). It does not cover downstream fine-tuning, RAG database architectures, multi-agent frameworks, or guardrail observability modules (which are detailed in subsequent modules).
- **Computational Complexity (Time & Memory)**
 - Reference lookup time complexity: $O(1)$ constant time.
 - VRAM memory complexity: $O(1)$ constant storage.
- **Component Variable Denotation Legend**
 - L : Sequence token length.
 - d : Model hidden dimension.
 - d_k : Attention head dimension (d/h).
 - h : Number of Query attention heads.
 - b : Batch size.

- n_{layers} : Total number of Transformer layers.
- $n_{\text{heads}_{\text{kv}}}$: Number of Key-Value attention heads.
- s_{len} : Total context sequence length.
- N : Total model parameters.
- D : Number of tokens in training dataset.
- C : Floating-point compute FLOP budget.
- E : Total number of experts in a Sparse Mixture of Experts (MoE) layer.
- k : Number of active experts routed per token.
- W : Attention sliding window size.

- θ : Multiplicative repetition penalty factor.

- **Production Use Cases**

- Designing scale-up configurations for low-latency inference endpoints.
- Estimating cluster VRAM requirements for scaling concurrent user traffic.
- Justifying architectural design decisions (e.g., electing GQA or MLA) during technical design reviews.

- **Follow-up questions interviewers ask**

- *How would you determine if a model's slow generation is compute-bound or memory-bandwidth bound in practice?*
 - **Answer:** By profiling the system's memory and compute bandwidth utilization. If the GPU's memory bandwidth is near maximum (e.g., $> 85\%$ HBM bandwidth utilization) while SM active occupancy (TFLOPs utilization) is extremely low (e.g., $< 15\%$), the generation is memory-bandwidth bound (typical of the decode phase).
- *Describe the mathematical difference in gradient propagation between Pre-LN and Post-LN architectures.*
 - **Answer:** In Pre-LN, the gradient has an additive term that bypasses normalization layers, making gradient scales near constant across layers. In Post-LN, gradients are scaled by the inverse of normalization terms at each step, causing exponential decay or growth.